

Visual Analytics – Clustering

PD Dr. Monique Meuschke

Slides based on material provided by Prof. Dr. Kai Lawonn

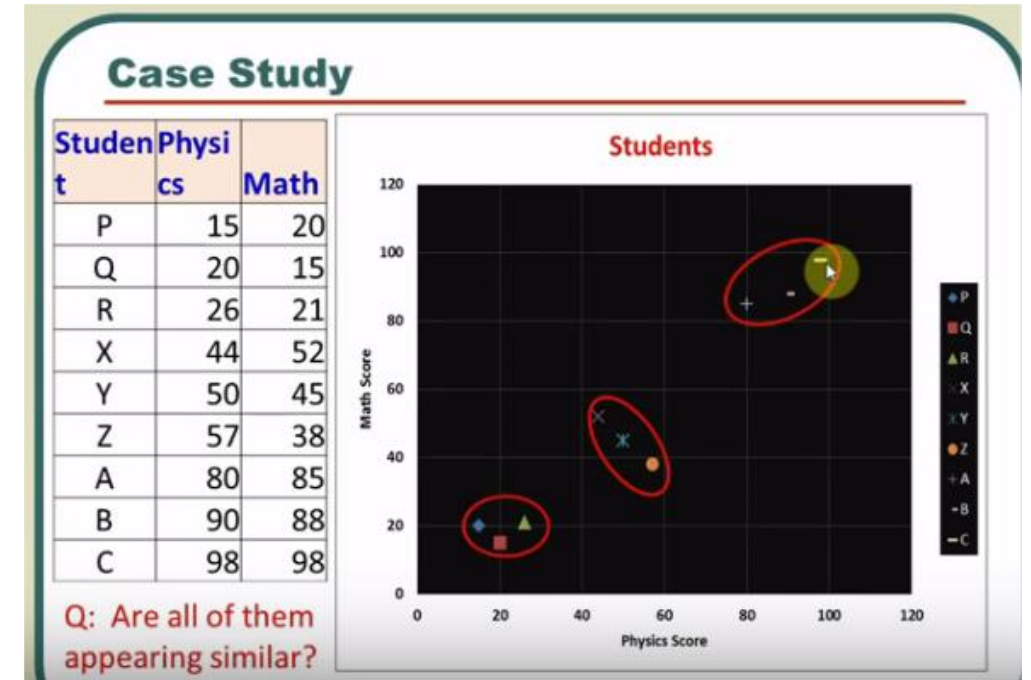
Outline

- Introduction
- Clustering Methods
 - K-Means
 - Optics
 - Density-Based
 - Agglomerative Hierarchical
 - Clustering Ensembles
- Clustering Categorical and Mixed Data
- Multiple Clusterings
- Temporal Clustering

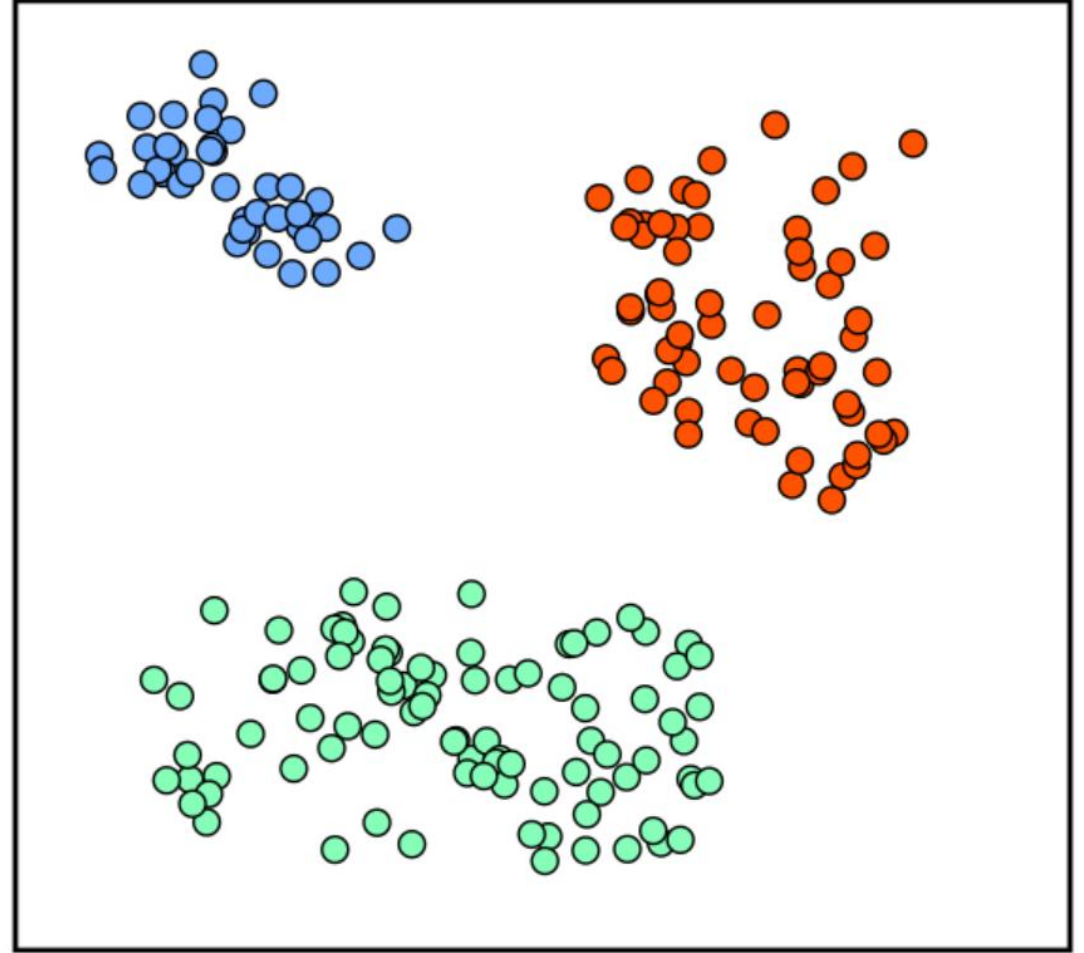
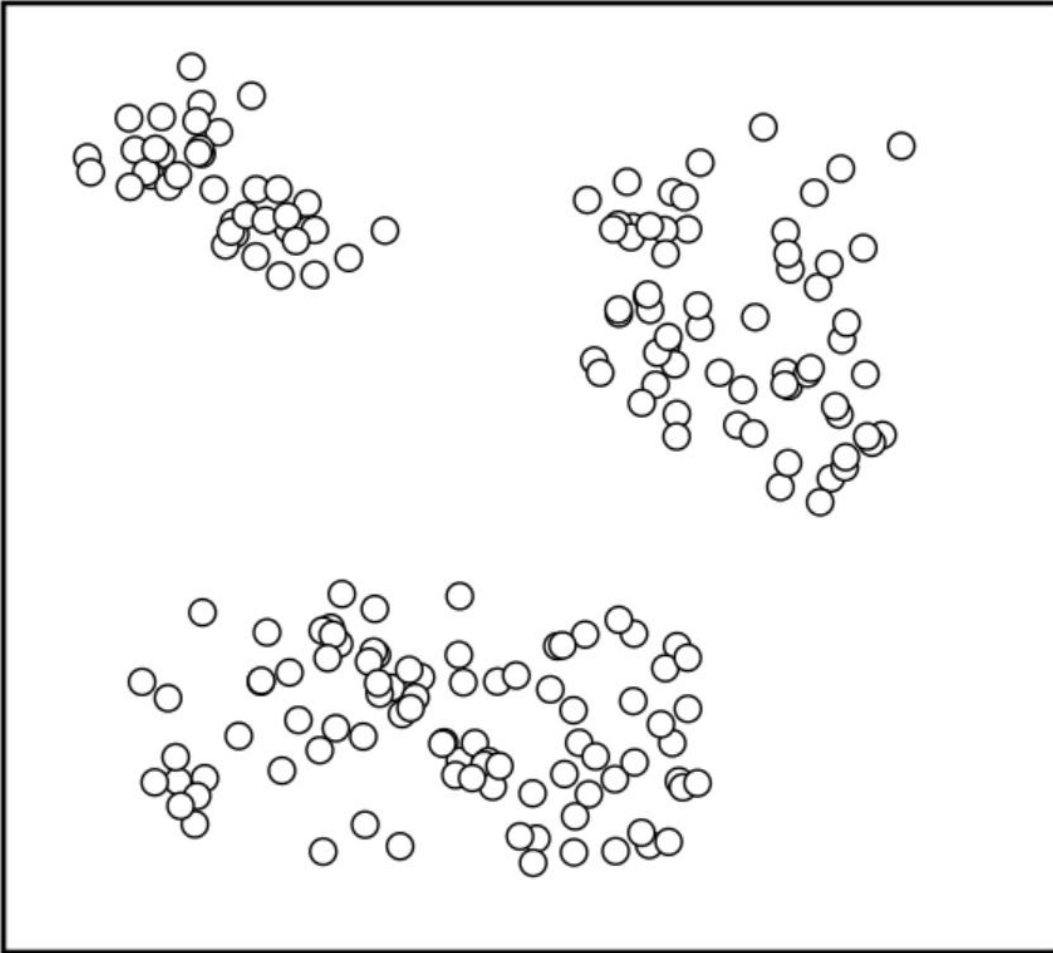
Introduction

Clustering

- part of an exploratory data analysis (users have little a priori information).
- serves to *define groups automatically* (unsupervised learning).



Introduction



Introduction

- Members of a cluster are closer to each other than to members of other clusters.
 - Definition of proximity based on a distance measure that may incorporate different weights for the attributes.
- A cluster *summarizes homogeneous regions*.
- A cluster may contain subclusters – the understanding of this hierarchical relation is essential.

Introduction

Relation between *clustering* and *visual analytics*?

- „Since data organization and presentation are among the objectives of cluster analysis, graphic tools to display the results of clustering are important. They should let one „see“ whether the grouping of the data has brought out any latent structure ...“ (Ling, 1973)

Introduction: Motivation

- Learn the structure of data, e.g., define subgroups for customer segmentation (business analytics)
- Preprocess for selective visualization (show only certain clusters) or for focus-context visualization (show cluster representatives as overview and all instances as detailed view)
- Preprocess for classification, e.g., as input for a decision tree search

Introduction: Motivation

- Learn the structure of data, e.g., define subgroups for customer segmentation (business analytics)
- Preprocess for selective visualization (show only certain clusters) or for focus-context visualization (show cluster representatives as overview and all instances as detailed view)
- Preprocess for classification, e.g., as input for a decision tree search

Two (related) tasks:

- Clustering to efficiently partition a dataset
- Displaying and exploring clustering results to interpret partitioning and support reasoning

Introduction

- Clustering algorithms maximizes *intercluster coherence*
- When distribution unknown → clustering *with different methods and parameters* until results are plausible.
- Selection of a clustering method based on assumptions about the data and a *clustering model*
- The cluster model relates to
 - the distribution of data,
 - the expected shapes of clusters and
 - their relations.

Introduction

According to the model, clustering is performed

- In a hierarchic or non-hierarchic manner
- In a fuzzy or binary manner (hard)
- In a deterministic or non-deterministic manner
- Using various *distance measures*

Introduction

According to the model, clustering is performed

- In a hierarchic or non-hierarchic manner
- In a fuzzy or binary manner (hard)
- In a deterministic or non-deterministic manner
- Using various *distance measures*

Clustering results in a decomposition of the data elements.

Outliers (not belonging to any cluster) are possible with some approaches

For further analysis and visualization it is often useful to determine a *cluster representative*

Introduction

Clustering is applied

- to low and high-dimensional data,
- to documents (based on words),
- to scalar and categorical data or a mixture of both,
- to image data, such as CT and MRI,
- to streamlines (for supporting flow visualization),
- to fiber tracts (from brain MRI data) and
- even to time-dependent data where elements are clustered according to temporal changes.

If applied to image data, clustering is similar to segmentation.

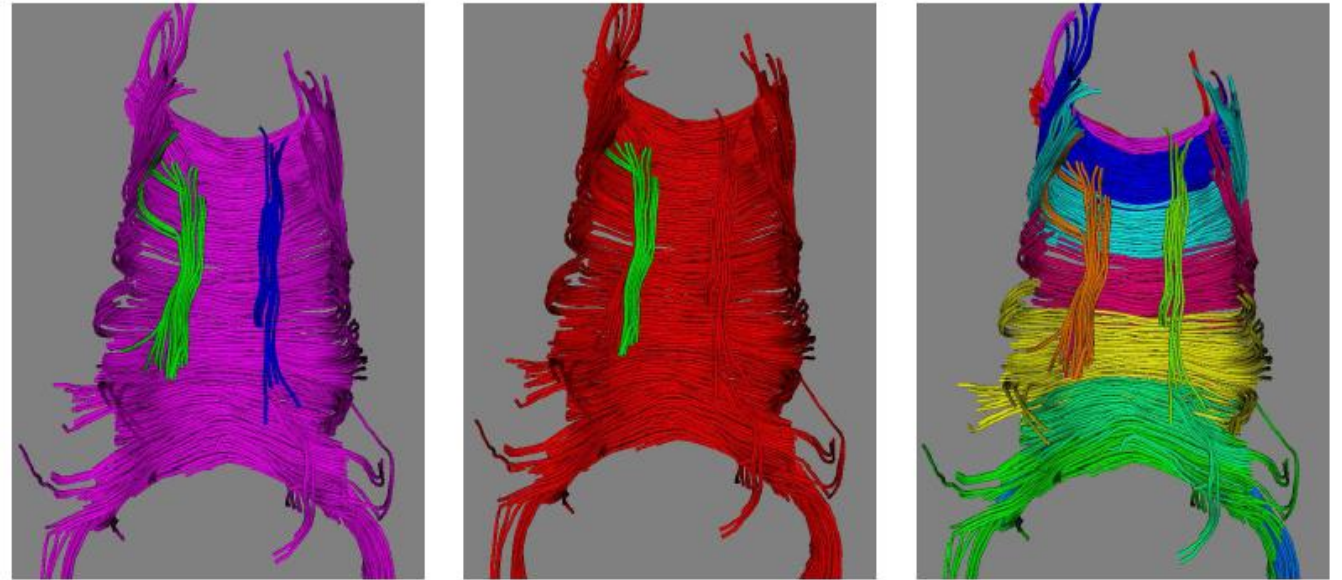
Introduction

Different results of clustering of fiber tract data to identify crucial fiber tracts (Moberts, 2005).

Left: correct, middle: incorrect, right: incomplete.

It is easier to interactively merge clusters (right image) than to separate an erroneously connected cluster.

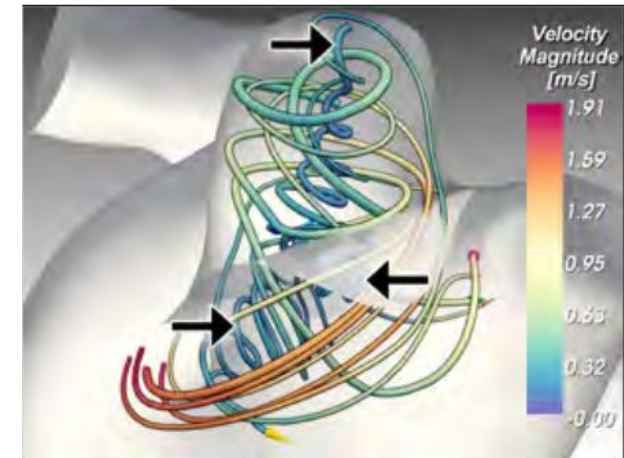
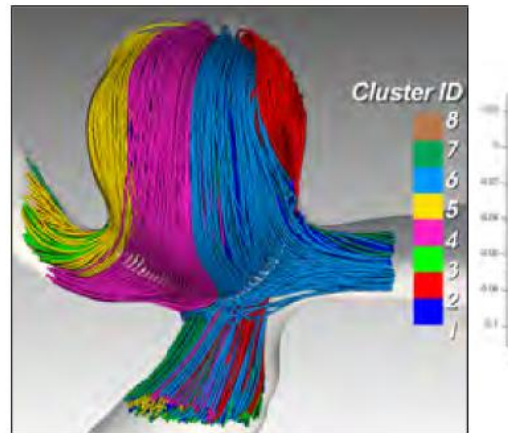
What is an appropriate distance measure?



Introduction

Clustering of streamlines to support the visual analysis of blood flow in aneurysms.

Left: full set of streamlines color-coded according to cluster-id. Right: (different dataset): Per cluster one cluster representative was determined (Oeltze, 2014)



Clustering Methods

- Choice of clustering methods strongly influences results
 - K-Means clustering favors Voronoi-cell-shaped clusters
 - Density-based clusters enables arbitrary shapes
- Even experienced users try different clustering methods and parameters
 - Quality assessment to support this explorative process.

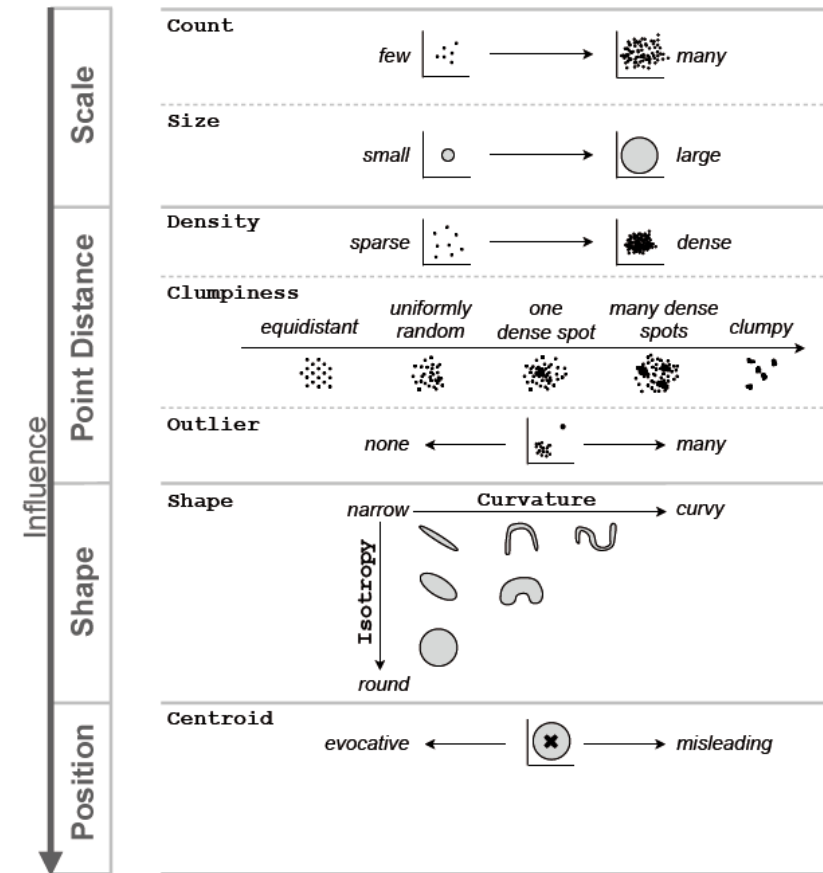
Clustering Methods

- Classification of Clustering problems
- Requirements
- Methods:
 - K-Means
 - Optics
 - Density-Based
 - Agglomerative Hierarchical
 - Clustering Ensembles

Clustering Methods: Classification

- Clusters discriminated according to 4 categories of within class features.
Features are not orthogonal, e.g., cluster shape and centroid mutually influence each other

(From: Sedlmaier, 2012)

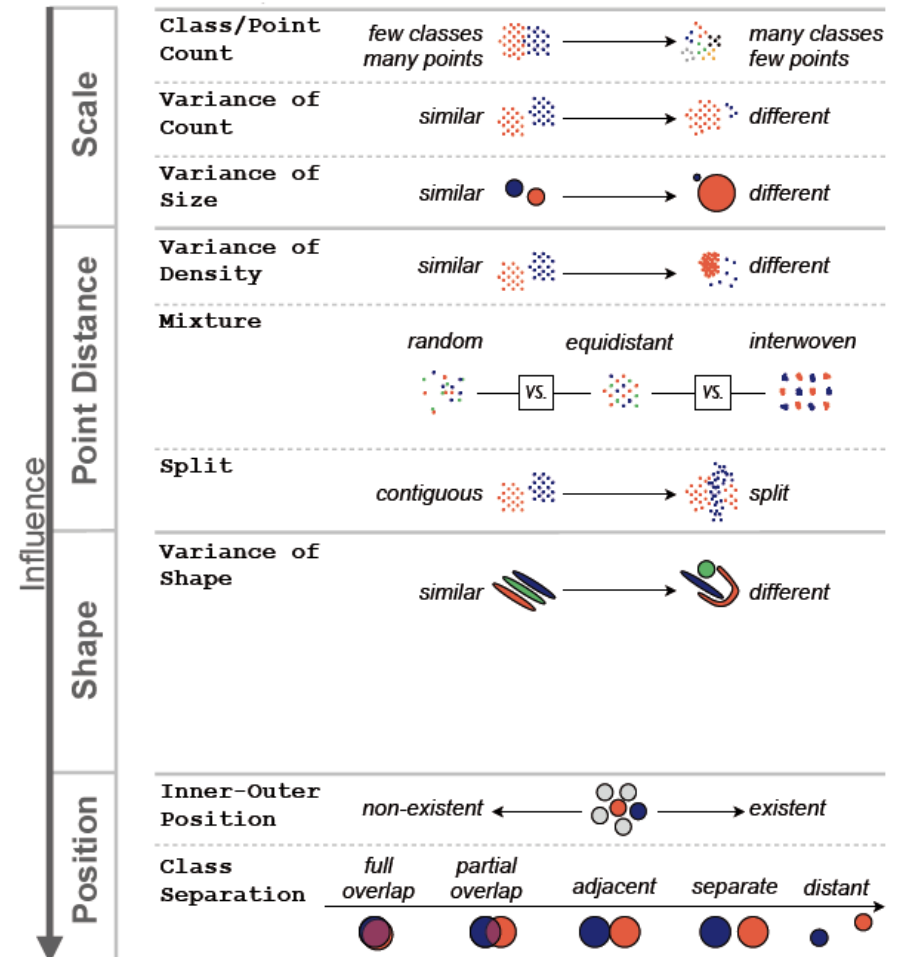


Clustering Methods: Classification

Why we need so many clustering methods (II)?

Because the relation between clusters are different.

- Clusters discriminated according to four categories of between class features. „Split“ - essential feature (whether or not clusters are spatially coherent) (From: Sedlmaier, 2012)



Clustering Methods

Results:

- Hard clustering: Each object is assigned to a cluster (or considered as outlier)
- Fuzzy Clustering: assignment of degree of membership
- Per cluster, a *representative* may be generated
- Prototype generating clustering: Generates hull surfaces, spherical hulls, regression lines

Clustering Methods: Requirements

Ideally clustering methods

- Are scalable to many objects and many dimensions
- May create clusters of arbitrary shapes
- Are robust against noise and outliers
- Create plausible results, even for regions with different amounts of density

Clustering Methods: Models

Clustering based on

- A distance model (objects belong to a *cluster_i* if they are closer to a *cluster center_i* than to any other *cluster center_j*) – k-Means and variants
- A density model (objects belong to a cluster if the density in the surrounding is high compared to the average density) – DBSCAN, Optics
- A hierarchy model where clusters are assumed to exist at different levels

K-Means

Cluster Analysis

- Partitioning of the N objects
- Every object is assigned a characteristic vector $\mathbf{m}_1, \mathbf{m}_2, \dots, \mathbf{m}_N$
- The goal is to find cluster $C_1, C_2, \dots, C_k \subseteq \{1, 2, \dots, N\}$ such that

$$\bigcup_{i=1}^k C_i = \{1, \dots, N\}$$
$$C_i \cap C_j = \emptyset, \quad i \neq j$$

Cluster Analysis

$$\bigcup_{i=1}^k C_i = \{\mathbf{p}_1, \dots, \mathbf{p}_N\}$$
$$C_i \cap C_j = \emptyset, \quad i \neq j$$

- One objective (square error criterion):
 - The variance of the clusters should be minimal

$$E = \sum_{i=1}^k \sum_{j \in C_i} \|\mathbf{m}_j - \bar{\mathbf{m}}_i\|^2$$

where $\bar{\mathbf{m}}_i$ is the characteristic mean vector of the cluster i

Cluster Analysis

$$\bigcup_{i=1}^k C_i = \{\mathbf{p}_1, \dots, \mathbf{p}_N\}$$
$$C_i \cap C_j = \emptyset, \quad i \neq j$$

- The number of possible partitions of N points in k clusters is given by the Stirling numbers of the second kind:

$$S(N, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^N$$

- With $N \gg k$:

$$S(N, k) \approx \frac{k^N}{k!}$$

Cluster Analysis

$$\bigcup_{i=1}^k C_i = \{\mathbf{p}_1, \dots, \mathbf{p}_N\}$$
$$C_i \cap C_j = \emptyset, \quad i \neq j$$

- If we do not know the number of cluster:

$$B(N) = \sum_{k=1}^N S(N, k)$$

- Challenging to find the optimal cluster ($B(50) \approx 1,8 \cdot 10^{47}$)

Cluster Analysis

$$\bigcup_{i=1}^k C_i = \{\mathbf{p}_1, \dots, \mathbf{p}_N\}$$
$$C_i \cap C_j = \emptyset, \quad i \neq j$$

- Iterative partitioning clustering (k-means)
- The most common used algorithm is Lloyd's algorithm

Cluster Analysis

Lloyd's algorithm:

Input: image I , initial set of k-means $\bar{\mathbf{m}}_1^{(1)}, \dots, \bar{\mathbf{m}}_k^{(1)}$

repeat

if $\left\| \mathbf{m} - \bar{\mathbf{m}}_s^{(i)} \right\| == \min_{j=1, \dots, k} \left\| \mathbf{m} - \bar{\mathbf{m}}_j^{(i)} \right\|$

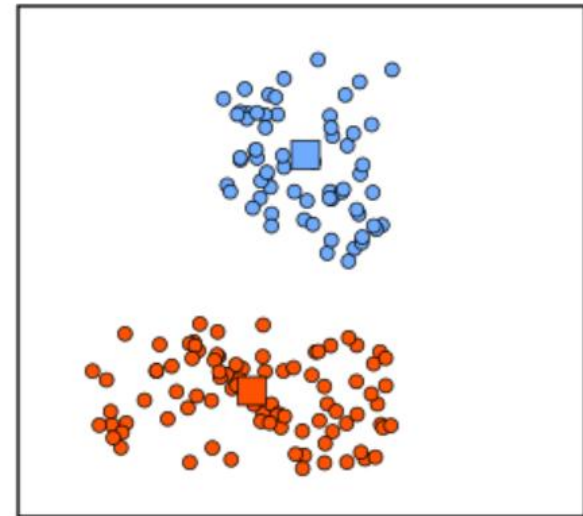
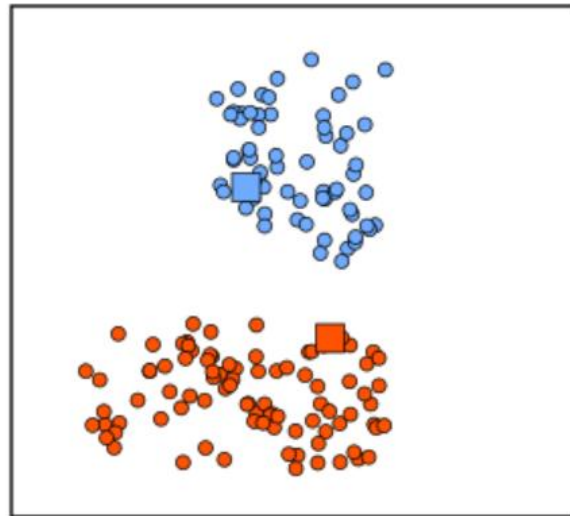
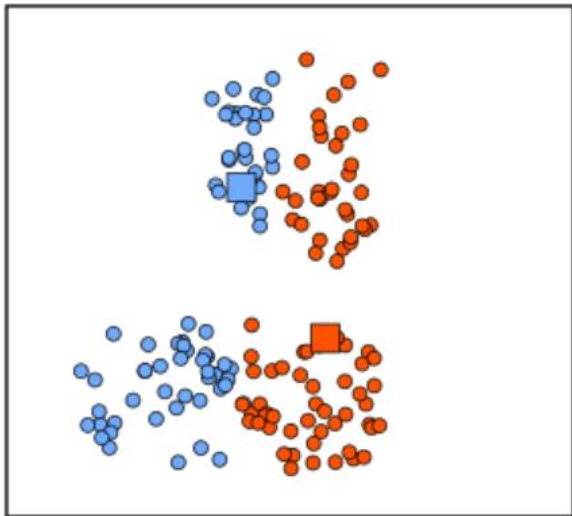
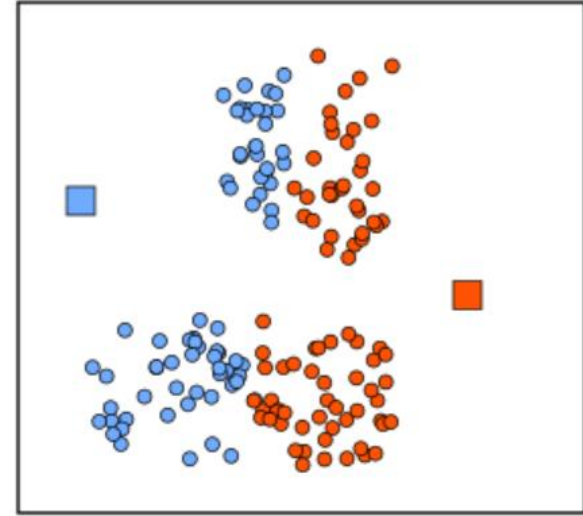
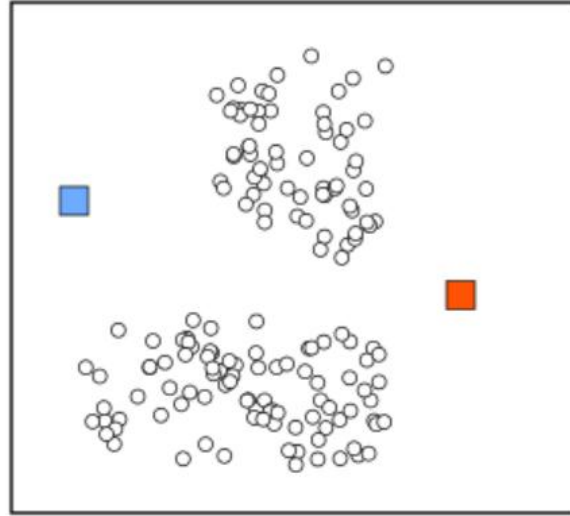
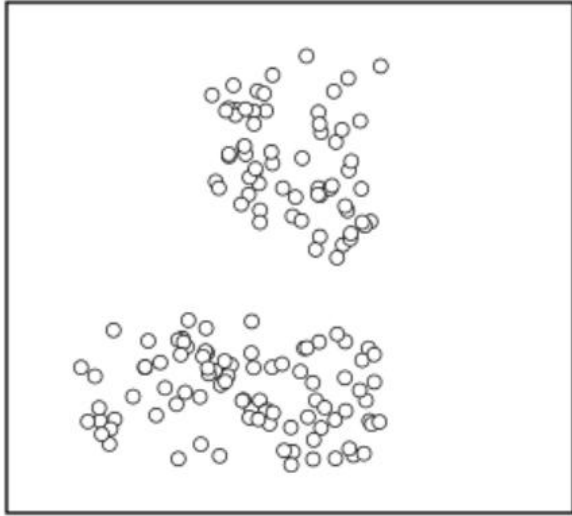
Assign $\mathbf{m}$ to the cluster C_s

Calculate the new means: $\bar{\mathbf{m}}_j^{(i+1)} = \frac{1}{|C_j|} \sum_{i \in C_j} \mathbf{m}_i$

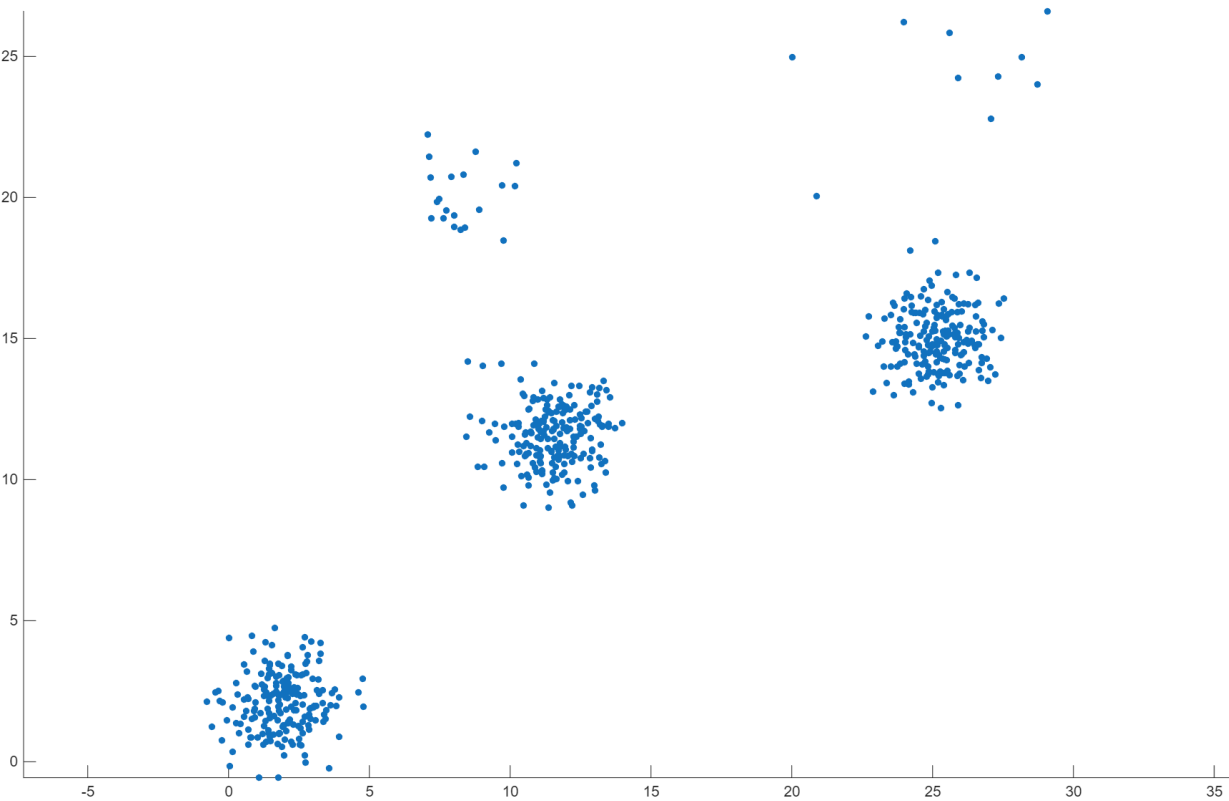
until $E^{(i)}$ minimal

Output: Cluster C

Cluster Analysis



Example



Cluster Analysis

Properties:

- Euclidean distance metric: only for numerical data.
- Clusters are always convex.
- Low robustness against outliers
- k must be known in advance (with cluster validity measures, k may be estimated, but that is time-consuming)
 - A priori knowledge of k is not typical for exploratory data analysis.
 - If k is too small, clustering groups the wrong elements.
 - If k is too high, clustering separates elements that belong together.
- Clusters should have a similar size (boundaries are in the middle between cluster centers).
- Result strongly depends on the initialization (not deterministic).

Clustering Methods: k-means

Improvements:

- Better performance with
 - Hierarchical data structures such as the k-d tree or
 - Better start points to improve convergence (k-Means++, k-Medoids)
- Extend results to fuzzy membership (*fuzzy c-means*). Membership is maximum for the cluster center and minimum at the borders (Bezdek, 1981)
 - Often used in image segmentation, e.g., brain data with gray matter, white matter, cerebrospinal fluid
 - Similar properties like k-means

Fuzzy C-Means

Fuzzy C-Means

- Fuzzy clustering is a clustering, in which each data point can belong to more than one cluster
- Given are N objects, the algorithm returns:
 - a list of c cluster centers: $C = \{\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_c\}$
 - a partition matrix: $U = (u)_{i,j} \in [0, 1], \quad i = 1, \dots, N, \quad j = 1, \dots, c$

Fuzzy C-Means

- Fuzzy clustering is a clustering, in which each data point can belong to more than one cluster
- Given are N objects, the algorithm returns:
 - a list of c cluster centers: $C = \{\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_c\}$
 - a partition matrix: $U = (u)_{i,j} \in [0, 1], \quad i = 1, \dots, N, \quad j = 1, \dots, c$
- Minimize:

$$J_m = \sum_{i=1}^N \sum_{j=1}^c u_{i,j}^m \|\mathbf{x}_i - \mathbf{c}_j\|^2$$

Fuzzy C-Means

$$J_m = \sum_{i=1}^N \sum_{j=1}^c u_{i,j}^m \|\mathbf{x}_i - \mathbf{c}_j\|^2$$

- Here, $m > 1$ is the ,fuzzifier‘ that determines the level of cluster fuzziness
- Large values results in smaller membership values
- In the limit $m = 1$ the weights are either 0 or 1 resulting in a crisp clustering

Fuzzy C-Means

$$J_m = \sum_{i=1}^N \sum_{j=1}^c u_{i,j}^m \|\mathbf{x}_i - \mathbf{c}_j\|^2$$

- The optimization has two constraints:

1. $\sum_{i=1}^N u_{i,j} > 0, \quad \text{for all } j$
2. $\sum_{j=1}^c u_{i,j} = 1, \quad \text{for all } i$

Fuzzy C-Means

- Setting $F_k(\lambda, \mathbf{u}_k)$ by using Lagrange multiplier yields:

$$F_k(\lambda, \mathbf{u}_k) = \sum_{j=1}^c u_{k,j}^m \|\mathbf{x}_k - \mathbf{c}_j\|^2 - \lambda \left(\sum_{j=1}^c u_{k,j} - 1 \right)$$

- By differentiating $F_k(\lambda, \mathbf{u}_k)$, can derive update rules for membership values and cluster centers

$$J_m = \sum_{i=1}^N \sum_{j=1}^c u_{i,j}^m \|\mathbf{x}_i - \mathbf{c}_j\|^2$$

1. $\sum_{i=1}^N u_{i,j} > 0$, for all j
2. $\sum_{j=1}^c u_{i,j} = 1$, for all i

Fuzzy C-Means

$$F_k(\lambda, \mathbf{u}_k) = \sum_{j=1}^c u_{k,j}^m \|\mathbf{x}_k - \mathbf{c}_j\|^2 - \lambda \left(\sum_{j=1}^c u_{k,j} - 1 \right)$$

- Setting this gradient equal to zero yields:

$$\frac{\partial F_k(\lambda, \mathbf{u}_k)}{\partial \lambda} = \left(\sum_{j=1}^c u_{k,j} - 1 \right) = 0$$

$$\frac{\partial F_k(\lambda, \mathbf{c}_k)}{\partial u_{s,t}} = m u_{s,t}^{m-1} \|\mathbf{x}_s - \mathbf{c}_t\|^2 - \lambda = 0$$

Fuzzy C-Means

$$F_k(\lambda, \mathbf{u}_k) = \sum_{j=1}^c u_{k,j}^m \|\mathbf{x}_k - \mathbf{c}_j\|^2 - \lambda \left(\sum_{j=1}^c u_{k,j} - 1 \right)$$

- Setting this gradient equal to zero yields:

$$\frac{\partial F_k(\lambda, \mathbf{u}_k)}{\partial \lambda} = \left(\sum_{j=1}^c u_{k,j} - 1 \right) = 0 \rightarrow \sum_{j=1}^c u_{k,j} = 1$$

$$\frac{\partial F_k(\lambda, \mathbf{c}_k)}{\partial u_{s,t}} = m u_{s,t}^{m-1} \|\mathbf{x}_s - \mathbf{c}_t\|^2 - \lambda = 0 \rightarrow u_{s,t} = \left(\frac{\lambda}{m \|\mathbf{x}_s - \mathbf{c}_t\|^2} \right)^{\frac{1}{m-1}}$$

Fuzzy C-Means

$$\frac{\partial F_k(\lambda, \mathbf{u}_k)}{\partial \lambda} = \left(\sum_{j=1}^c u_{k,j} - 1 \right) = 0 \rightarrow \sum_{j=1}^c u_{k,j} = 1$$

$$\frac{\partial F_k(\lambda, \mathbf{c}_k)}{\partial u_{s,t}} = m u_{s,t}^{m-1} \|\mathbf{x}_s - \mathbf{c}_t\|^2 - \lambda = 0 \rightarrow u_{s,t} = \left(\frac{\lambda}{m \|\mathbf{x}_s - \mathbf{c}_t\|^2} \right)^{\frac{1}{m-1}}$$

- Setting this gradient equal to zero yields:

$$\begin{aligned} 1 = \sum_{j=1}^c u_{k,j} &= \sum_{j=1}^c \left(\frac{\lambda}{m \|\mathbf{x}_k - \mathbf{c}_j\|^2} \right)^{\frac{1}{m-1}} \\ &= \left(\frac{\lambda}{m} \right)^{\frac{1}{m-1}} \sum_{j=1}^c \left(\frac{1}{\|\mathbf{x}_k - \mathbf{c}_j\|^2} \right)^{\frac{1}{m-1}} \\ \left(\frac{\lambda}{m} \right)^{\frac{1}{m-1}} &= \left[\sum_{j=1}^c \left(\frac{1}{\|\mathbf{x}_k - \mathbf{c}_j\|^2} \right)^{\frac{1}{m-1}} \right]^{-1} \end{aligned}$$

Fuzzy C-Means

$$\sum_{j=1}^c u_{k,j} = \left(\frac{\lambda}{m}\right)^{\frac{1}{m-1}} \sum_{j=1}^c \left(\frac{1}{\|\mathbf{x}_k - \mathbf{c}_j\|^2}\right)^{\frac{1}{m-1}}$$

- Setting this gradient equal to zero yields:

$$\left(\frac{\lambda}{m}\right)^{\frac{1}{m-1}} = \left[\sum_{j=1}^c \left(\frac{1}{\|\mathbf{x}_k - \mathbf{c}_j\|^2}\right)^{\frac{1}{m-1}} \right]^{-1}, \quad u_{s,t} = \left(\frac{\lambda}{m\|\mathbf{x}_s - \mathbf{c}_t\|^2}\right)^{\frac{1}{m-1}}$$

$$u_{s,t} = \left[\sum_{k=1}^c \left(\frac{\|\mathbf{x}_s - \mathbf{c}_t\|}{\|\mathbf{x}_k - \mathbf{c}_j\|}\right)^{\frac{2}{m-1}} \right]^{-1}$$

Fuzzy C-Means

- Next, we want to optimize $\mathbf{c}_j$ (again setting the gradient to zero):

$$J_m = \sum_{i=1}^N \sum_{j=1}^c u_{i,j}^m \|\mathbf{x}_i - \mathbf{c}_j\|^2$$

$$\frac{\partial J_m}{\partial \mathbf{c}_s} = 2 \sum_{i=1}^N u_{i,s}^m (\mathbf{x}_i - \mathbf{c}_s) = 0$$

$$\mathbf{c}_s = \frac{\sum_{i=1}^N u_{i,s}^m \mathbf{x}_i}{\sum_{i=1}^N u_{i,s}^m}$$

Fuzzy C-Means

$$J_m = \sum_{i=1}^N \sum_{j=1}^c u_{i,j}^m \|\mathbf{x}_i - \mathbf{c}_j\|^2$$

- Finally, we get:

$$\mathbf{c}_j = \frac{\sum_{i=1}^N u_{i,j}^m \mathbf{x}_i}{\sum_{i=1}^N u_{i,j}^m}$$

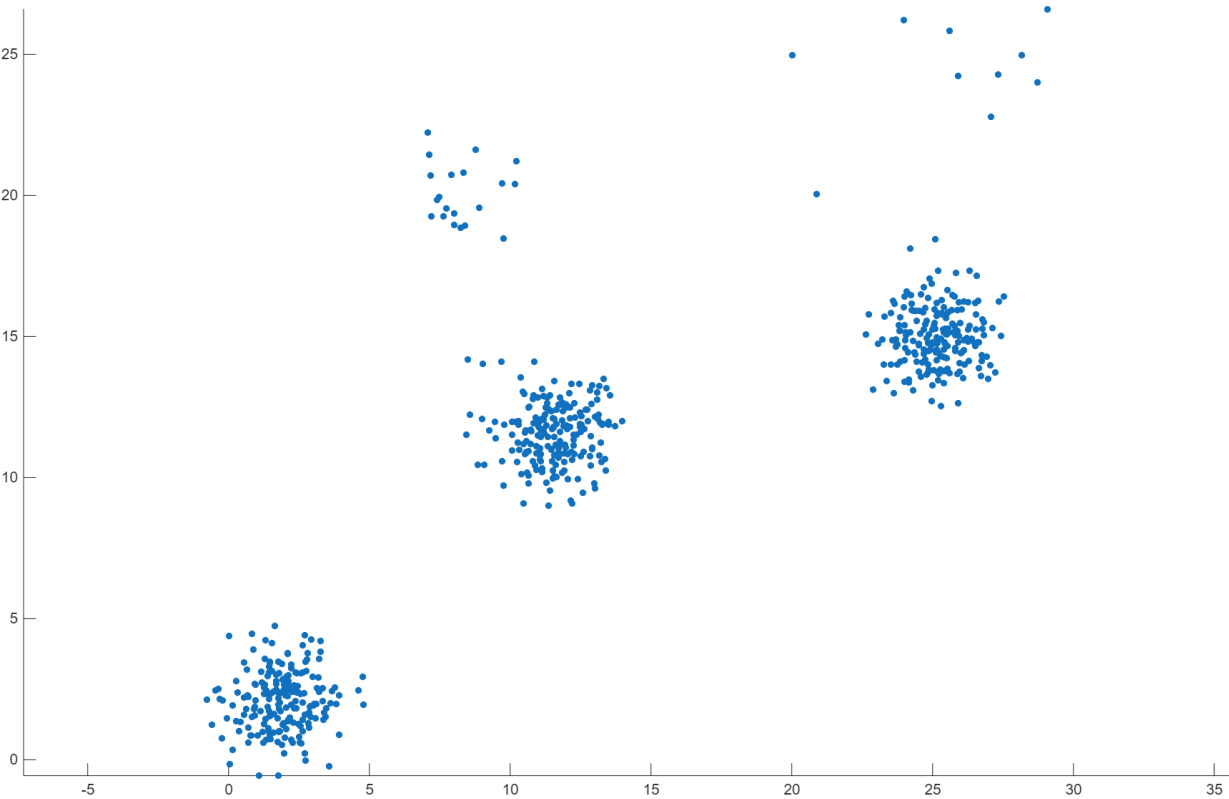
$$u_{i,j} = \frac{1}{\sum_{k=1}^c \left(\frac{\|\mathbf{x}_i - \mathbf{c}_j\|}{\|\mathbf{x}_i - \mathbf{c}_k\|} \right)^{\frac{2}{m-1}}}$$

Fuzzy C-Means

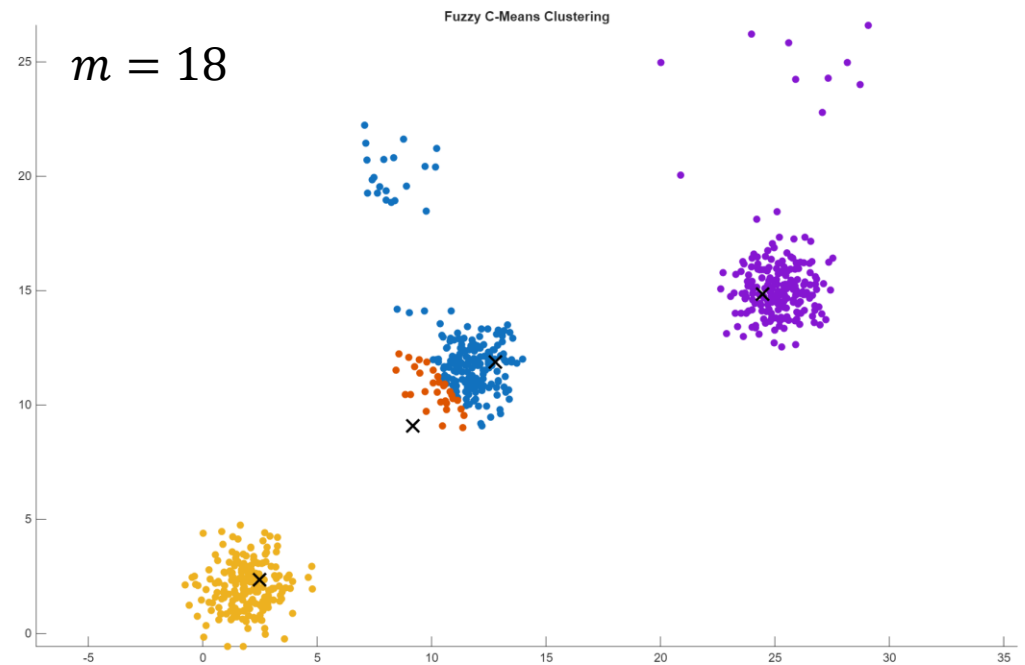
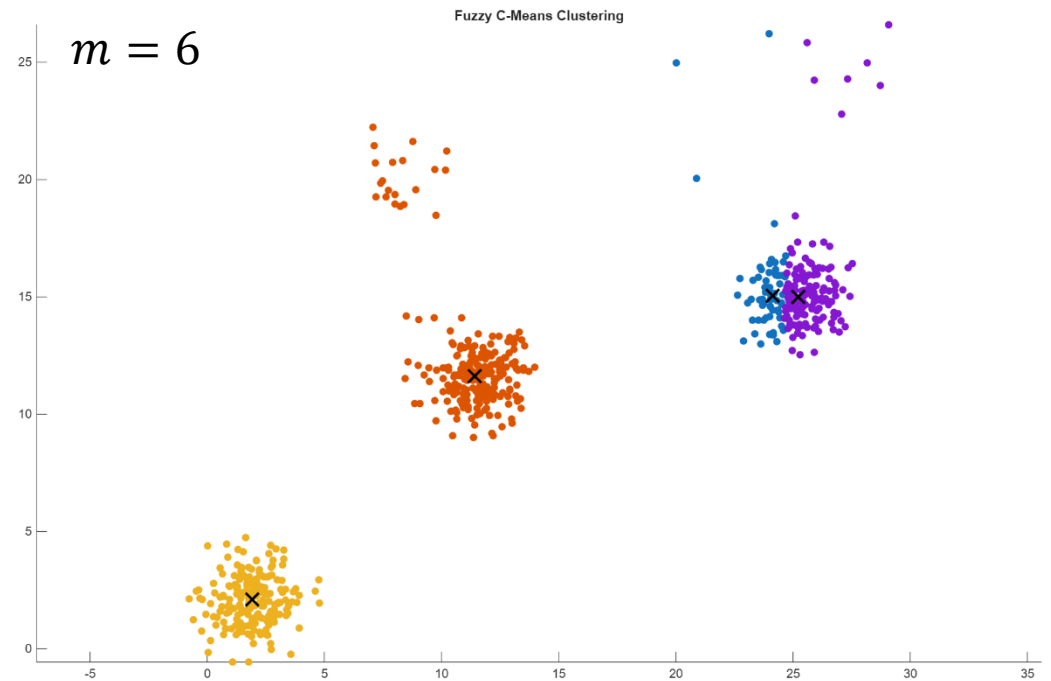
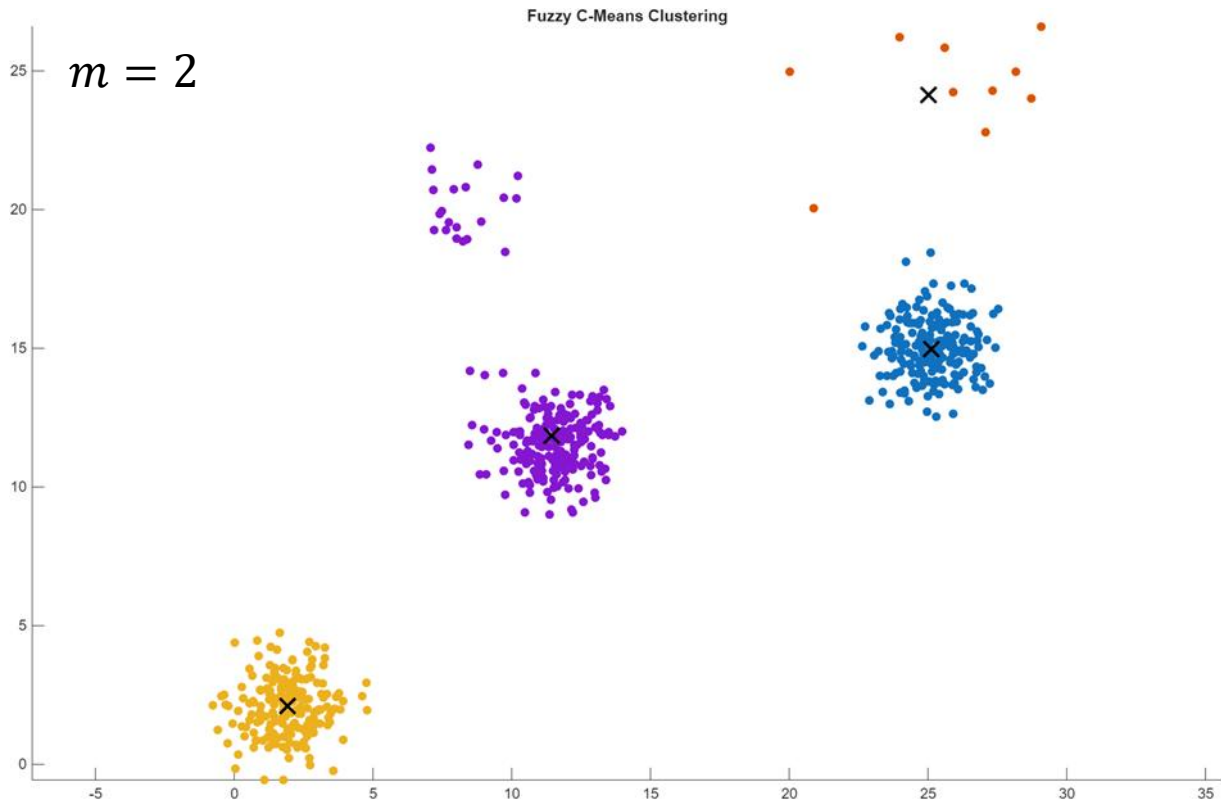
Very similar to k-means:

- Choose a number of clusters
- Assign coefficients randomly to each data point for being in the clusters
- Repeat until the algorithm has converged (coefficients' change between two iterations is no more than ε) :
 - Compute the centroid for each cluster
 - For each data point, compute its coefficients of being in the clusters

Example



Parameter – m



DBSCAN

Density-Based

- *Density-based clustering* searches for clusters that are characterized by an above-average local density
- Density-based methods:
 - Not a priori number of expected clusters
 - Generate clusters of *arbitrary shapes*.
- Essential examples: OPTICS and DBSCAN

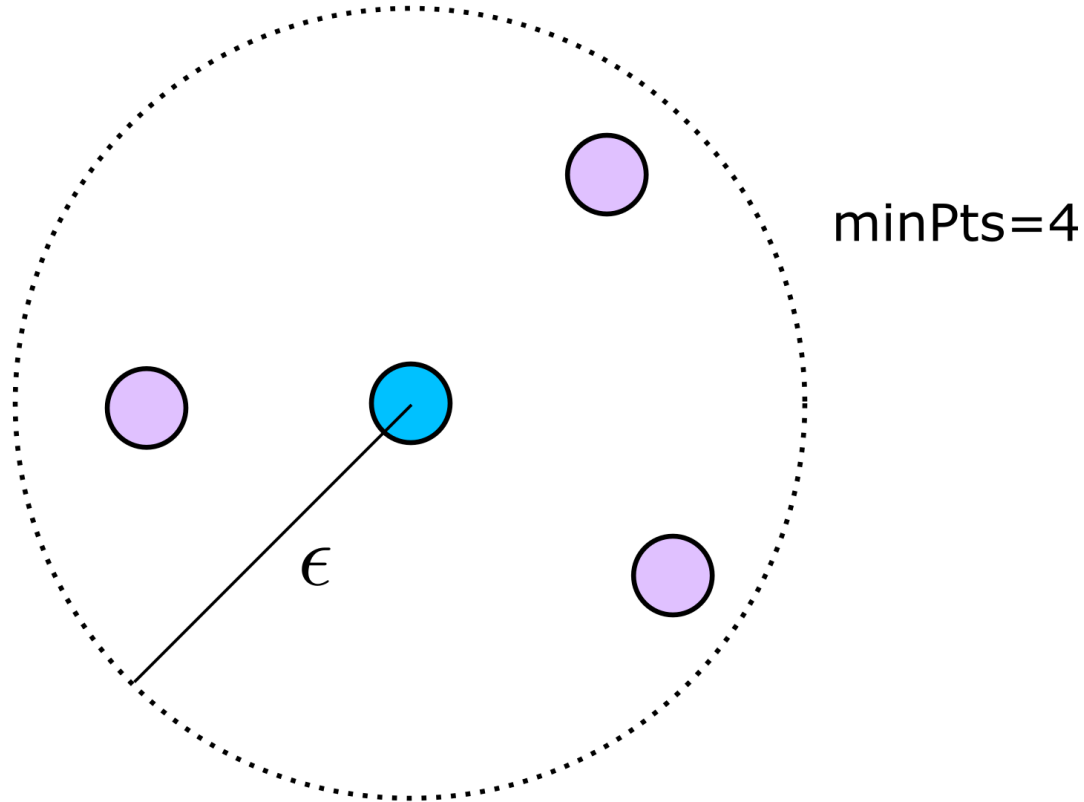
Density-Based

- DBSCAN classifies the points as:
 - Core points
 - Reachable points
 - Outliers

Core Points

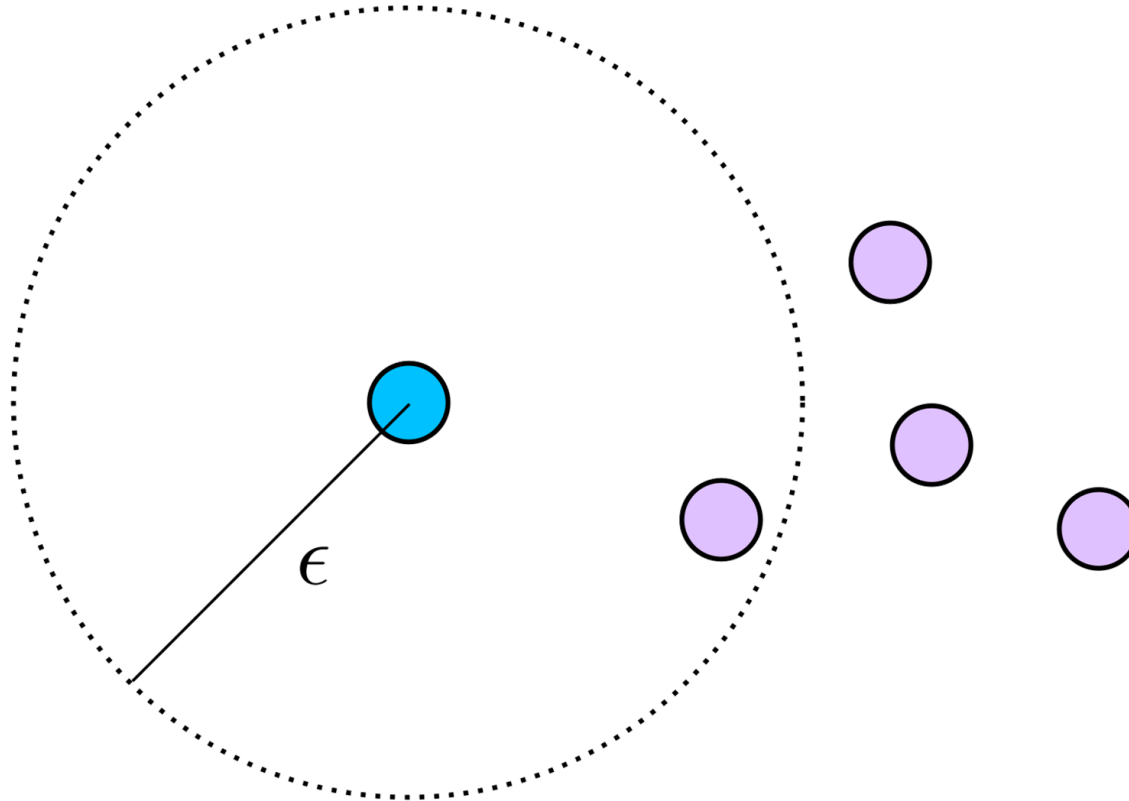
- A point is a **core point** if at least $minPts$ points are within a distance of ϵ

$$|N_{\epsilon}(p)| \geq minPts$$



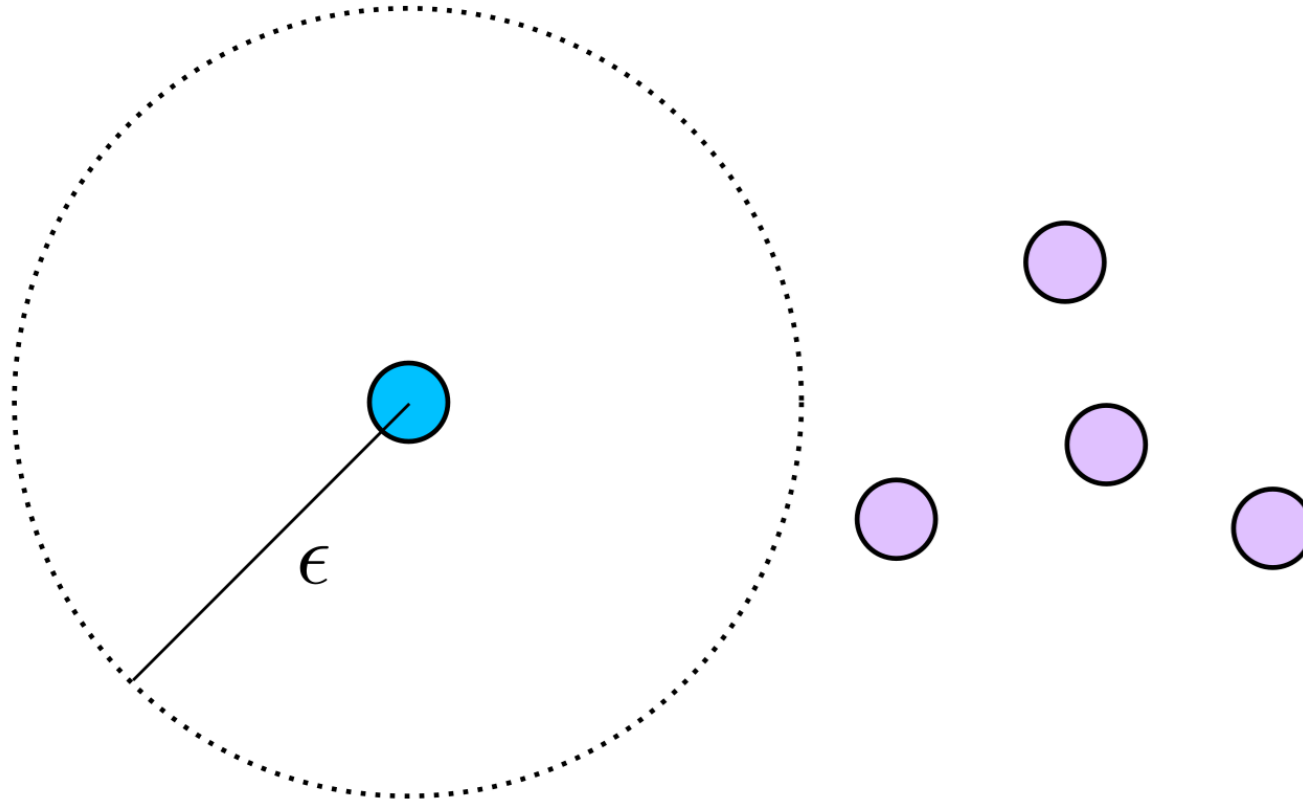
Reachable Points

- A point is a (directly density-)reachable point if at least one core point is within a distance of ϵ



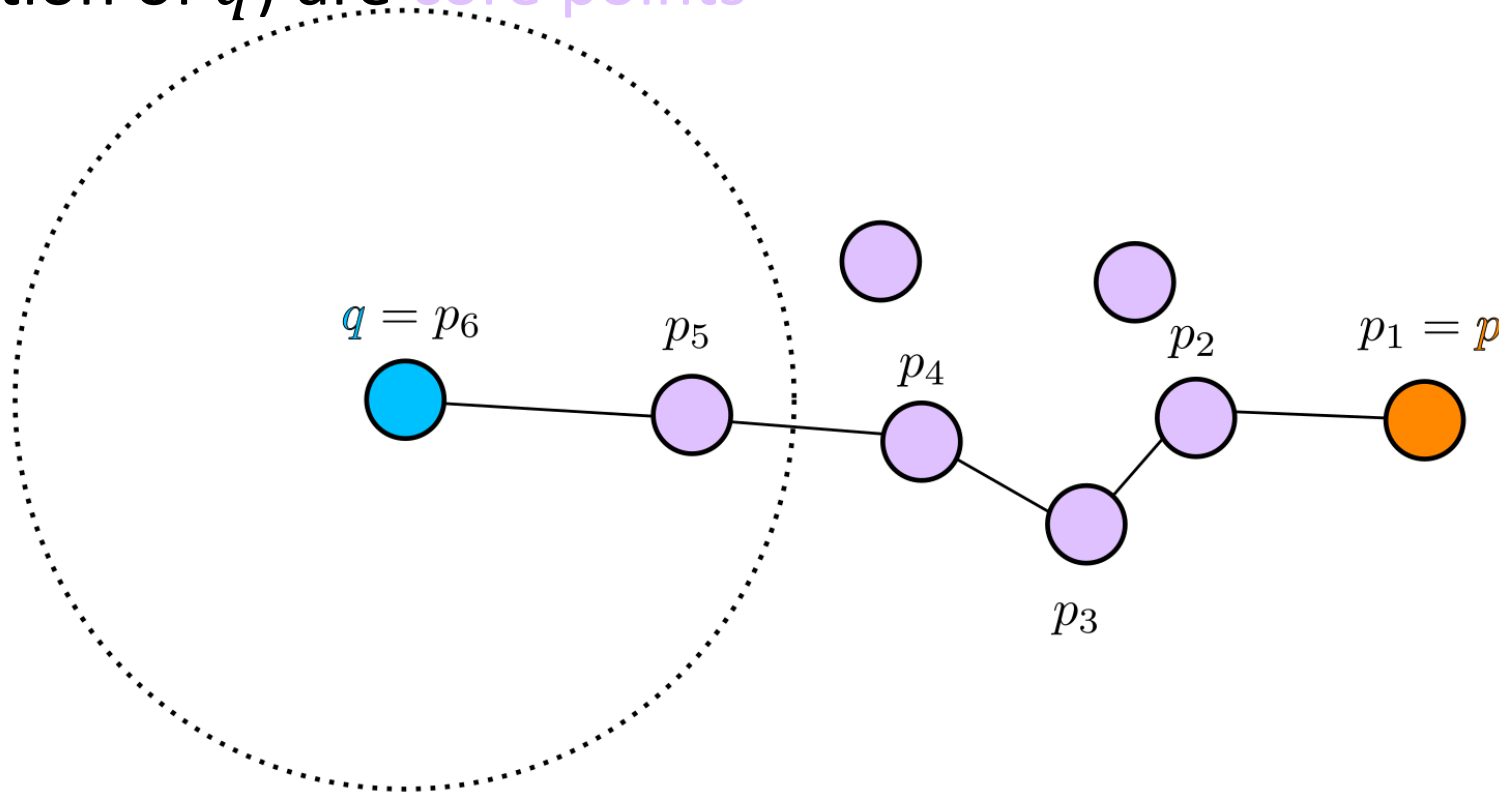
Outliers

- A point, which is not a reachable point is an **outlier**



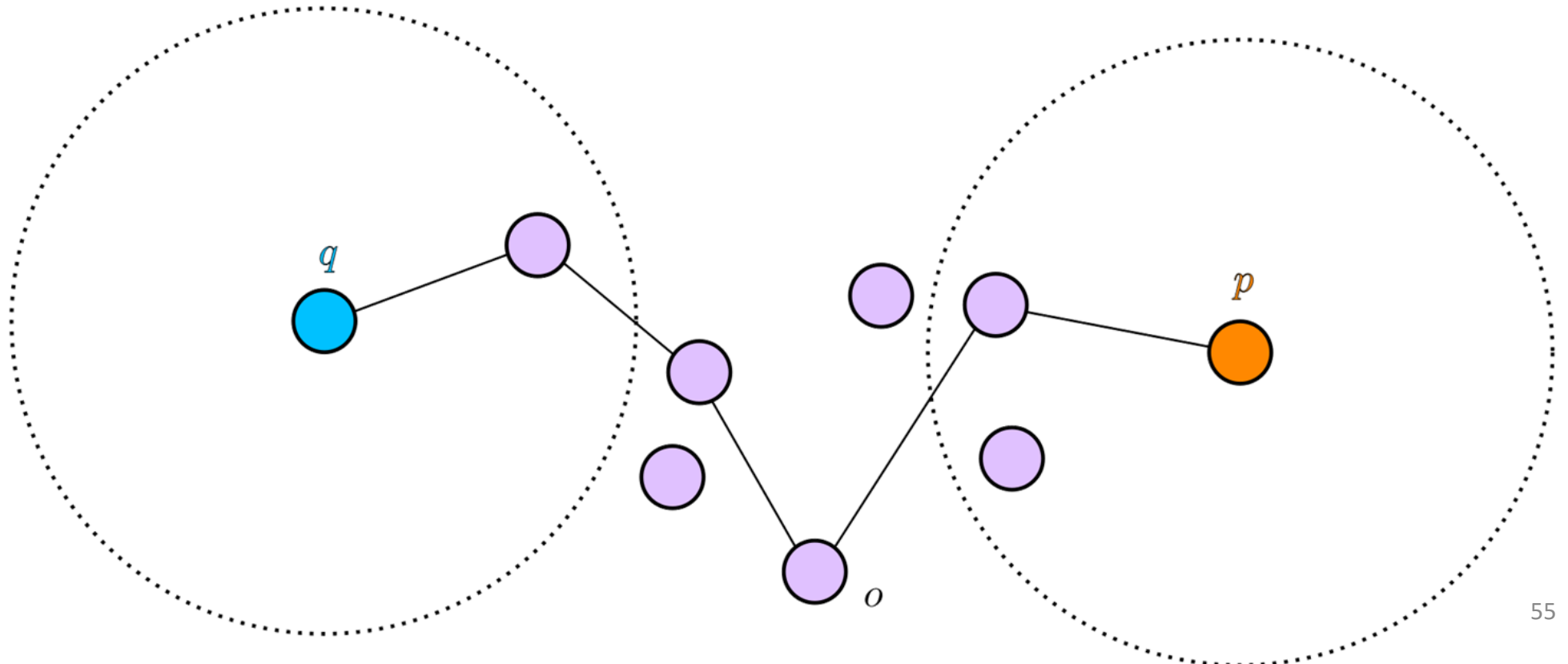
Reachable Path

- A point q is (density-)reachable by p , if there is a path $p_1, \dots, p_n$ with $p_1 = p$ and $p_n = q$, where p_{i+1} is reachable by p_i and all p_i (with the exception of q) are core points



Density Connected

- A point q is (density-)connected to p , if there is a point o such that both, q and p are (density-)reachable from o



Cluster

- Cluster is formed by at least one core point and all points that are (density-)reachable from it
- All points inside a cluster are (density-)connected
- Advantage:
 - detects arbitrary shapes
 - not limited to simple geometric forms

Noise

- Noise/Outliers are the set of points not belonging to any cluster
- Advantage over K-Means: does not force every point into a cluster
- Use cases:
 - detect measurement errors
 - identify anomalies
 - find rare events

Algorithm

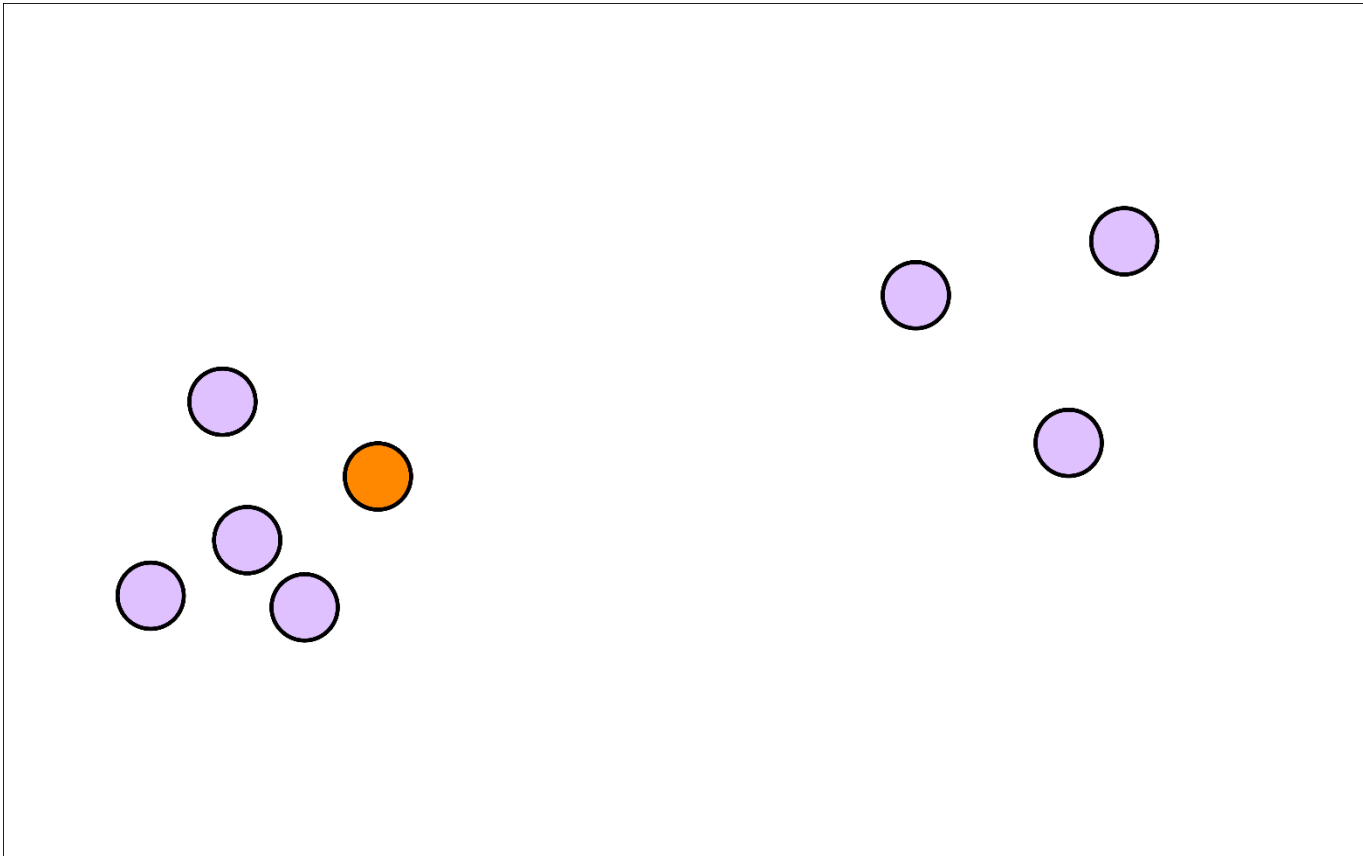
```
DBSCAN(DB, distFunc, eps, minPts) {  
    C = 0  
    for each point P in database DB {  
        if label(P) ≠ undefined then continue  
        Neighbors N = RangeQuery(DB, distFunc, P, eps)  
        if |N| < minPts then {  
            label(P) = Noise  
            continue  
        }  
        C = C + 1  
        label(P) = C  
        Seed set S = N \ {P}  
        for each point Q in S {  
            if label(Q) = Noise then label(Q) = C  
            if label(Q) ≠ undefined then continue  
            label(Q) = C  
            Neighbors N = RangeQuery(DB, distFunc, Q, eps)  
            if |N| ≥ minPts then {  
                S = S ∪ N  
            }  
        }  
    }  
}
```

/ Cluster counter */*
/ Previously processed in inner loop */*
/ Find neighbors */*
/ Density check */*
/ Label as Noise */*

/ next cluster label */*
/ Label initial point */*
/ Neighbors to expand */*
/ Process every seed point */*
/ Change Noise to border point */*
/ Previously processed */*
/ Label neighbor */*
/ Find neighbors */*
/ Density check */*
/ Add new neighbors to seed set */*

<https://en.wikipedia.org/wiki/DBSCAN>

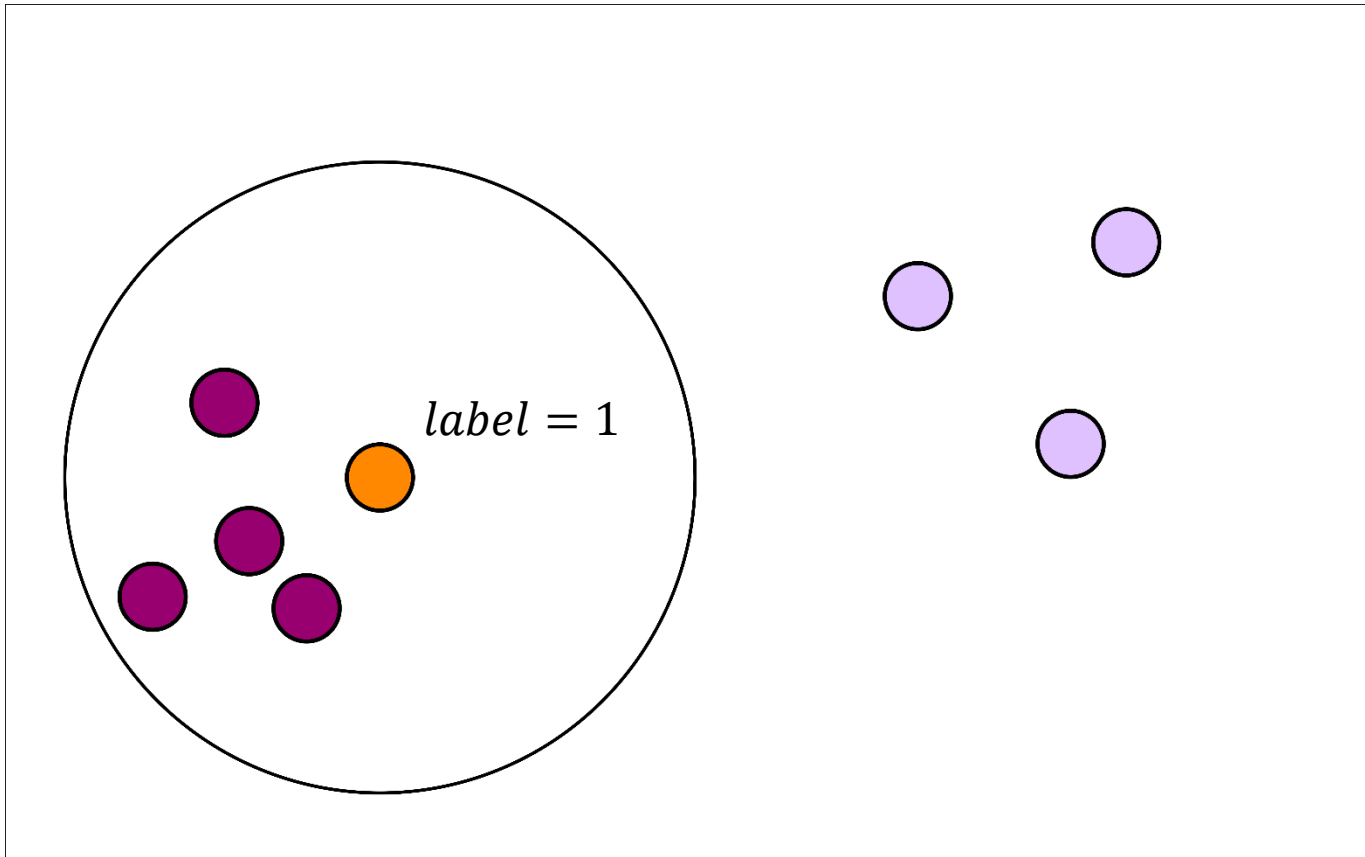
Algorithm



$\epsilon = 7.7$
 $minPts = 5$

```
DBSCAN(DB, distFunc, eps, minPts) {  
  C := 0  
  for each point P in database DB {  
    if label(P) = undefined then continue  
    Neighbors N := RangeQuery(DB, distFunc, P, eps)  
    if |N| < minPts then {  
      label(P) := Noise  
      continue  
    }  
    C := C + 1  
    label(P) := C  
    SeedSet S := N \ {P}  
    for each point Q in S {  
      if label(Q) = Noise then label(Q) := C  
      if label(Q) ≠ undefined then continue  
      label(Q) := C  
      Neighbors N := RangeQuery(DB, distFunc, Q, eps)  
      if |N| ≥ minPts then {  
        S := S ∪ N  
      }  
    }  
  }  
}
```

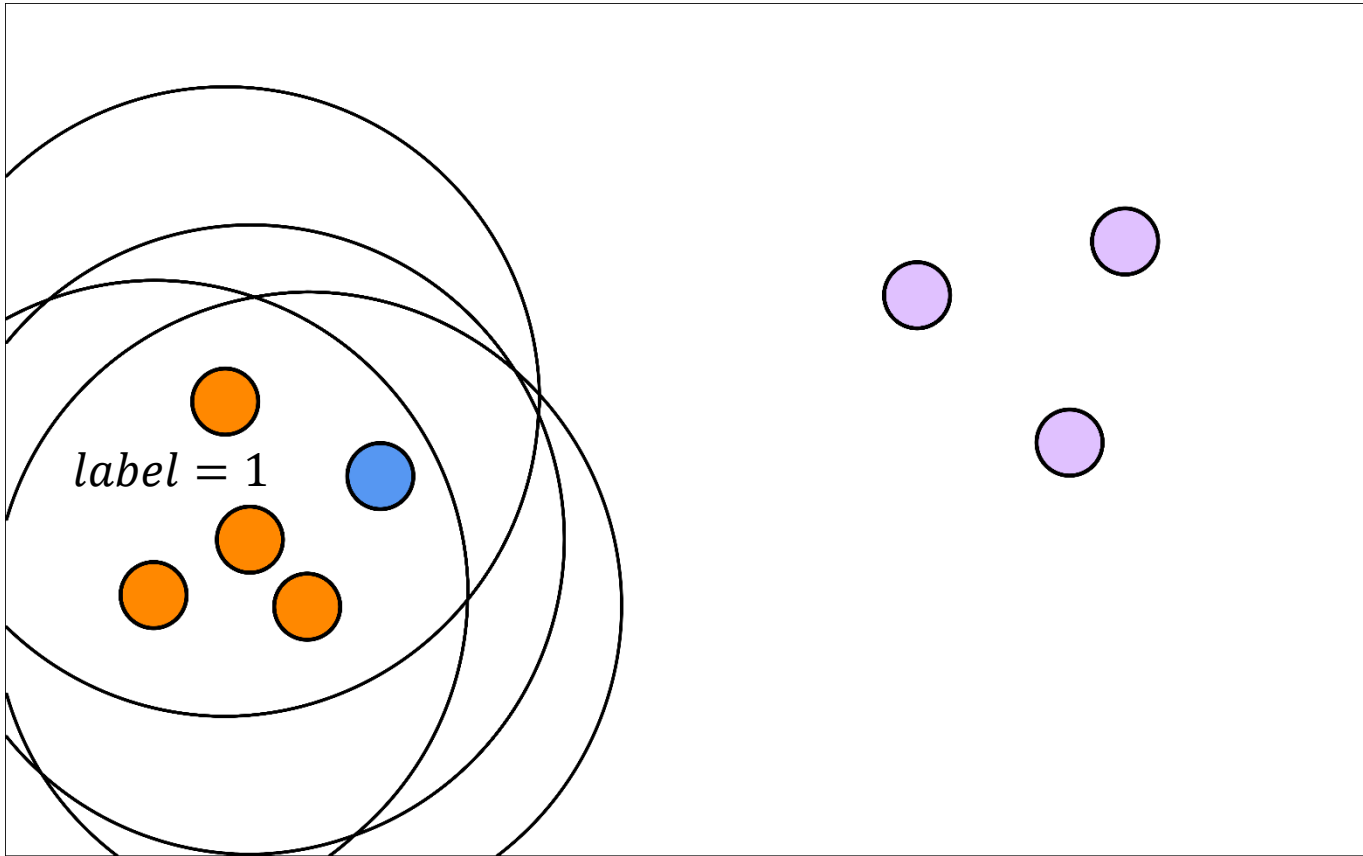
Algorithm



$\epsilon = 7.7$
 $minPts = 5$

```
DBSCAN(DB, distFunc, eps, minPts) {  
  C := 0  
  for each point P in database DB {  
    if label(P) = undefined then continue  
    Neighbors N := RangeQuery(DB, distFunc, P, eps)  
    if |N| < minPts then {  
      label(P) := Noise  
      continue  
    }  
    C := C + 1  
    label(P) := C  
    SeedSet S := N \ {P}  
    for each point Q in S {  
      if label(Q) = Noise then label(Q) := C  
      if label(Q) ≠ undefined then continue  
      label(Q) := C  
      Neighbors N := RangeQuery(DB, distFunc, Q, eps)  
      if |N| ≥ minPts then {  
        S := S ∪ N  
      }  
    }  
  }  
}
```

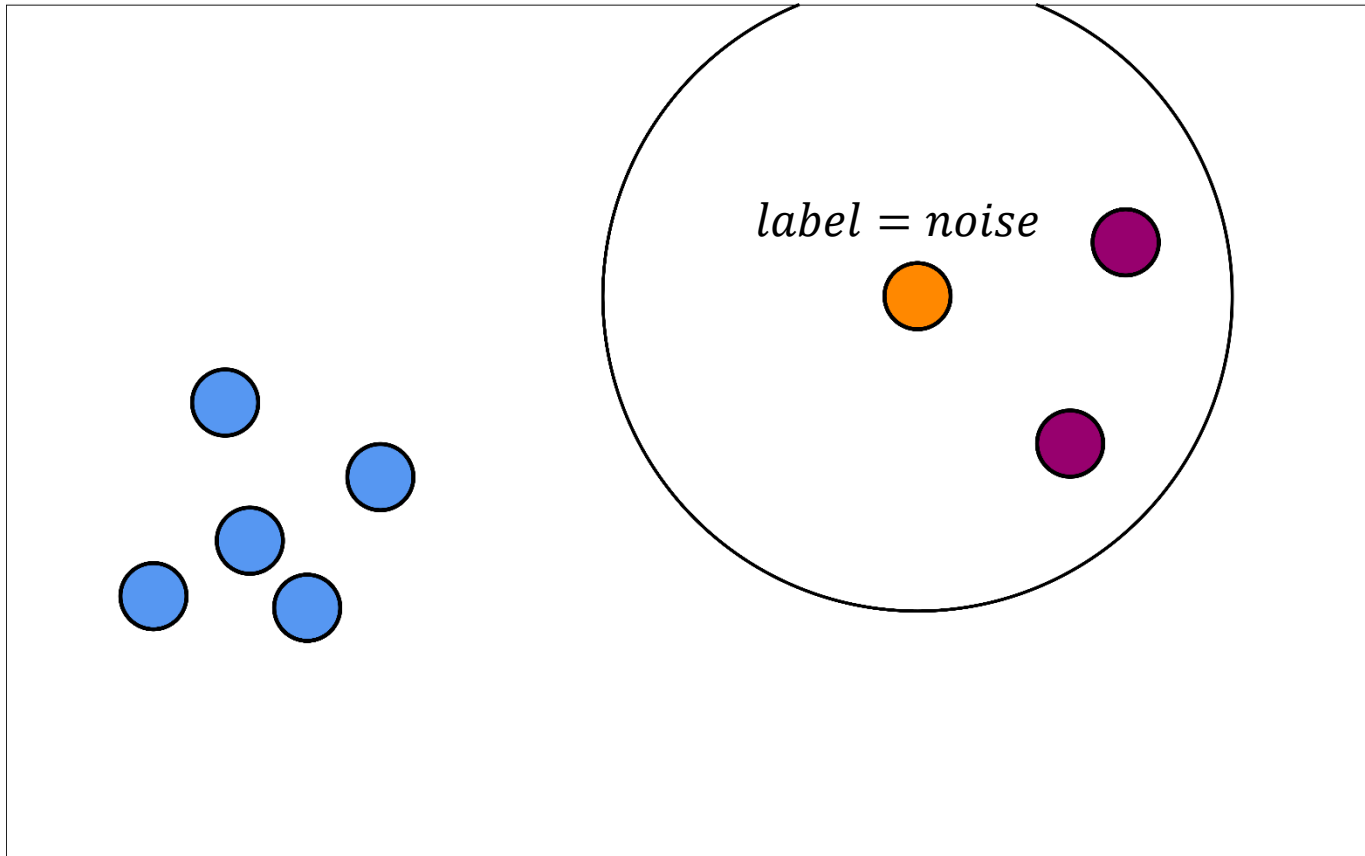
Algorithm



$\epsilon = 7.7$
 $minPts = 5$

```
DBSCAN(DB, distFunc, eps, minPts) {  
  C := 0  
  for each point P in database DB {  
    if label(P) = undefined then continue  
    Neighbors N := RangeQuery(DB, distFunc, P, eps)  
    if |N| < minPts then {  
      label(P) := Noise  
      continue  
    }  
    C := C + 1  
    label(P) := C  
    SeedSet S := N \ {P}  
    for each point Q in S {  
      if label(Q) = Noise then label(Q) := C  
      if label(Q) ≠ undefined then continue  
      label(Q) := C  
      Neighbors N := RangeQuery(DB, distFunc, Q, eps)  
      if |N| ≥ minPts then {  
        S := S ∪ N  
      }  
    }  
  }  
}
```

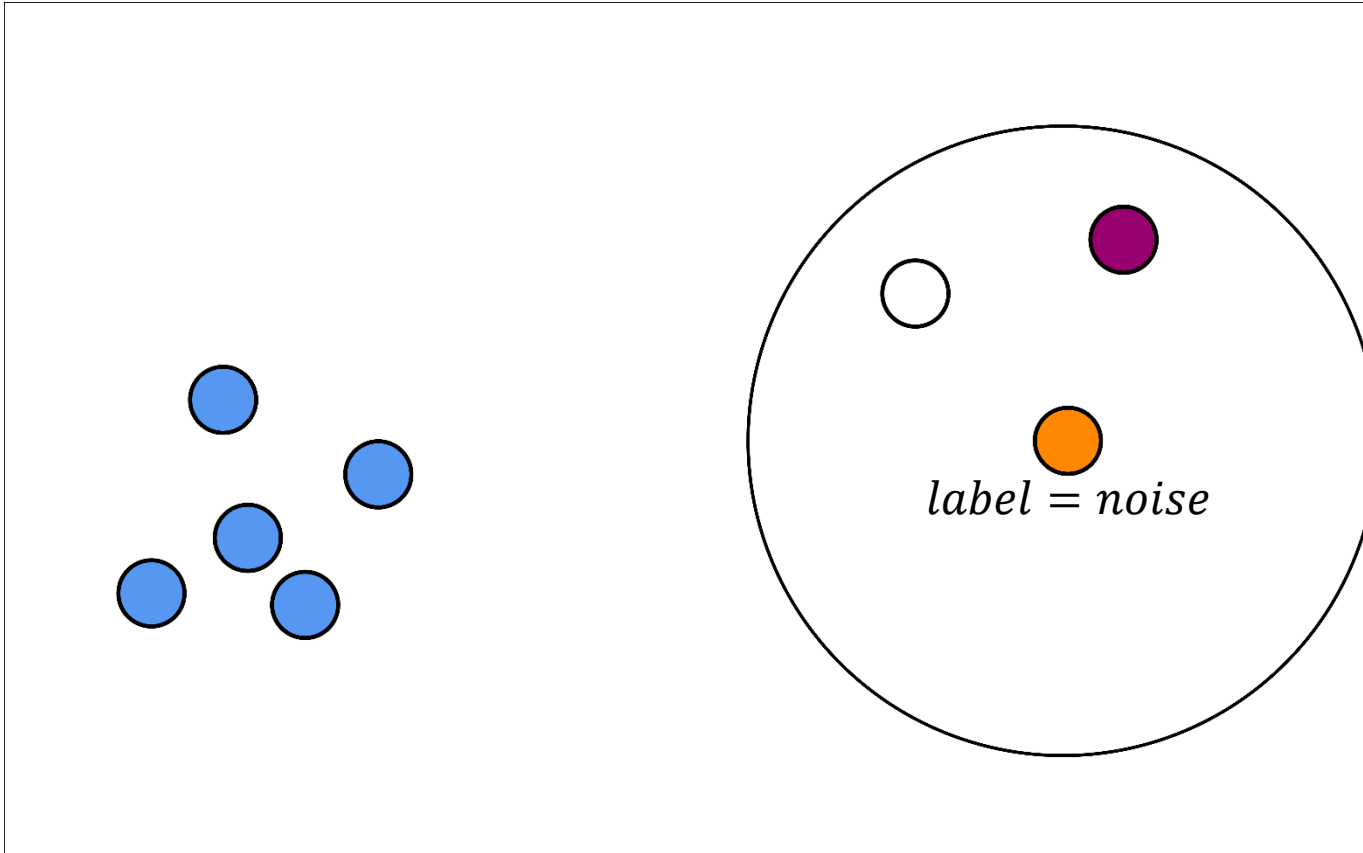
Algorithm



$\epsilon = 7.7$
 $minPts = 5$

```
DBSCAN(DB, distFunc, eps, minPts) {  
  C := 0  
  for each point P in database DB {  
    if label(P) = undefined then continue  
    Neighbors N := RangeQuery(DB, distFunc, P, eps)  
    if |N| < minPts then {  
      label(P) := Noise  
      continue  
    }  
    C := C + 1  
    label(P) := C  
    SeedSet S := N \ {P}  
    for each point Q in S {  
      if label(Q) = Noise then label(Q) := C  
      if label(Q) ≠ undefined then continue  
      label(Q) := C  
      Neighbors N := RangeQuery(DB, distFunc, Q, eps)  
      if |N| ≥ minPts then {  
        S := S ∪ N  
      }  
    }  
  }  
}
```

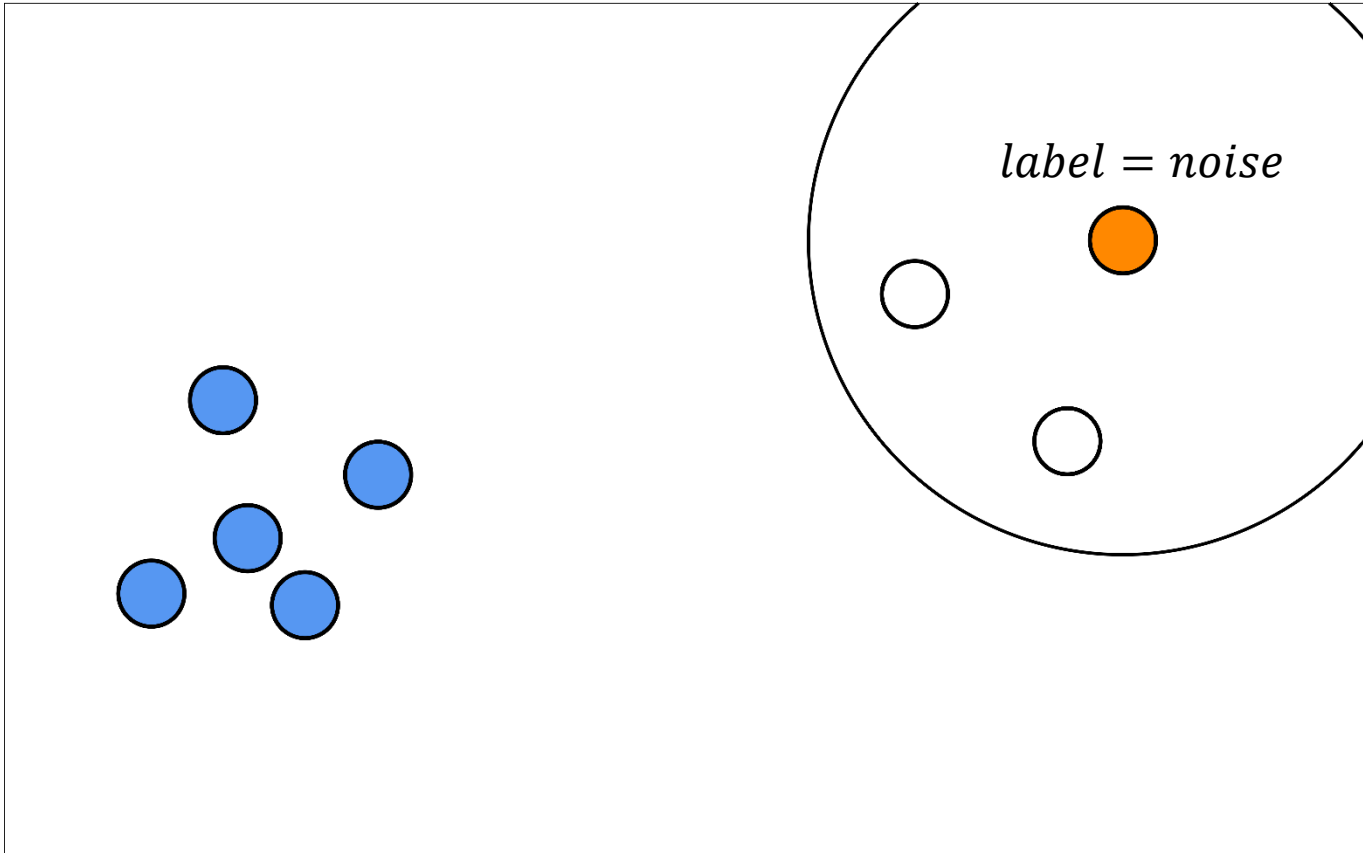
Algorithm



$\epsilon = 7.7$
 $minPts = 5$

```
DBSCAN(DB, distFunc, eps, minPts) {  
  C := 0  
  for each point P in database DB {  
    if label(P) = undefined then continue  
    Neighbors N := RangeQuery(DB, distFunc, P, eps)  
    if |N| < minPts then {  
      label(P) := Noise  
      continue  
    }  
    C := C + 1  
    label(P) := C  
    SeedSet S := N \ {P}  
    for each point Q in S {  
      if label(Q) = Noise then label(Q) := C  
      if label(Q) ≠ undefined then continue  
      label(Q) := C  
      Neighbors N := RangeQuery(DB, distFunc, Q, eps)  
      if |N| ≥ minPts then {  
        S := S ∪ N  
      }  
    }  
  }  
}
```

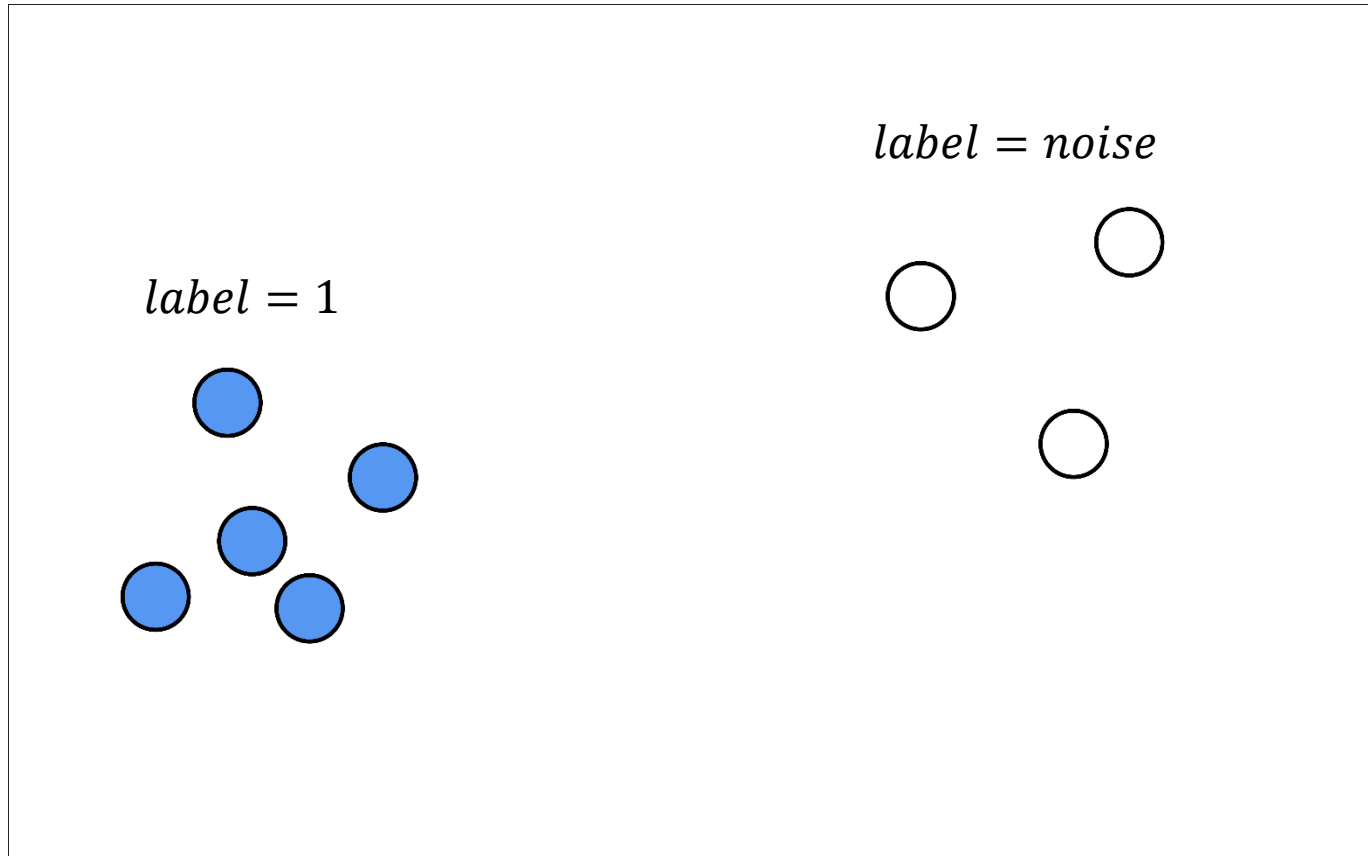
Algorithm



$\epsilon = 7.7$
 $minPts = 5$

```
DBSCAN(DB, distFunc, eps, minPts) {  
  C := 0  
  for each point P in database DB {  
    if label(P) = undefined then continue  
    Neighbors N := RangeQuery(DB, distFunc, P, eps)  
    if |N| < minPts then {  
      label(P) := Noise  
      continue  
    }  
    C := C + 1  
    label(P) := C  
    SeedSet S := N \ {P}  
    for each point Q in S {  
      if label(Q) = Noise then label(Q) := C  
      if label(Q) ≠ undefined then continue  
      label(Q) := C  
      Neighbors N := RangeQuery(DB, distFunc, Q, eps)  
      if |N| ≥ minPts then {  
        S := S ∪ N  
      }  
    }  
  }  
}
```


Algorithm

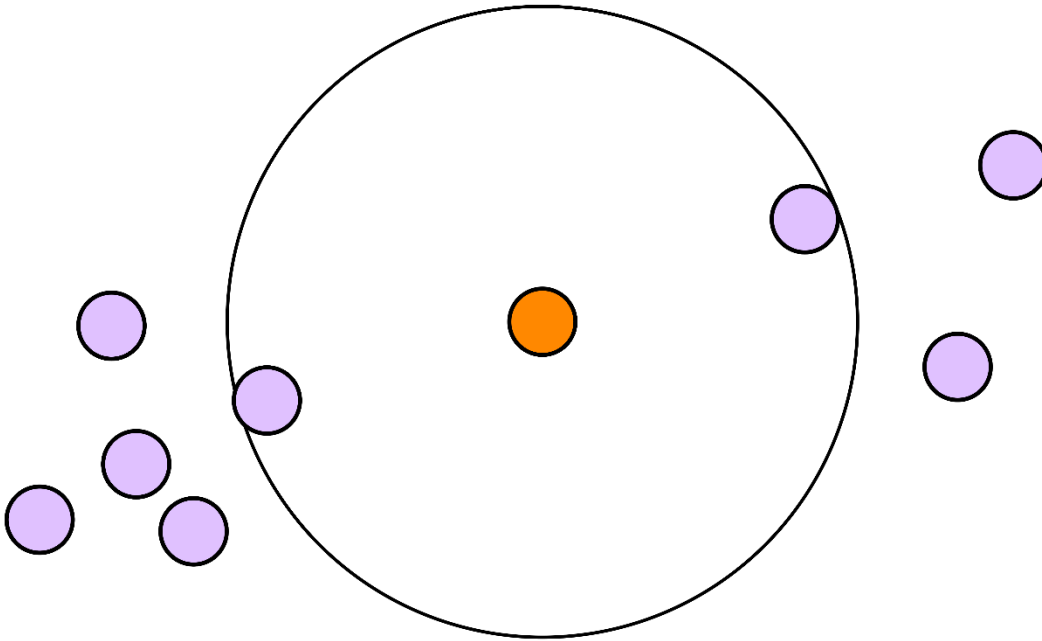


$\epsilon = 7.7$
 $minPts = 5$

```
DBSCAN(DB, distFunc, eps, minPts) {  
  C := 0  
  for each point P in database DB {  
    if label(P) = undefined then continue  
    Neighbors N := RangeQuery(DB, distFunc, P, eps)  
    if |N| < minPts then {  
      label(P) := Noise  
      continue  
    }  
    C := C + 1  
    label(P) := C  
    SeedSet S := N \ {P}  
    for each point Q in S {  
      if label(Q) = Noise then label(Q) := C  
      if label(Q) ≠ undefined then continue  
      label(Q) := C  
      Neighbors N := RangeQuery(DB, distFunc, Q, eps)  
      if |N| ≥ minPts then {  
        S := S ∪ N  
      }  
    }  
  }  
}
```

Algorithm

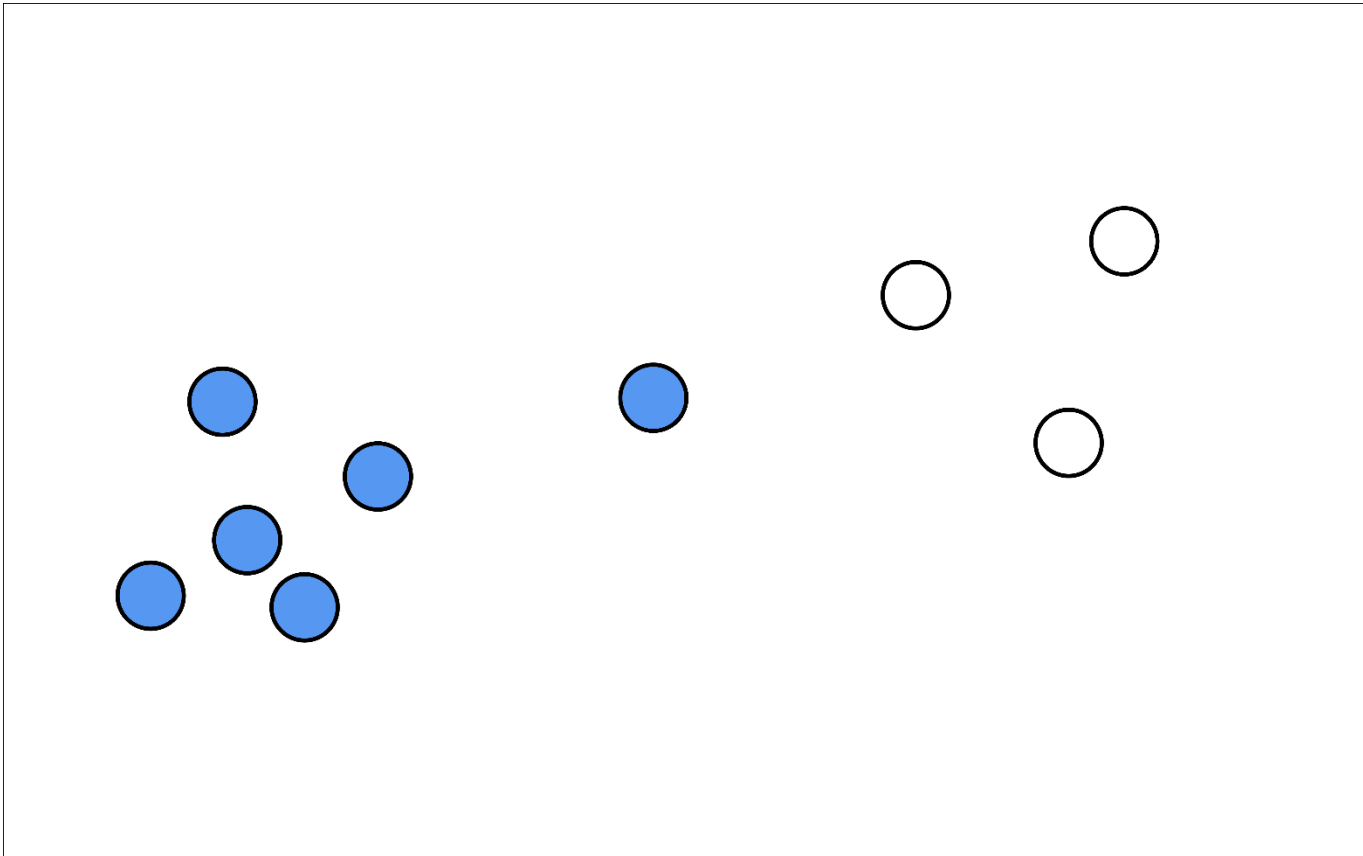
What do the clustering results look like?



$\epsilon = 7.7$
 $minPts = 5$

```
DBSCAN(DB, distFunc, eps, minPts) {  
  C := 0  
  for each point P in database DB {  
    if label(P) = undefined then continue  
    Neighbors N := RangeQuery(DB, distFunc, P, eps)  
    if |N| < minPts then {  
      label(P) := Noise  
      continue  
    }  
    C := C + 1  
    label(P) := C  
    SeedSet S := N \ {P}  
    for each point Q in S {  
      if label(Q) = Noise then label(Q) := C  
      if label(Q) ≠ undefined then continue  
      label(Q) := C  
      Neighbors N := RangeQuery(DB, distFunc, Q, eps)  
      if |N| ≥ minPts then {  
        S := S ∪ N  
      }  
    }  
  }  
}
```

Algorithm

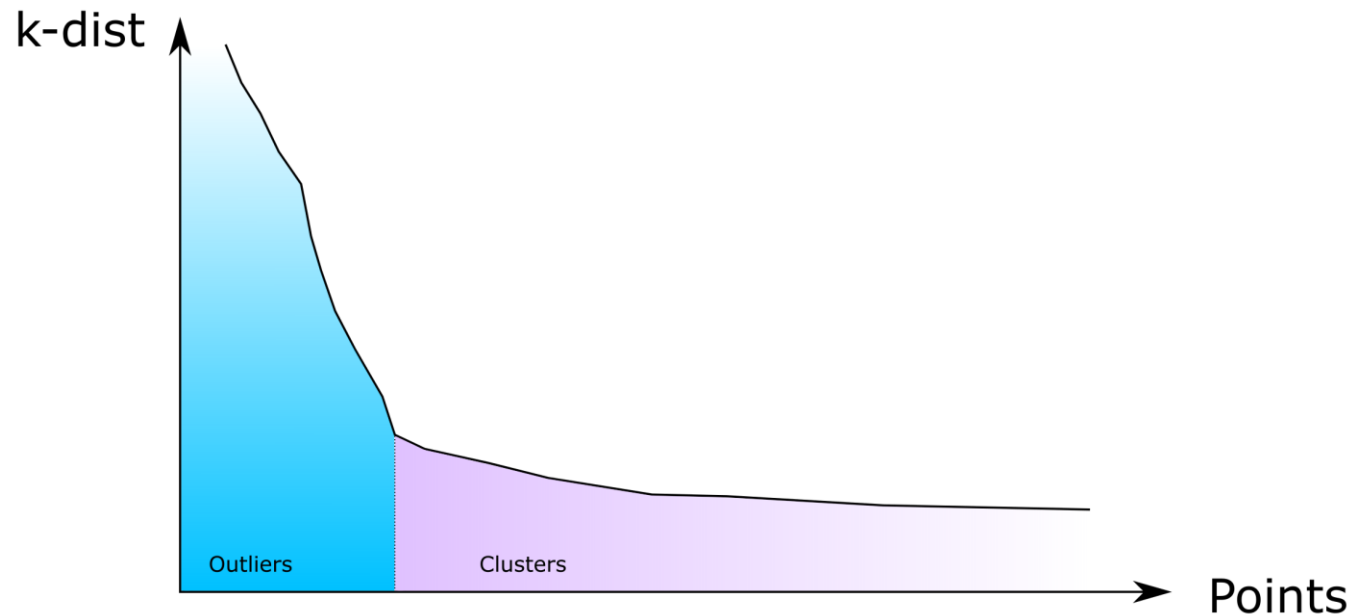


$\epsilon = 7.7$
 $minPts = 5$

```
DBSCAN(DB, distFunc, eps, minPts) {  
  C := 0  
  for each point P in database DB {  
    if label(P) = undefined then continue  
    Neighbors N := RangeQuery(DB, distFunc, P, eps)  
    if |N| < minPts then {  
      label(P) := Noise  
      continue  
    }  
    C := C + 1  
    label(P) := C  
    SeedSet S := N \ {P}  
    for each point Q in S {  
      if label(Q) = Noise then label(Q) := C  
      if label(Q) ≠ undefined then continue  
      label(Q) := C  
      Neighbors N := RangeQuery(DB, distFunc, Q, eps)  
      if |N| ≥ minPts then {  
        S := S ∪ N  
      }  
    }  
  }  
}
```

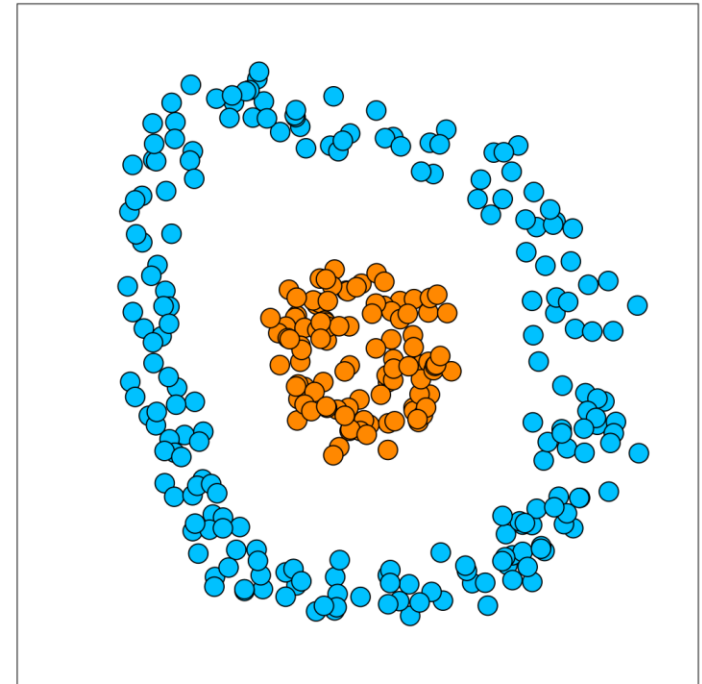
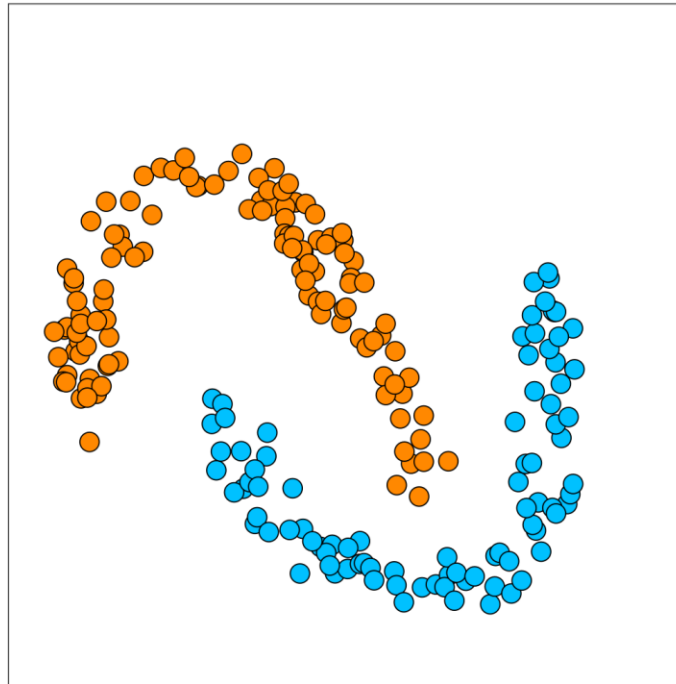
Remarks

- As a rule of thumb start with $\text{minPts} = 2 \cdot \text{dim}$
- The parameter ε can be chosen by a k-distance graph:
 - Determine for each point the minimal distance to the nearest $k = \text{minPts} - 1$ points
 - Order the distances and plot the graph (find the elbow)



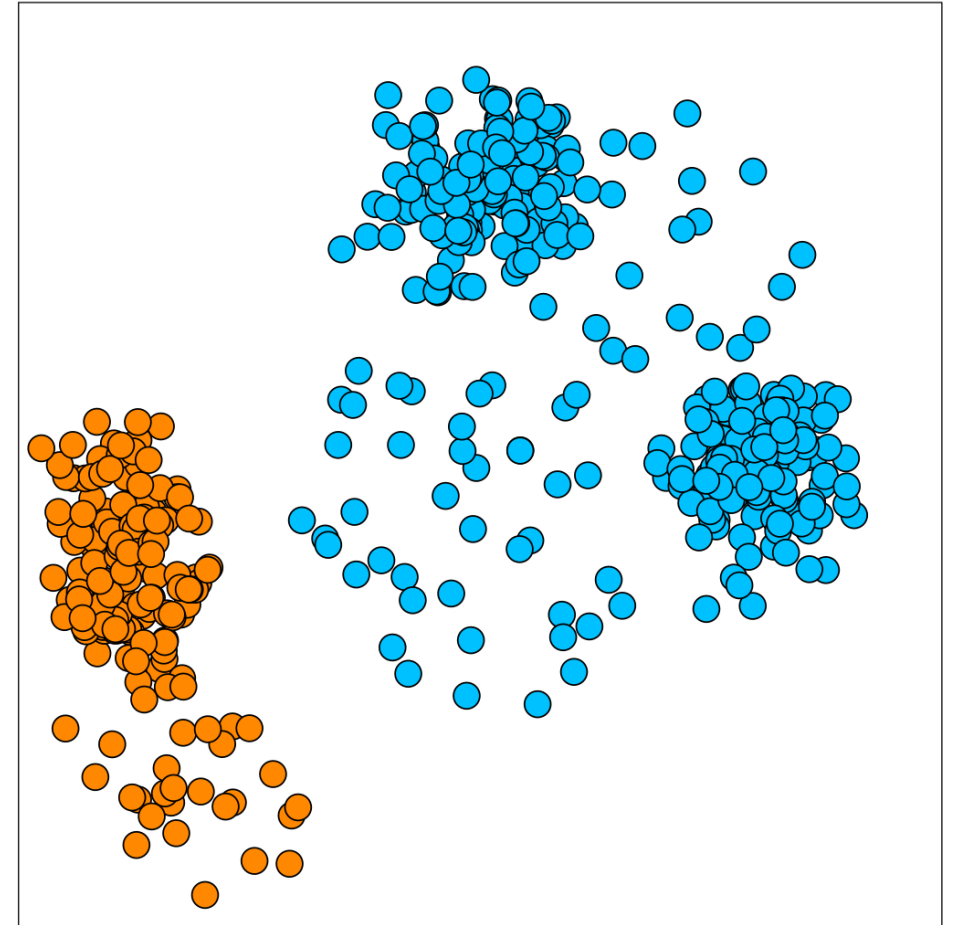
Summary

- Does not need number of clusters
- Cluster may be arbitrarily shaped and detected
- Robust to outliers

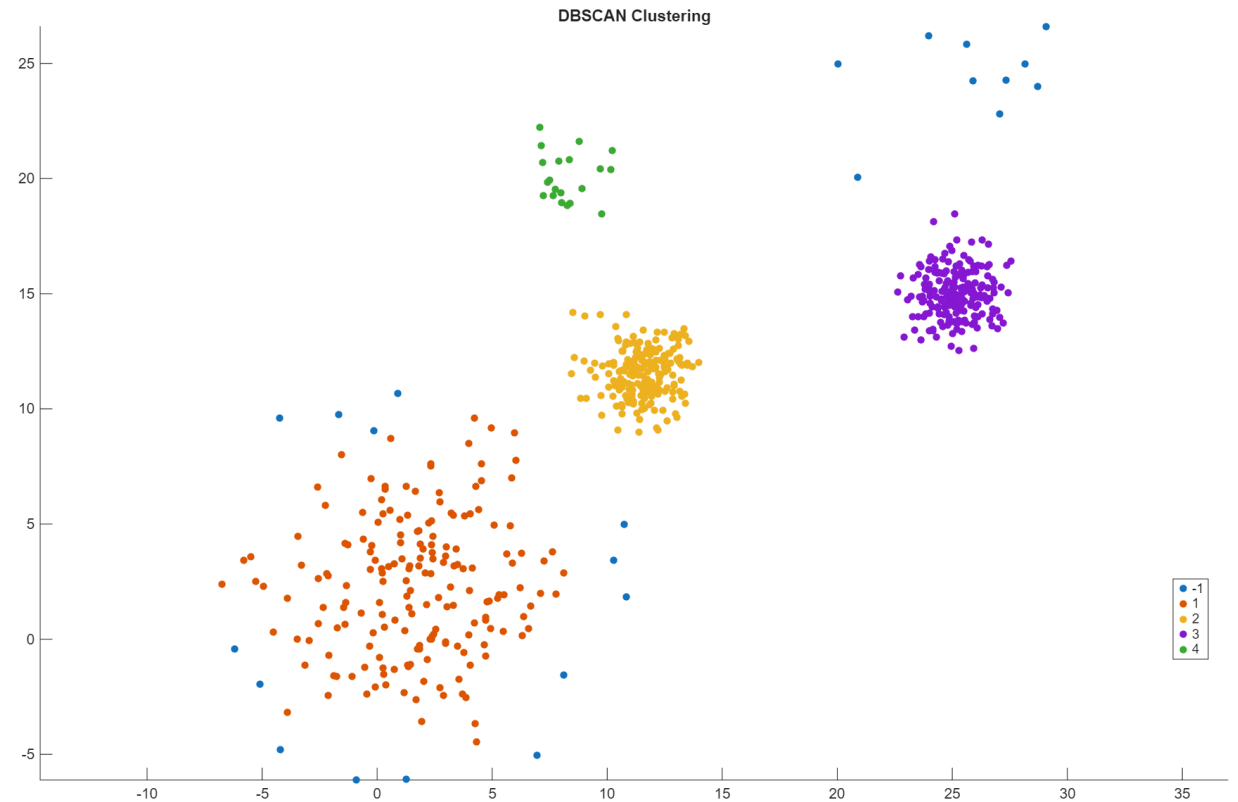
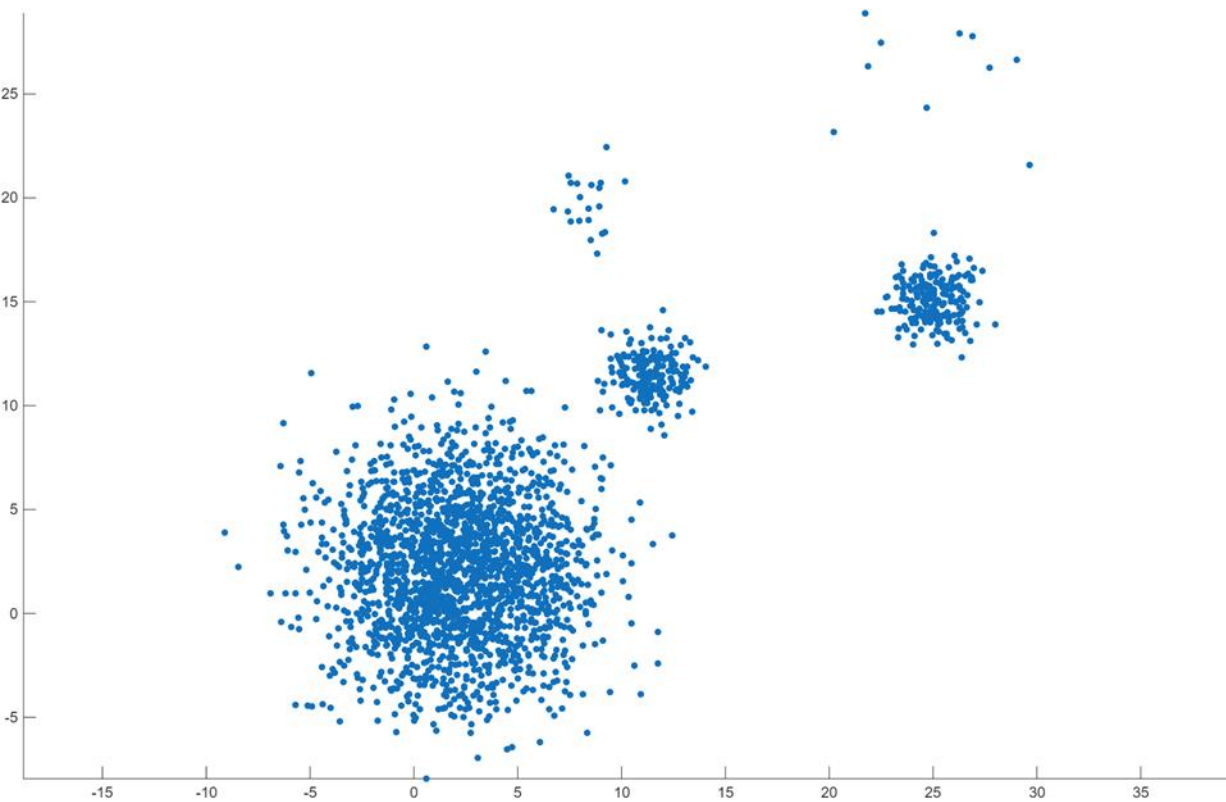


Summary

- Cannot cluster well with large differences in densities
- Choosing a meaningful ε can be difficult



Example



$$\text{minPts} = 6, \epsilon = 2$$

OPTICS

(Ordering Points To Identify the Clustering Structure)

Introduction

- OPTICS is basically DBSCAN, but it addresses one of the major weaknesses: finding meaningful clusters of varying density
- Like DBSCAN, OPTICS requires two parameters: *minPts* and ε both with the same meaning as in DBSCAN

Vocabularies

- Like DBSCAN, we need the same definitions of:
 - Core points
 - (Density-)reachable
 - (Density-)connected
 - Clusters
 - Noise/Outliers

Idea

- The idea is to assign a core-distance to each point yielding the distance to at least *minPts* points that are within this distance
- This gives us the opportunity to find more densely clusters

Core-Distance

- Let $\text{minPts}_{\text{dist}}(p)$ be the distance from p to its minPts ' neighbors

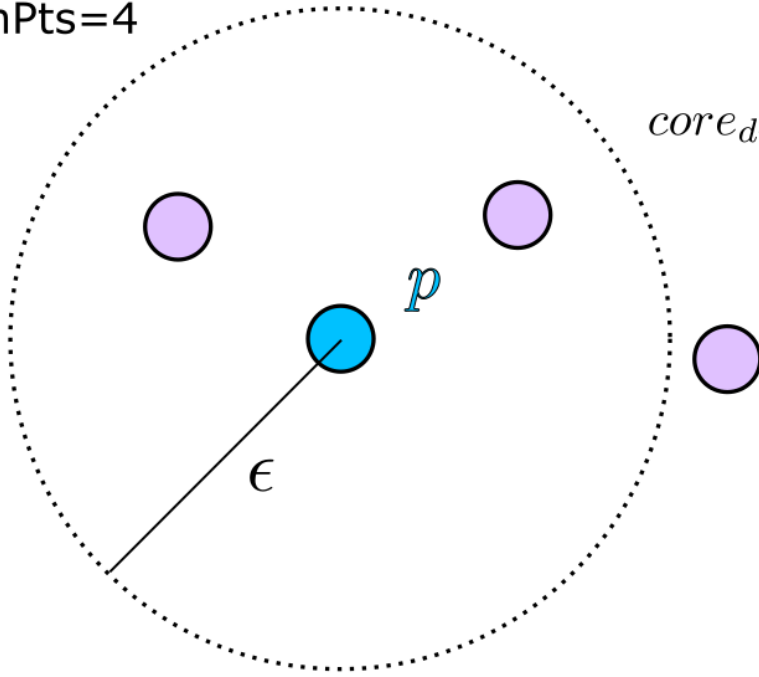
Core-Distance

- Let $minPts_{dist}(p)$ be the distance from p to its $minPts$ ' neighbors
- Then core-distance $core_{dist}(p)$ of p is defined as:

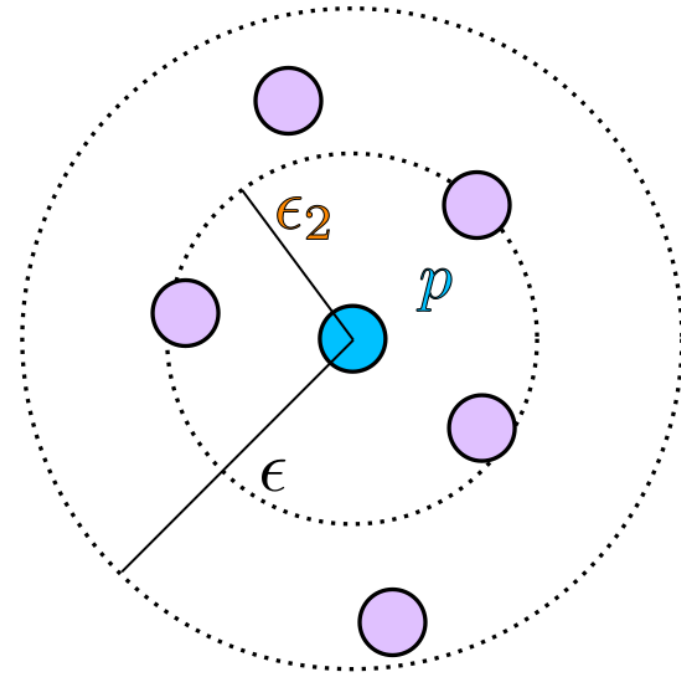
$$core_{dist}(p) = \begin{cases} \text{UNDEFINED} & \text{if } |N_{\varepsilon}(p)| < minPts \\ minPts_{dist}(p) & \text{otherwise} \end{cases} \quad (\varepsilon = dist)$$

Core-Distance

minPts=4



$core_{dist}(p) = UNDEFINED$



$core_{dist}(p) = \epsilon_2$

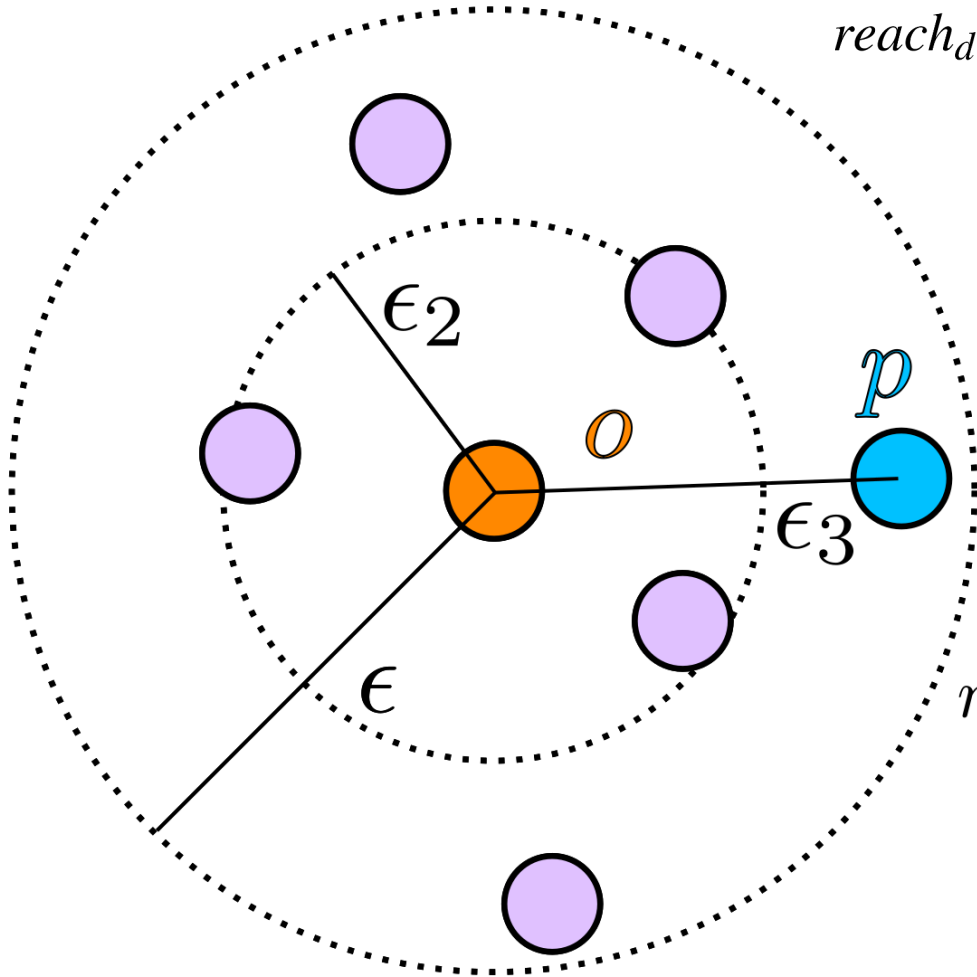
Reachability-Distance

- The reachability-distance $reach_{dist}(p, o)$ of p with respect to o is defined as:

$$reach_{dist}(p, o) = \begin{cases} \text{UNDEFINED} & \text{if } |N_{\varepsilon}(o)| < minPts \\ \max(core_{dist}(o), dist(o, p)) & \text{otherwise} \end{cases}$$

Reachability-Distance

$$reach_{dist}(p, o) = \begin{cases} \text{UNDEFINED} & \text{if } |N_\epsilon(o)| < minPts \\ \max(core_{dist}(o), dist(o, p)) & \text{otherwise} \end{cases}$$



$$\begin{aligned} core_{dist}(o) &= \epsilon_2 \\ \wedge \\ dist(o, p) &= \epsilon_3 \end{aligned}$$

$$reach_{dist}(p, o) = \epsilon_3$$

Algorithm

```
OPTICS(DB, eps, MinPts)
    Initialization
    for each point p of DB
        p.reachability-distance = UNDEFINED
    for each unprocessed point p of DB
        N = getNeighbors(p, eps)
        mark p as processed
        output p to the ordered list
        if (core-distance(p, eps, Minpts) != UNDEFINED)
            Seeds = empty priority queue
            update(N, p, Seeds, eps, Minpts)
            for each next q in Seeds
                N' = getNeighbors(q, eps)
                mark q as processed
                output q to the ordered list
                if (core-distance(q, eps, Minpts) != UNDEFINED)
                    update(N', q, Seeds, eps, Minpts)
```

```
update(N, p, Seeds, eps, Minpts)
    coredist = core-distance(p, eps, MinPts)
    for each o in N
        if (o is not processed)
            new-reach-dist = max(coredist, dist(p,o))
            if (o.reachability-distance == UNDEFINED) // o is not in Seeds
                o.reachability-distance = new-reach-dist
                Seeds.insert(o, new-reach-dist)
            else // o in Seeds, check for improvement
                if (new-reach-dist < o.reachability-distance)
                    o.reachability-distance = new-reach-dist
                    Seeds.move-up(o, new-reach-dist)
```

https://en.wikipedia.org/wiki/OPTICS_algorithm

Algorithm

```
OPTICS(DB, eps, MinPts)
  for each point p of DB
    p.reachability-distance = UNDEFINED
  for each unprocessed point p of DB
    N = getNeighbors(p, eps)
    mark p as processed
    output p to the ordered list
  if (core-distance(p, eps, Minpts) != UNDEFINED)
    Seeds = empty priority queue
    update(N, p, Seeds, eps, Minpts)
    for each next q in Seeds
      N' = getNeighbors(q, eps)
      mark q as processed
      output q to the ordered list
      if (core-distance(q, eps, Minpts) != UNDEFINED)
        update(N', q, Seeds, eps, Minpts)
```

Main loop

```
update(N, p, Seeds, eps, Minpts)
  coredist = core-distance(p, eps, MinPts)
  for each o in N
    if (o is not processed)
      new-reach-dist = max(coredist, dist(p,o))
      if (o.reachability-distance == UNDEFINED) // o is not in Seeds
        o.reachability-distance = new-reach-dist
        Seeds.insert(o, new-reach-dist)
    else // o in Seeds, check for improvement
      if (new-reach-dist < o.reachability-distance)
        o.reachability-distance = new-reach-dist
        Seeds.move-up(o, new-reach-dist)
```

https://en.wikipedia.org/wiki/OPTICS_algorithm

Algorithm

```
OPTICS(DB, eps, MinPts)
  for each point p of DB
    p.reachability-distance = UNDEFINED
  for each unprocessed point p of DB
    N = getNeighbors(p, eps)
    mark p as processed
    output p to the ordered list
    if (core-distance(p, eps, Minpts) != UNDEFINED)
      Seeds = empty priority queue
      update(N, p, Seeds, eps, Minpts)
      for each next q in Seeds
        N' = getNeighbors(q, eps)
        mark q as processed
        output q to the ordered list
        if (core-distance(q, eps, Minpts) != UNDEFINED)
          update(N', q, Seeds, eps, Minpts)
```

Core point check

```
update(N, p, Seeds, eps, Minpts)
  coredist = core-distance(p, eps, MinPts)
  for each o in N
    if (o is not processed)
      new-reach-dist = max(coredist, dist(p,o))
      if (o.reachability-distance == UNDEFINED) // o is not in Seeds
        o.reachability-distance = new-reach-dist
        Seeds.insert(o, new-reach-dist)
    else // o in Seeds, check for improvement
      if (new-reach-dist < o.reachability-distance)
        o.reachability-distance = new-reach-dist
        Seeds.move-up(o, new-reach-dist)
```

https://en.wikipedia.org/wiki/OPTICS_algorithm

Algorithm

```
OPTICS(DB, eps, MinPts)
  for each point p of DB
    p.reachability-distance = UNDEFINED
  for each unprocessed point p of DB
    N = getNeighbors(p, eps)
    mark p as processed
    output p to the ordered list
    if (core-distance(p, eps, MinPts) != UNDEFINED)
      Seeds = empty priority queue
      update(N, p, Seeds, eps, MinPts) Cluster expansion
      for each next q in Seeds
        N' = getNeighbors(q, eps)
        mark q as processed
        output q to the ordered list
        if (core-distance(q, eps, MinPts) != UNDEFINED)
          update(N', q, Seeds, eps, MinPts)
```

```
update(N, p, Seeds, eps, MinPts)
  coredist = core-distance(p, eps, MinPts)
  for each o in N
    if (o is not processed)
      new-reach-dist = max(coredist, dist(p,o))
      if (o.reachability-distance == UNDEFINED) // o is not in Seeds
        o.reachability-distance = new-reach-dist
        Seeds.insert(o, new-reach-dist)
    else // o in Seeds, check for improvement
      if (new-reach-dist < o.reachability-distance)
        o.reachability-distance = new-reach-dist
        Seeds.move-up(o, new-reach-dist)
```

https://en.wikipedia.org/wiki/OPTICS_algorithm

Algorithm

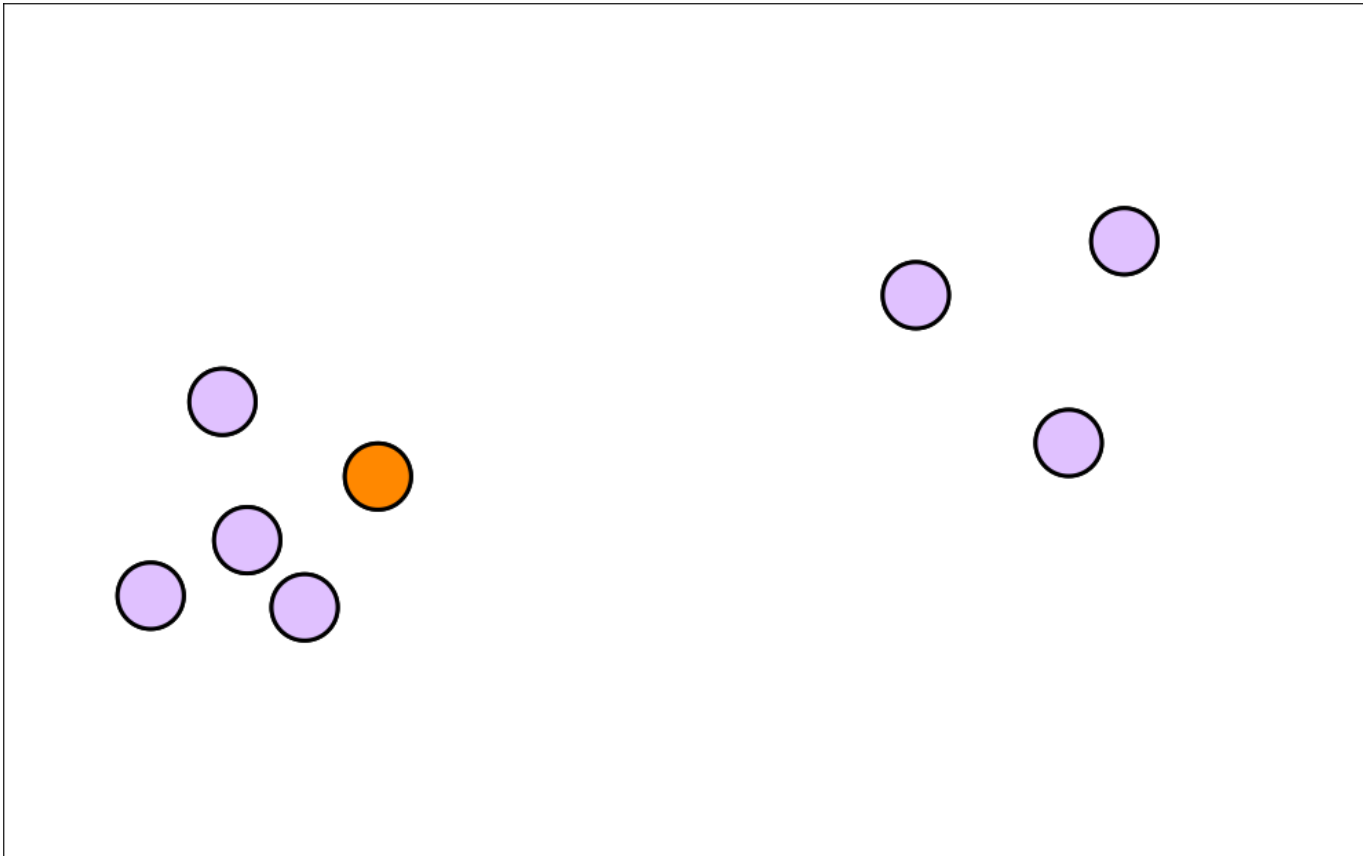
```
OPTICS(DB, eps, MinPts)
  for each point p of DB
    p.reachability-distance = UNDEFINED
  for each unprocessed point p of DB
    N = getNeighbors(p, eps)
    mark p as processed
    output p to the ordered list
    if (core-distance(p, eps, MinPts) != UNDEFINED)
      Seeds = empty priority queue
      update(N, p, Seeds, eps, MinPts)
      for each next q in Seeds
        N' = getNeighbors(q, eps)
        mark q as processed
        output q to the ordered list
        if (core-distance(q, eps, MinPts) != UNDEFINED)
          update(N', q, Seeds, eps, MinPts)
```

Update reachability-distances

```
update(N, p, Seeds, eps, MinPts)
  coredist = core-distance(p, eps, MinPts)
  for each o in N
    if (o is not processed)
      new-reach-dist = max(coredist, dist(p,o))
      if (o.reachability-distance == UNDEFINED) // o is not in Seeds
        o.reachability-distance = new-reach-dist
        Seeds.insert(o, new-reach-dist)
    else // o in Seeds, check for improvement
      if (new-reach-dist < o.reachability-distance)
        o.reachability-distance = new-reach-dist
        Seeds.move-up(o, new-reach-dist)
```

https://en.wikipedia.org/wiki/OPTICS_algorithm

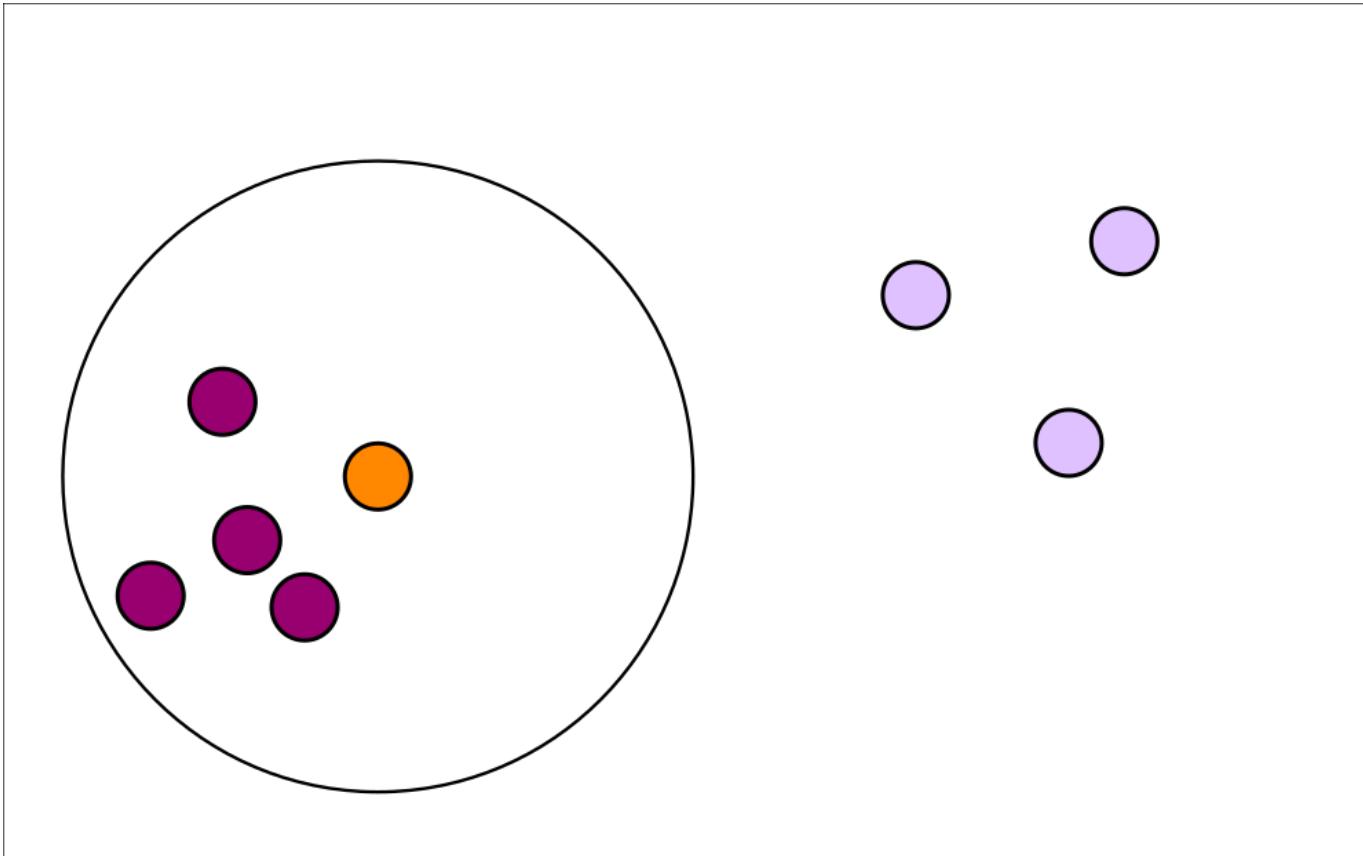
Algorithm



$\epsilon = 7.7$
 $minPts = 4$

```
OPTICS(DB, eps, MinPts)
  for each point p of DB
    p.reachability-distance = UNDEFINED
  for each unprocessed point p of DB
    N = getNeighbors(p, eps)
    mark p as processed
    output p to the ordered list
    if (core-distance(p, eps, Minpts) != UNDEFINED)
      Seeds = empty priority queue
      update(N, p, Seeds, eps, Minpts)
      for each next q in Seeds
        N' = getNeighbors(q, eps)
        mark q as processed
        output q to the ordered list
        if (core-distance(q, eps, Minpts) != UNDEFINED)
          update(N', q, Seeds, eps, Minpts)
```

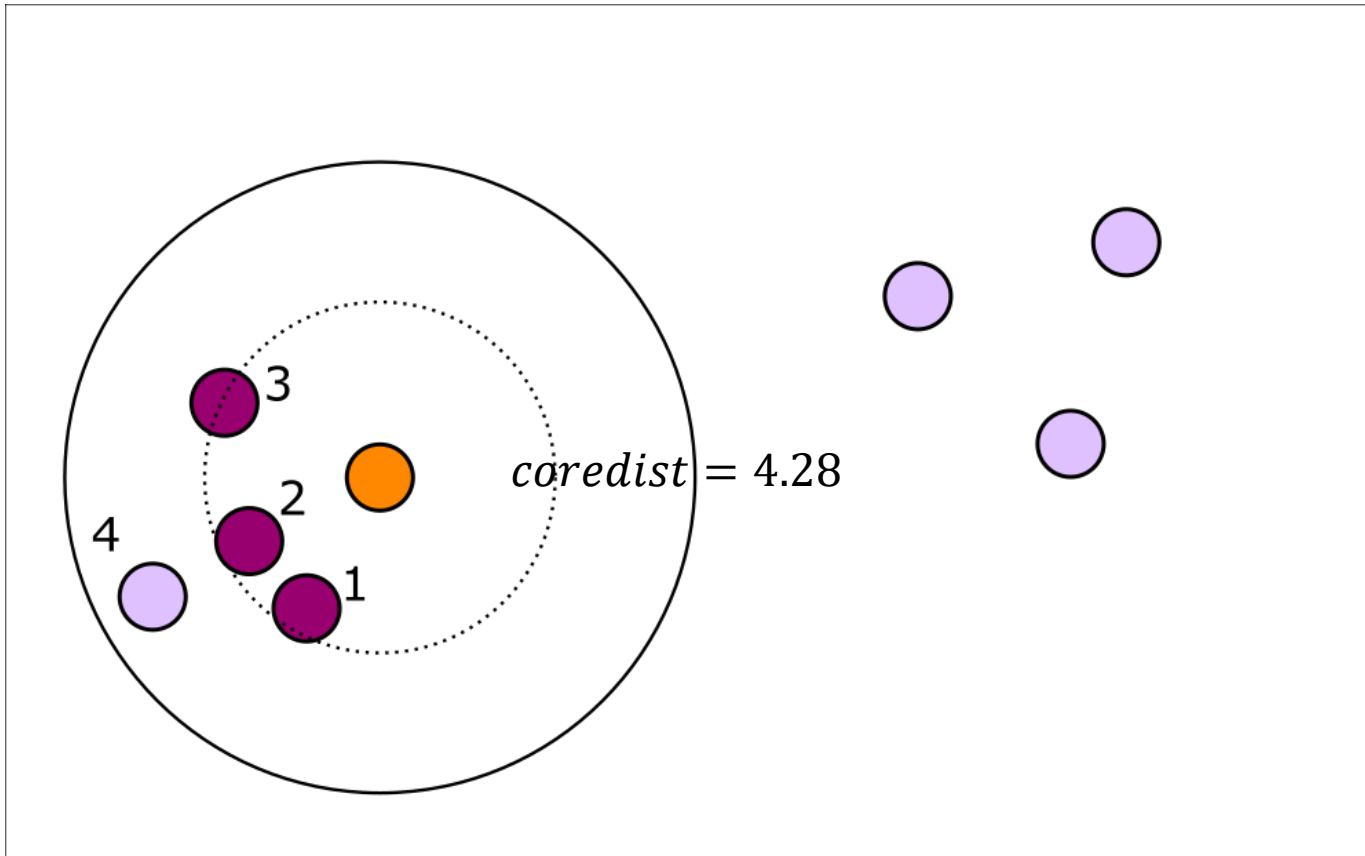
Algorithm



$\epsilon = 7.7$
 $minPts = 4$

```
OPTICS(DB, eps, MinPts)
  for each point p of DB
    p.reachability-distance = UNDEFINED
  for each unprocessed point p of DB
    N = getNeighbors(p, eps)
    mark p as processed
    output p to the ordered list
    if (core-distance(p, eps, Minpts) != UNDEFINED)
      Seeds = empty priority queue
      update(N, p, Seeds, eps, Minpts)
      for each next q in Seeds
        N' = getNeighbors(q, eps)
        mark q as processed
        output q to the ordered list
        if (core-distance(q, eps, Minpts) != UNDEFINED)
          update(N', q, Seeds, eps, Minpts)
```

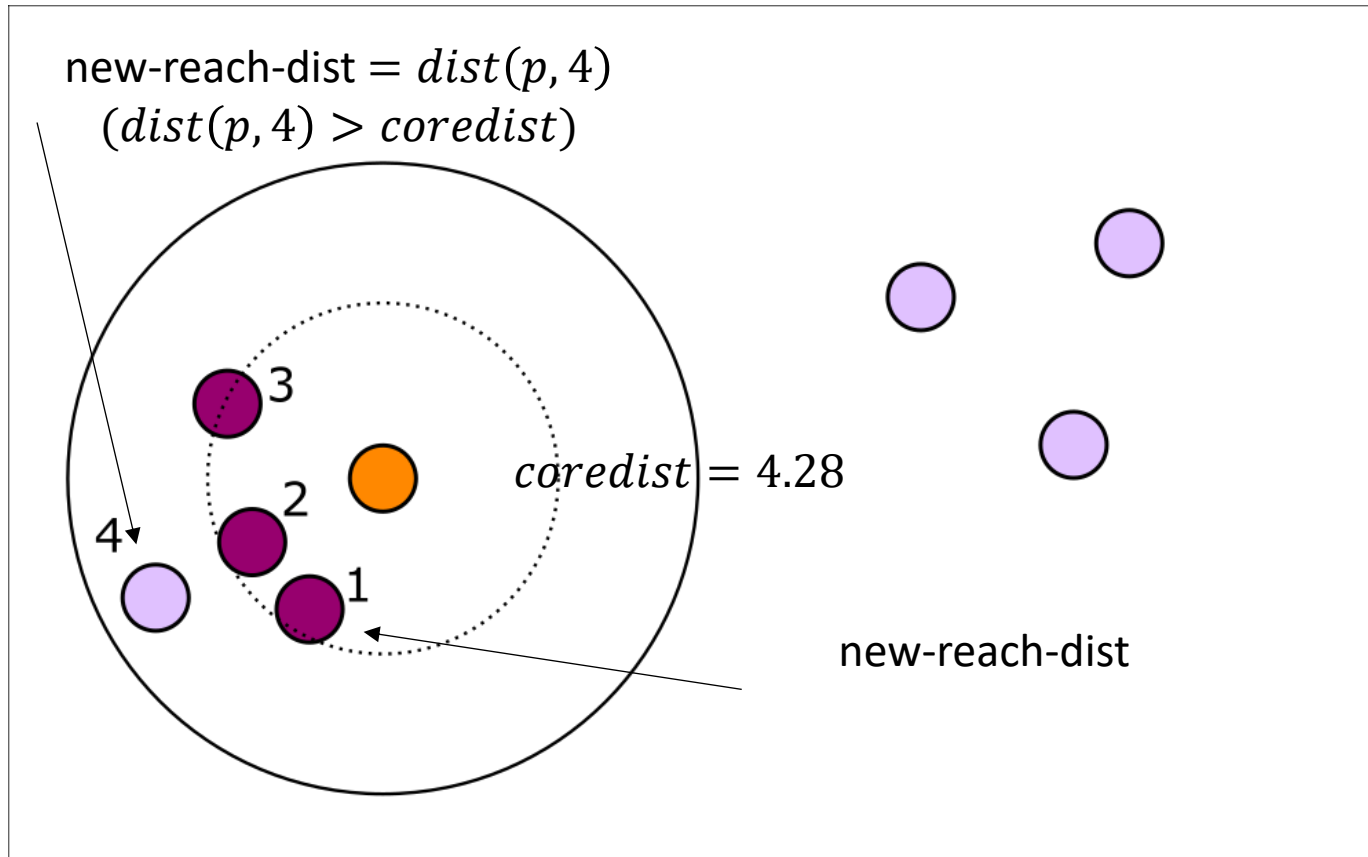
Algorithm



$\epsilon = 7.7$
 $minPts = 4$

```
OPTICS(DB, eps, MinPts)
  for each point p of DB
    p.reachability-distance = UNDEFINED
  for each unprocessed point p of DB
    N = getNeighbors(p, eps)
    mark p as processed
    output p to the ordered list
    if (core-distance(p, eps, Minpts) != UNDEFINED)
      Seeds = empty priority queue
      update(N, p, Seeds, eps, Minpts)
    for each next q in Seeds
      N' = getNeighbors(q, eps)
      mark q as processed
      output q to the ordered list
      if (core-distance(q, eps, Minpts) != UNDEFINED)
        update(N', q, Seeds, eps, Minpts)
```

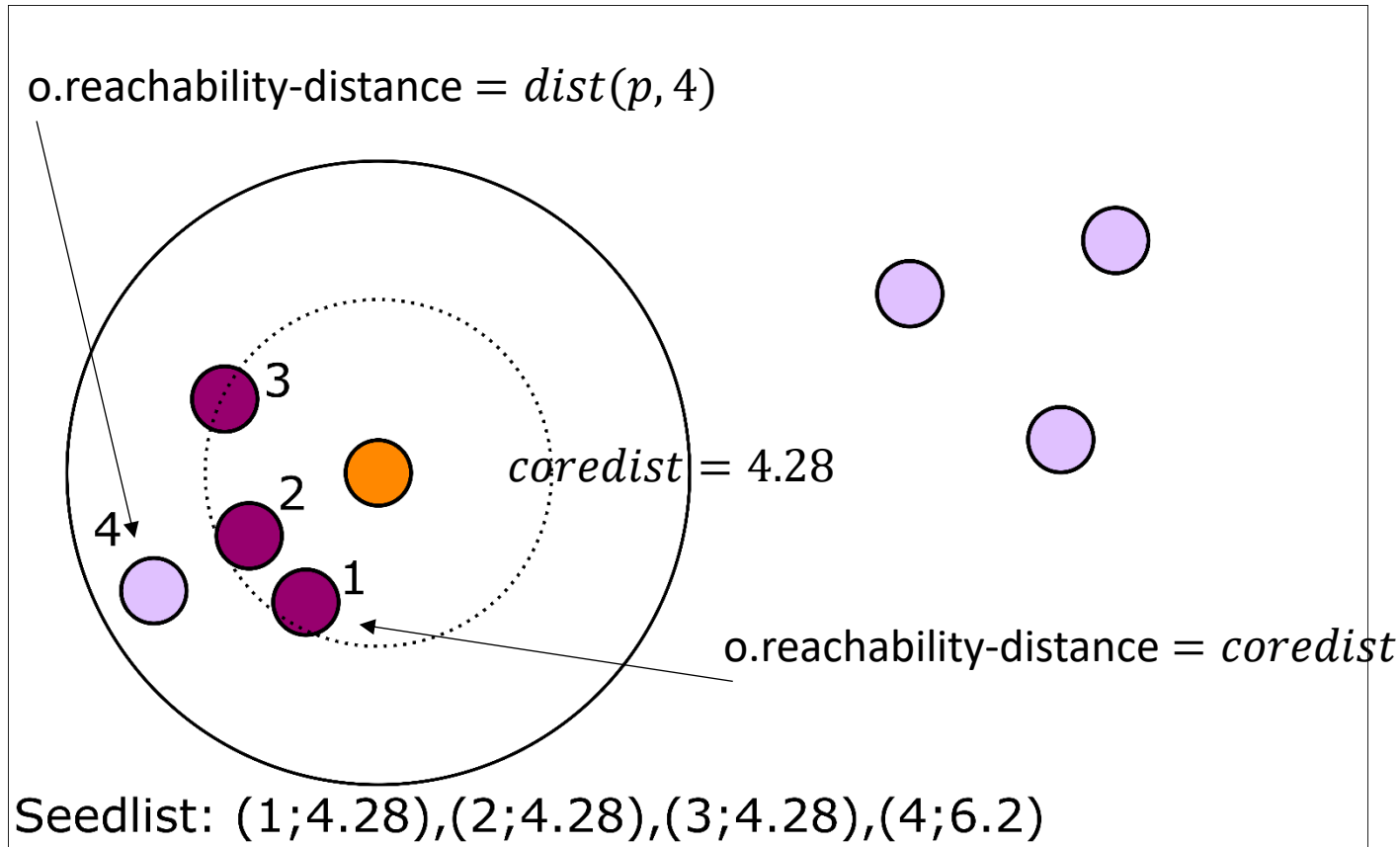

Algorithm



```
update(N, p, Seeds, eps, Minpts)
  coredist = core-distance(p, eps, MinPts)
  for each o in N
    if (o is not processed)
      new-reach-dist = max(coredist, dist(p,o))
      if (o.reachability-distance == UNDEFINED)
        // o is not in Seeds
        o.reachability-distance = new-reach-dist
        Seeds.insert(o, new-reach-dist)
      else // o in Seeds, check for improvement
        if (new-reach-dist < o.reachability-distance)
          o.reachability-distance = new-reach-dist
          Seeds.move-up(o, new-reach-dist)
```

$\varepsilon = 7.7$
 $\text{minPts} = 4$

Algorithm



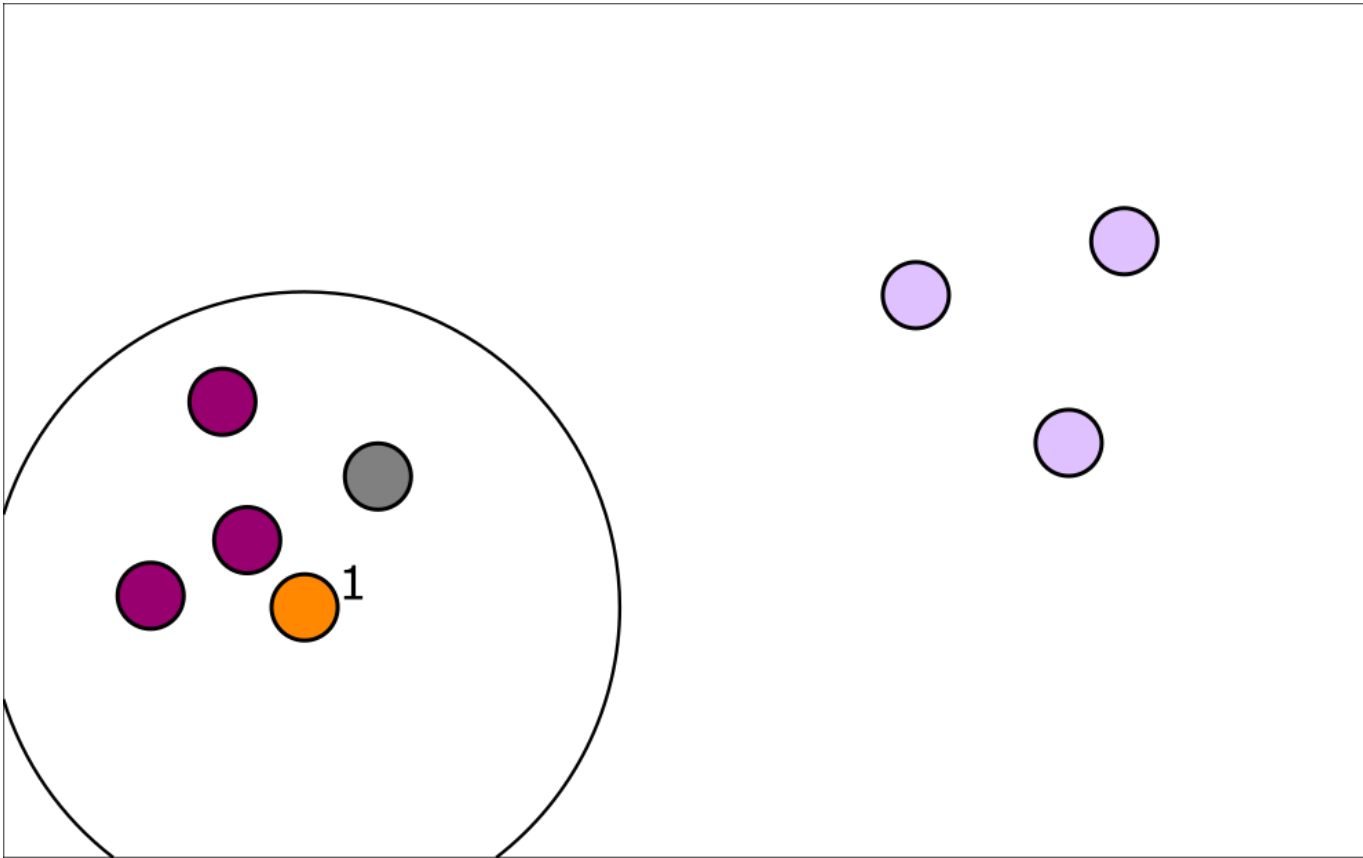
$\varepsilon = 7.7$

$\text{minPts} = 4$

```

update(N, p, Seeds, eps, Minpts)
  coredist = core-distance(p, eps, MinPts)
  for each o in N
    if (o is not processed)
      new-reach-dist = max(coredist, dist(p,o))
      if (o.reachability-distance == UNDEFINED)
        // o is not in Seeds
        o.reachability-distance = new-reach-dist
        Seeds.insert(o, new-reach-dist)
      else // o in Seeds, check for improvement
        if (new-reach-dist < o.reachability-distance)
          o.reachability-distance = new-reach-dist
          Seeds.move-up(o, new-reach-dist)
  
```

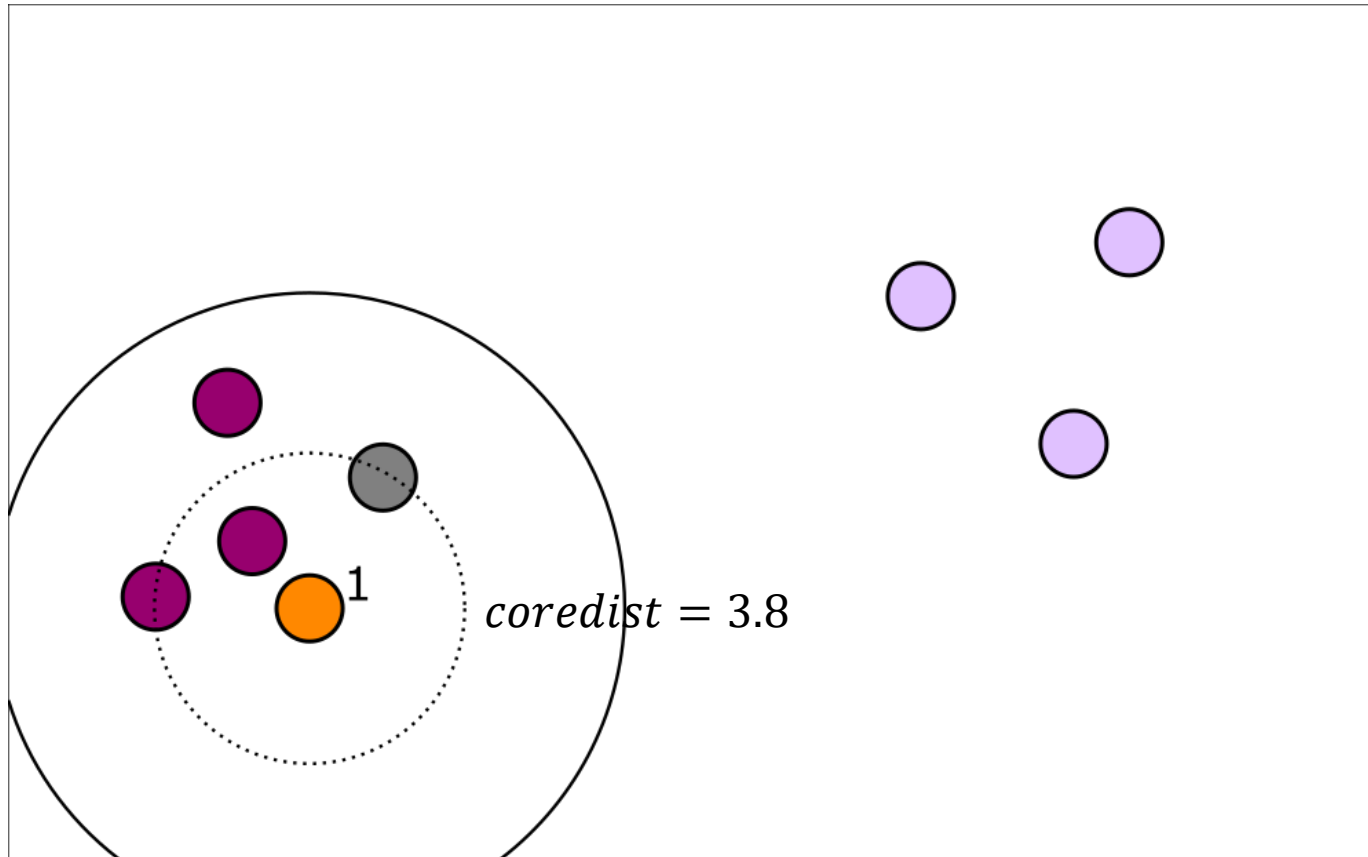
Algorithm



$\epsilon = 7.7$
 $minPts = 4$

```
OPTICS(DB, eps, MinPts)
  for each point p of DB
    p.reachability-distance = UNDEFINED
  for each unprocessed point p of DB
    N = getNeighbors(p, eps)
    mark p as processed
    output p to the ordered list
    if (core-distance(p, eps, Minpts) != UNDEFINED)
      Seeds = empty priority queue
      update(N, p, Seeds, eps, Minpts)
      for each next q in Seeds
        N' = getNeighbors(q, eps)
        mark q as processed
        output q to the ordered list
        if (core-distance(q, eps, Minpts) != UNDEFINED)
          update(N', q, Seeds, eps, Minpts)
```

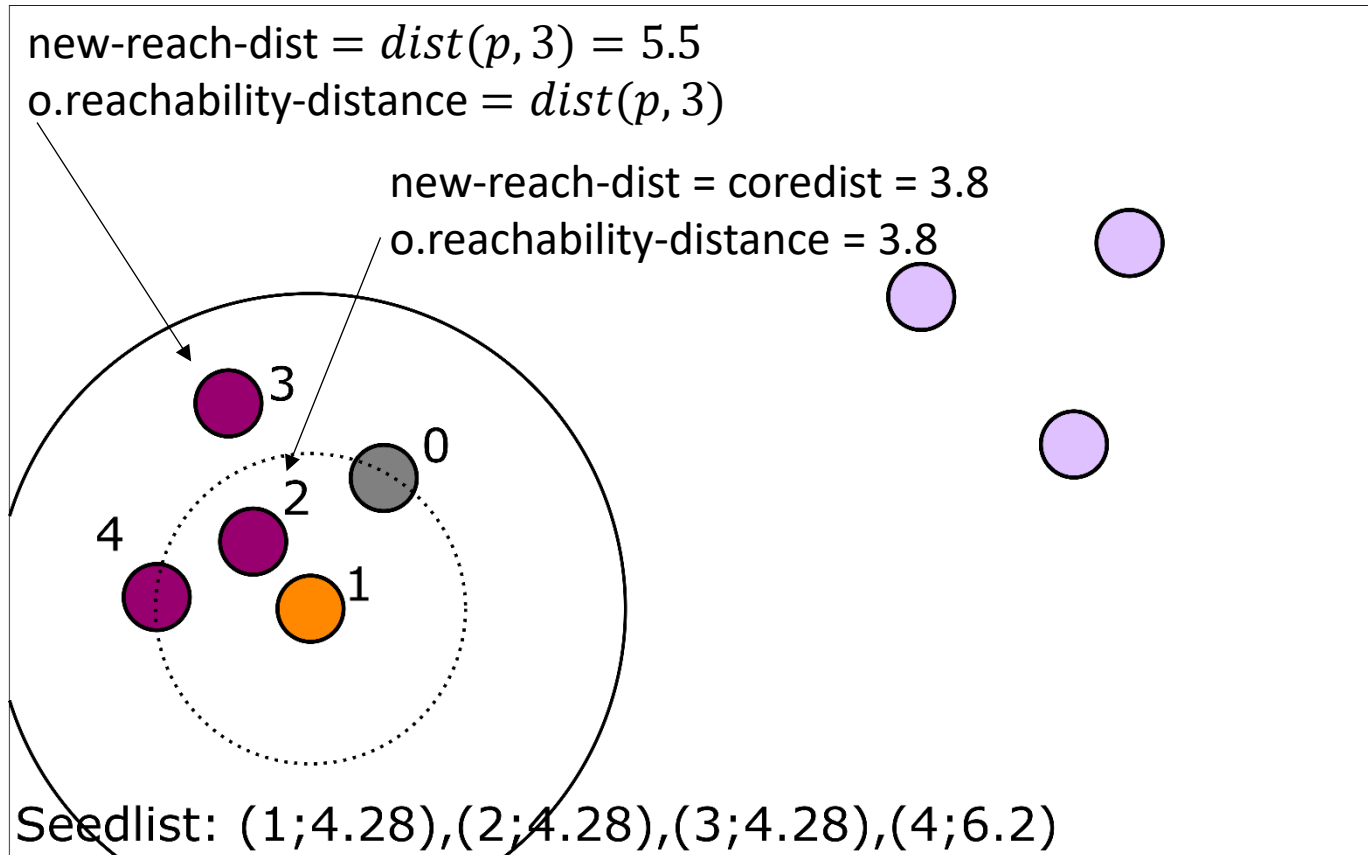
Algorithm



$\epsilon = 7.7$
 $minPts = 4$

```
OPTICS(DB, eps, MinPts)
  for each point p of DB
    p.reachability-distance = UNDEFINED
  for each unprocessed point p of DB
    N = getNeighbors(p, eps)
    mark p as processed
    output p to the ordered list
    if (core-distance(p, eps, Minpts) != UNDEFINED)
      Seeds = empty priority queue
      update(N, p, Seeds, eps, Minpts)
      for each next q in Seeds
        N' = getNeighbors(q, eps)
        mark q as processed
        output q to the ordered list
        if (core-distance(q, eps, Minpts) != UNDEFINED)
          update(N', q, Seeds, eps, Minpts)
```

Algorithm

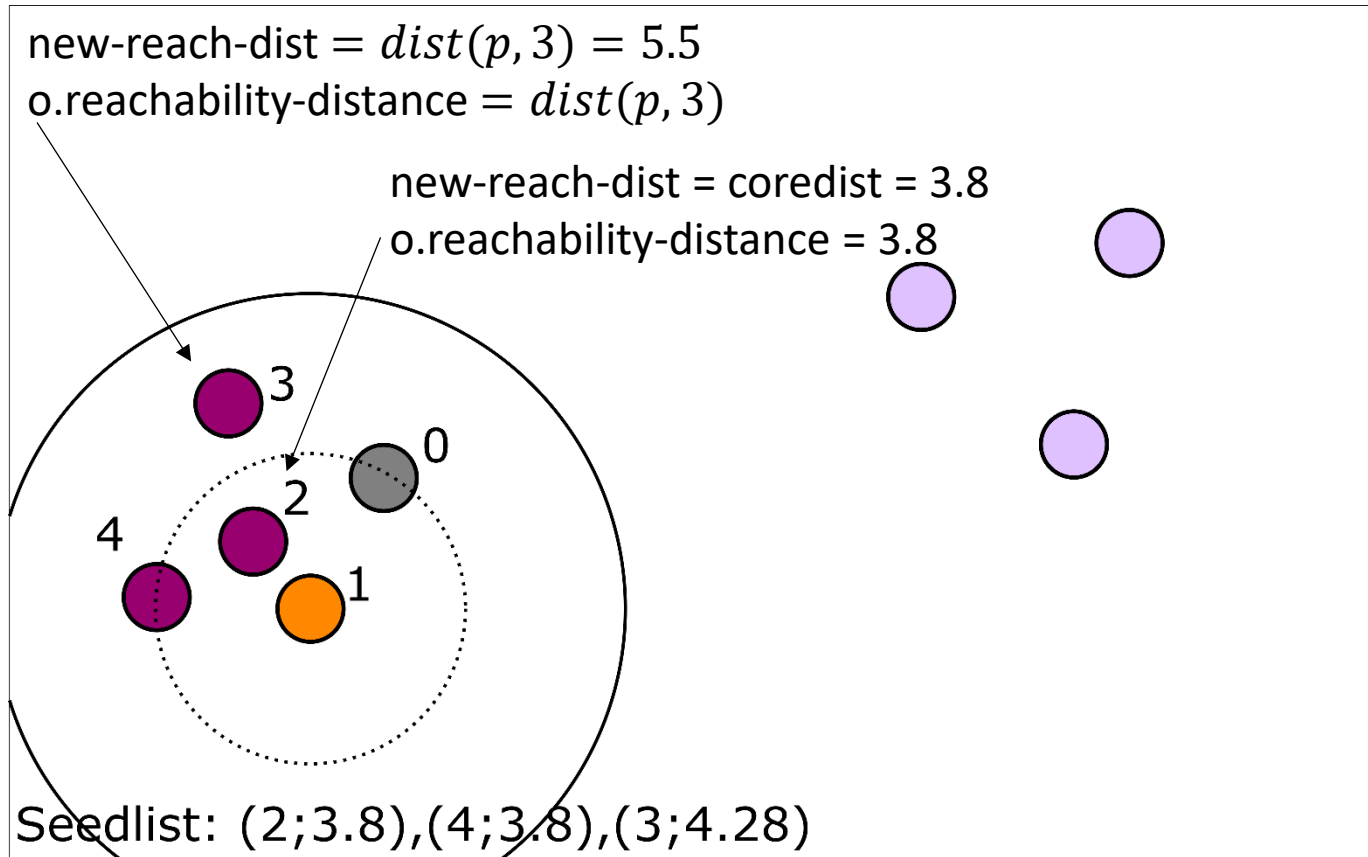


```

update(N, p, Seeds, eps, Minpts)
  coredist = core-distance(p, eps, MinPts)
  for each o in N
    if (o is not processed)
      new-reach-dist = max(coredist, dist(p,o))
      if (o.reachability-distance == UNDEFINED)
        // o is not in Seeds
        o.reachability-distance = new-reach-dist
        Seeds.insert(o, new-reach-dist)
      else // o in Seeds, check for improvement
        if (new-reach-dist < o.reachability-distance)
          o.reachability-distance = new-reach-dist
          Seeds.move-up(o, new-reach-dist)
  
```

$\varepsilon = 7.7$
 $\text{minPts} = 4$

Algorithm



$$\varepsilon = 7.7$$

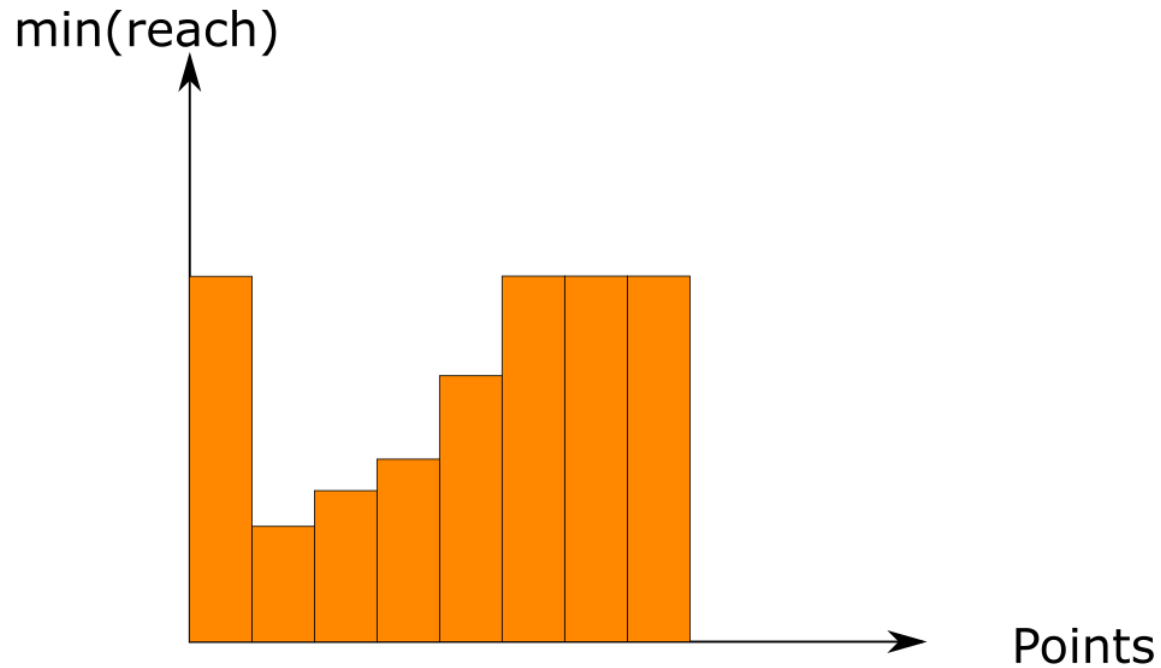
$$\text{minPts} = 4$$

```

update(N, p, Seeds, eps, Minpts)
  coredist = core-distance(p, eps, MinPts)
  for each o in N
    if (o is not processed)
      new-reach-dist = max(coredist, dist(p,o))
      if (o.reachability-distance == UNDEFINED)
        // o is not in Seeds
        o.reachability-distance = new-reach-dist
        Seeds.insert(o, new-reach-dist)
      else // o in Seeds, check for improvement
        if (new-reach-dist < o.reachability-distance)
          o.reachability-distance = new-reach-dist
          Seeds.move-up(o, new-reach-dist)
  
```

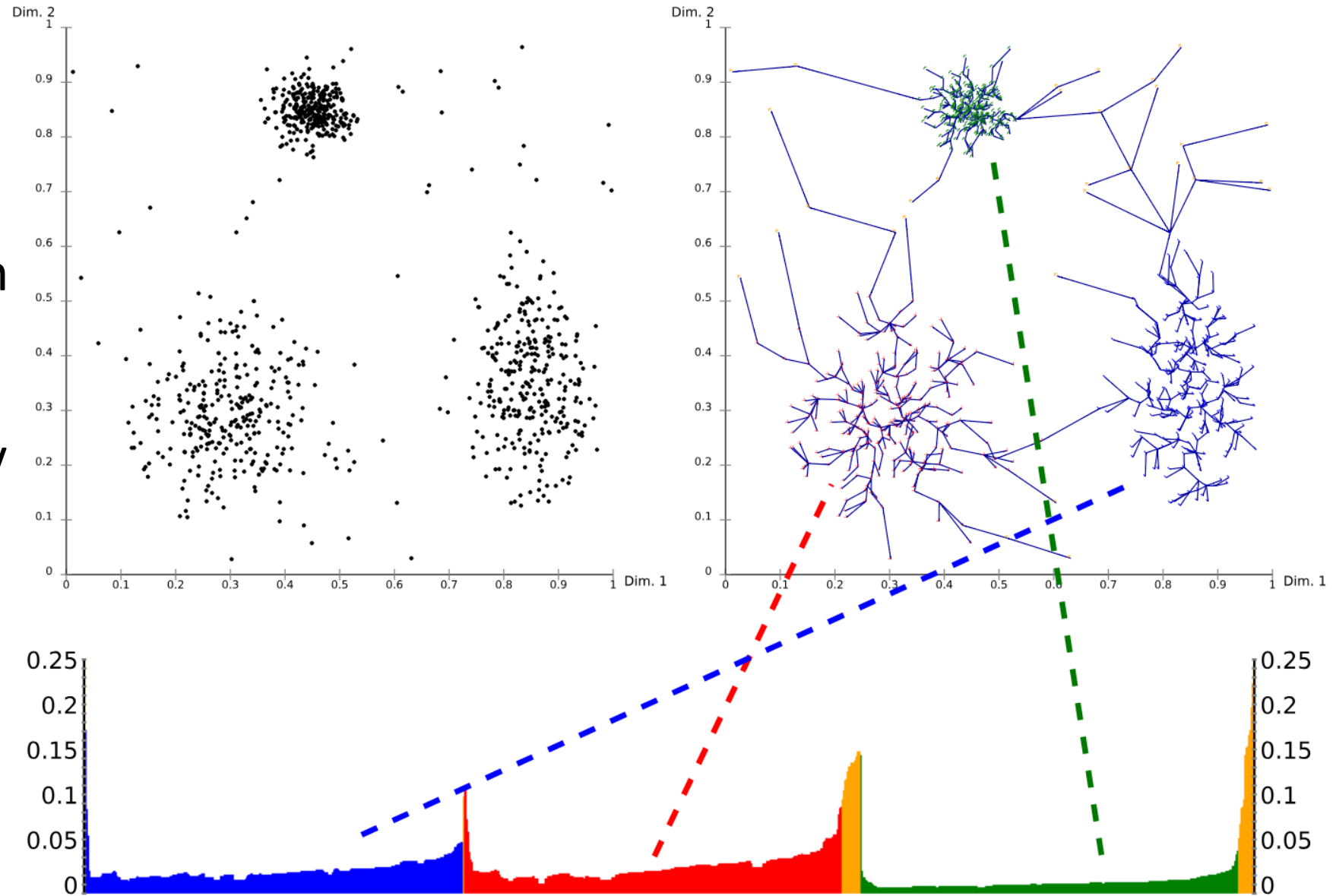
Algorithm

- The reachability-plot can be used for the final clustering



Algorithm

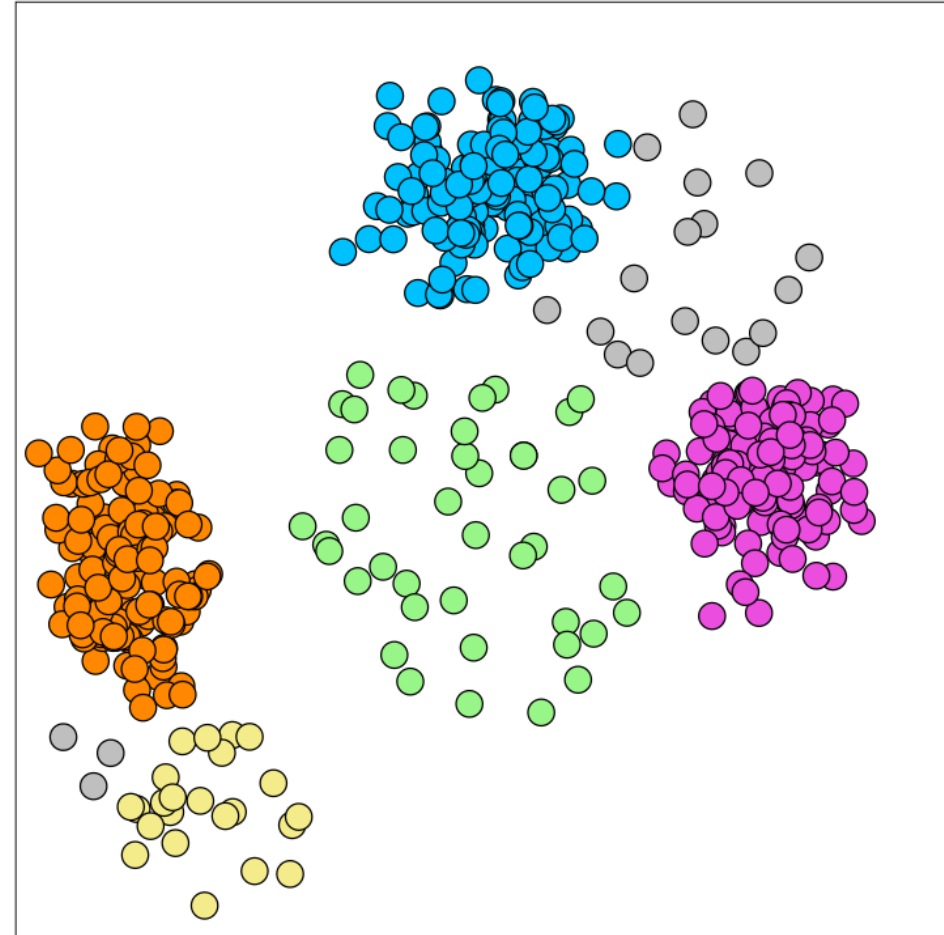
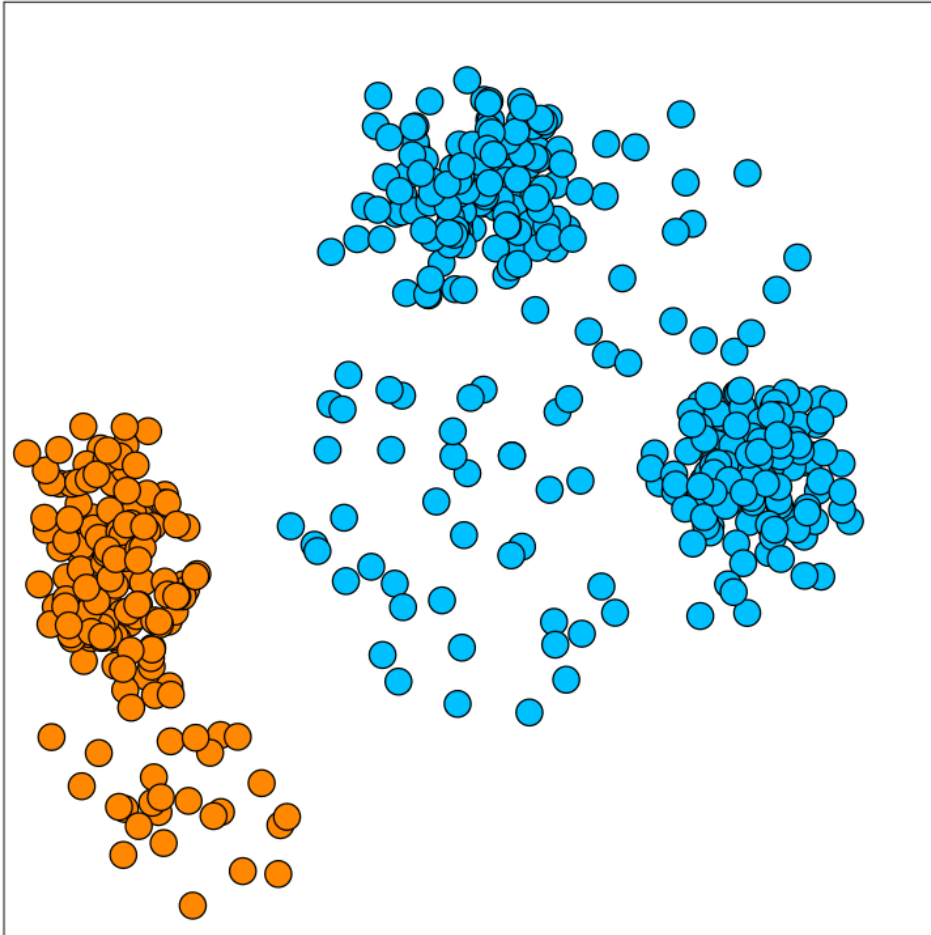
- Find the valleys in the graph either with a threshold value or manually



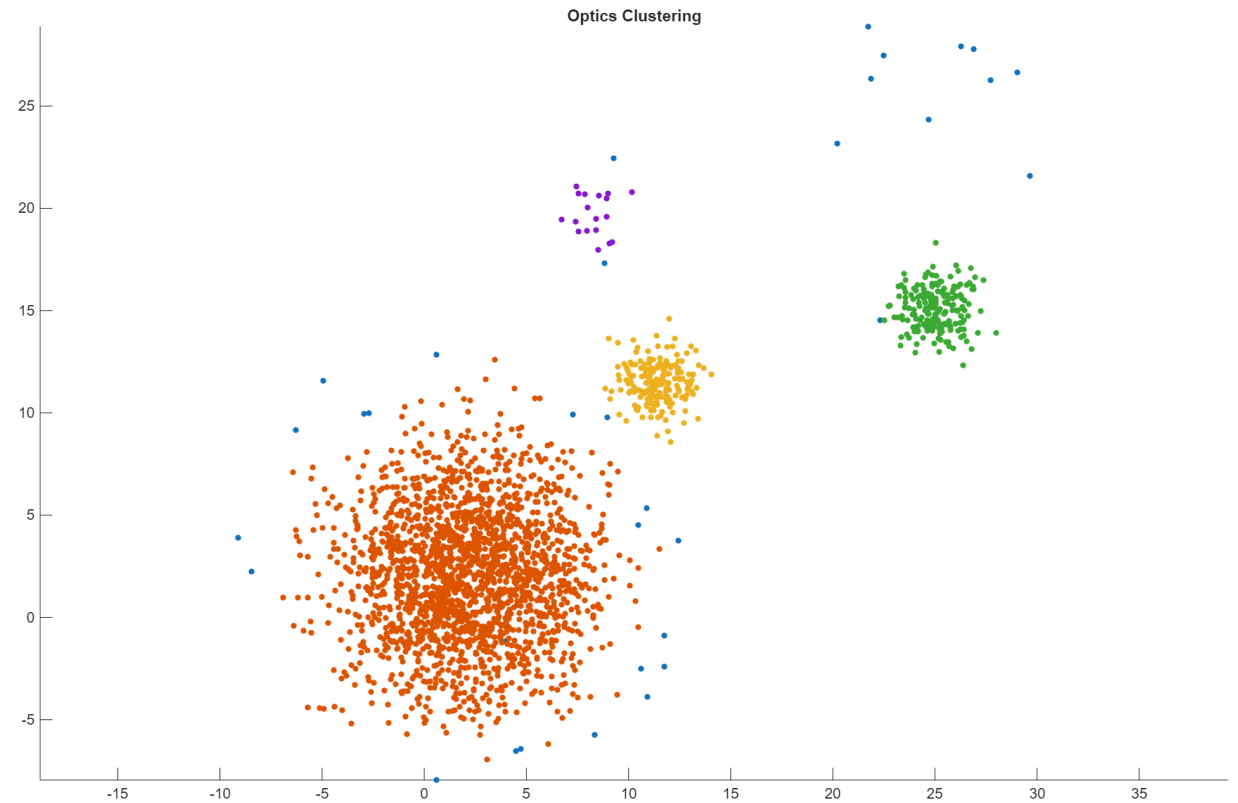
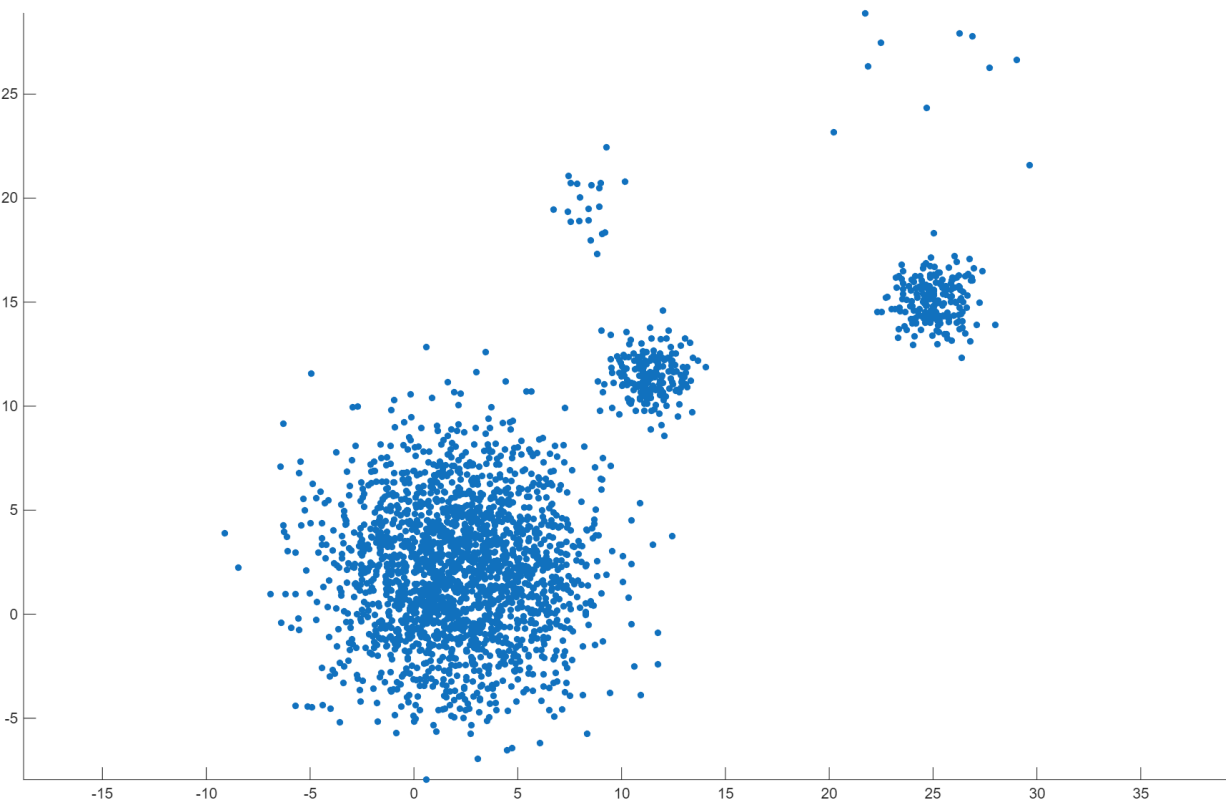
https://en.wikipedia.org/wiki/OPTICS_algorithm

Summary

- Comparison of DBSCAN left and OPTICS right



Example



$$\text{minPts} = 6, \epsilon = 1.5$$

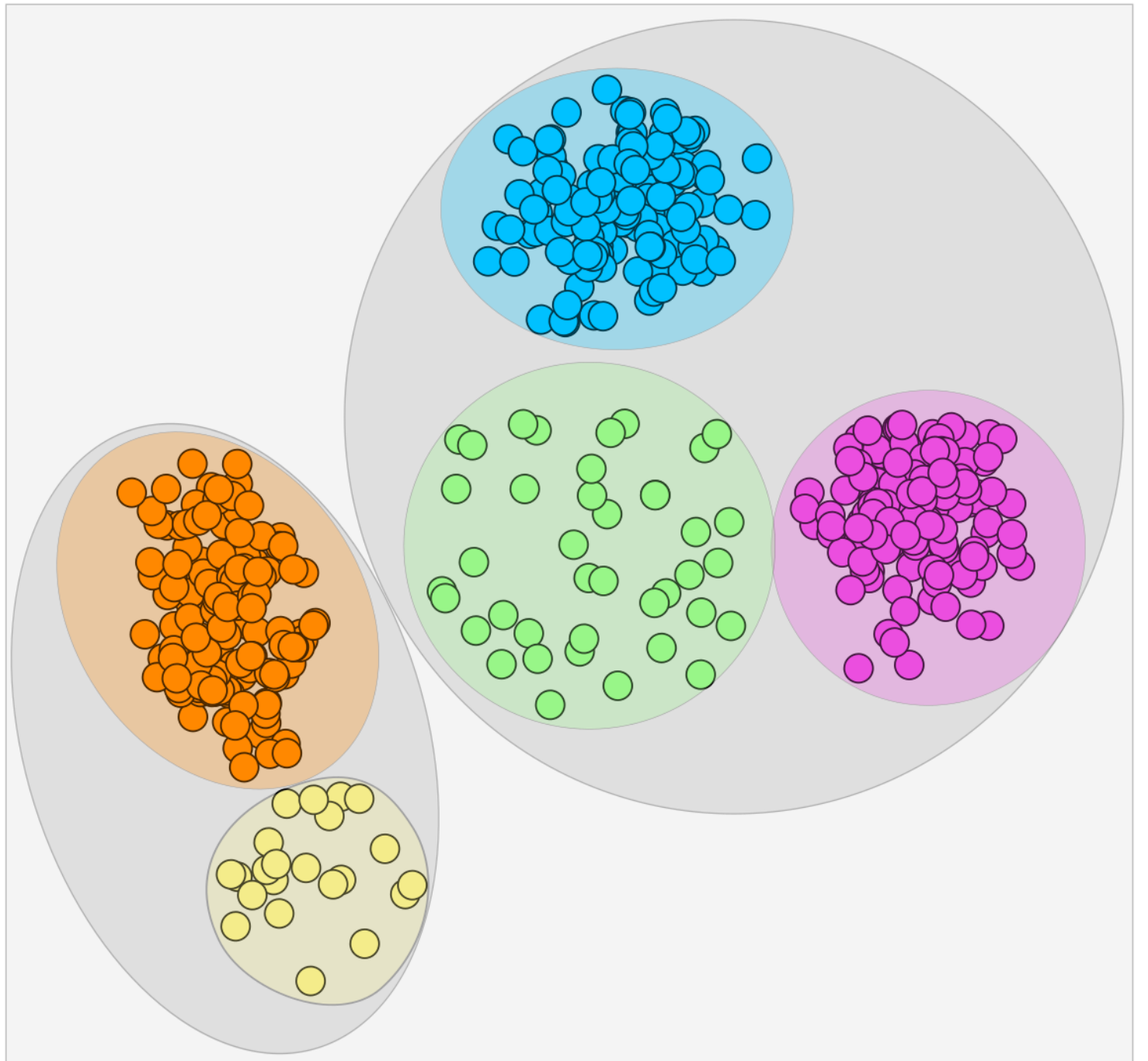
Hierarchical Clustering

Introduction

- Hierarchical clustering is a method that builds a hierarchy of clusters
- Two types:
 - Agglomerative: “bottom up”, each element starts in its own cluster, pairs are merged as one moves upwards the hierarchy
 - Divisive: “top down”, all elements start in one cluster, pairs of clusters are split as one moves downwards the hierarchy

Motivation

- Clusters that may be nested



Introduction

- This clustering depends on the choice of two important ingredients:
 - Metric
 - Linkage criteria

Metric $d(\mathbf{x}, \mathbf{y})$

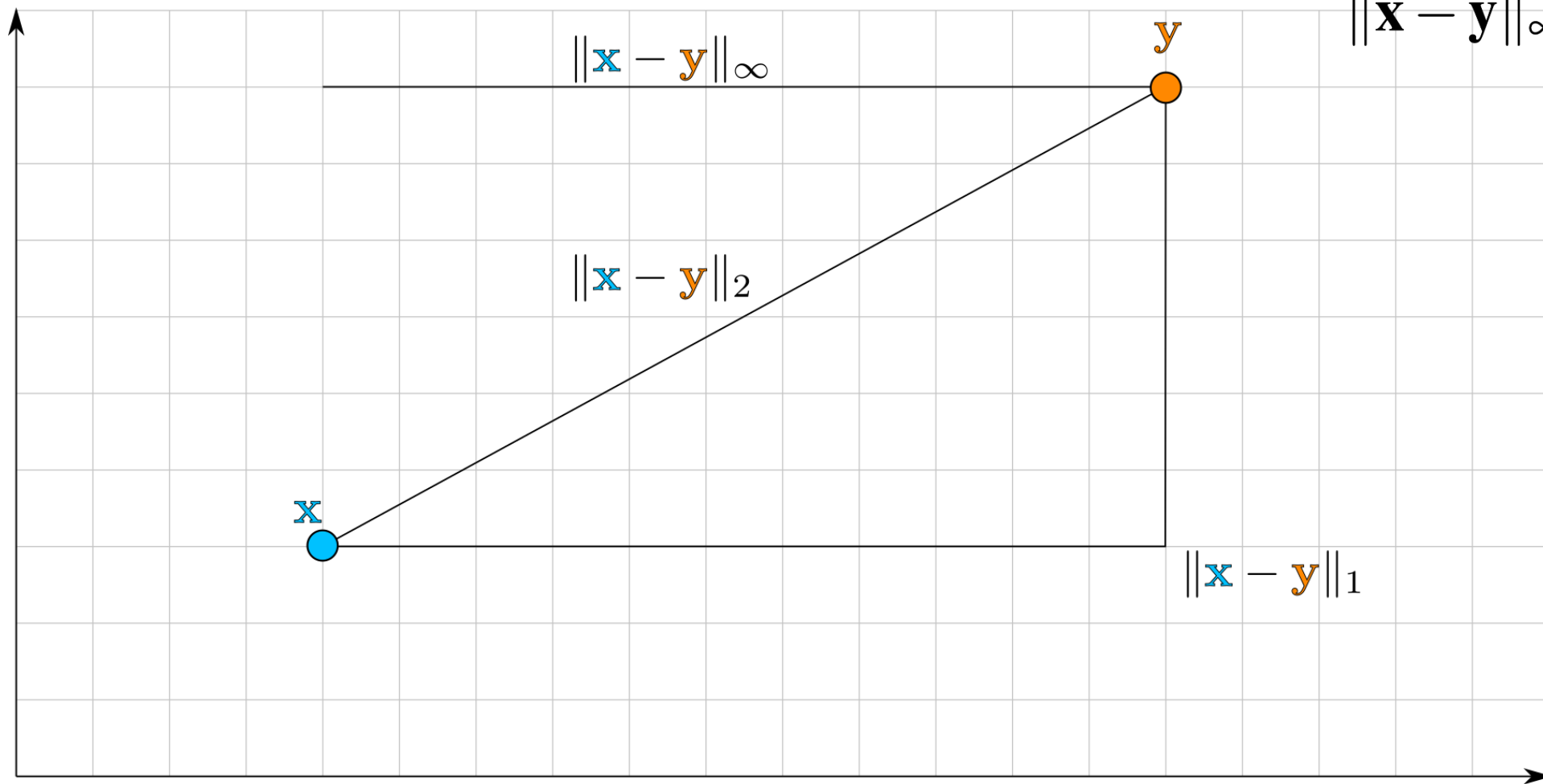
- Manhattan distance $\|\mathbf{x} - \mathbf{y}\|_1 = \sum_i |x_i - y_i|$
- Euclidean distance $\|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{\sum_i (x_i - y_i)^2}$
- Maximum distance $\|\mathbf{x} - \mathbf{y}\|_\infty = \max_i |x_i - y_i|$
- Mahalanobis distance $\|\mathbf{x} - \mathbf{y}\|_M = \sqrt{(\mathbf{x} - \mathbf{y})^T S^{-1} (\mathbf{x} - \mathbf{y})}$
(S covariance matrix)

Metric $d(\mathbf{x}, \mathbf{y})$

$$\|\mathbf{x} - \mathbf{y}\|_1 = \sum_i |x_i - y_i|$$

$$\|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{\sum_i (x_i - y_i)^2}$$

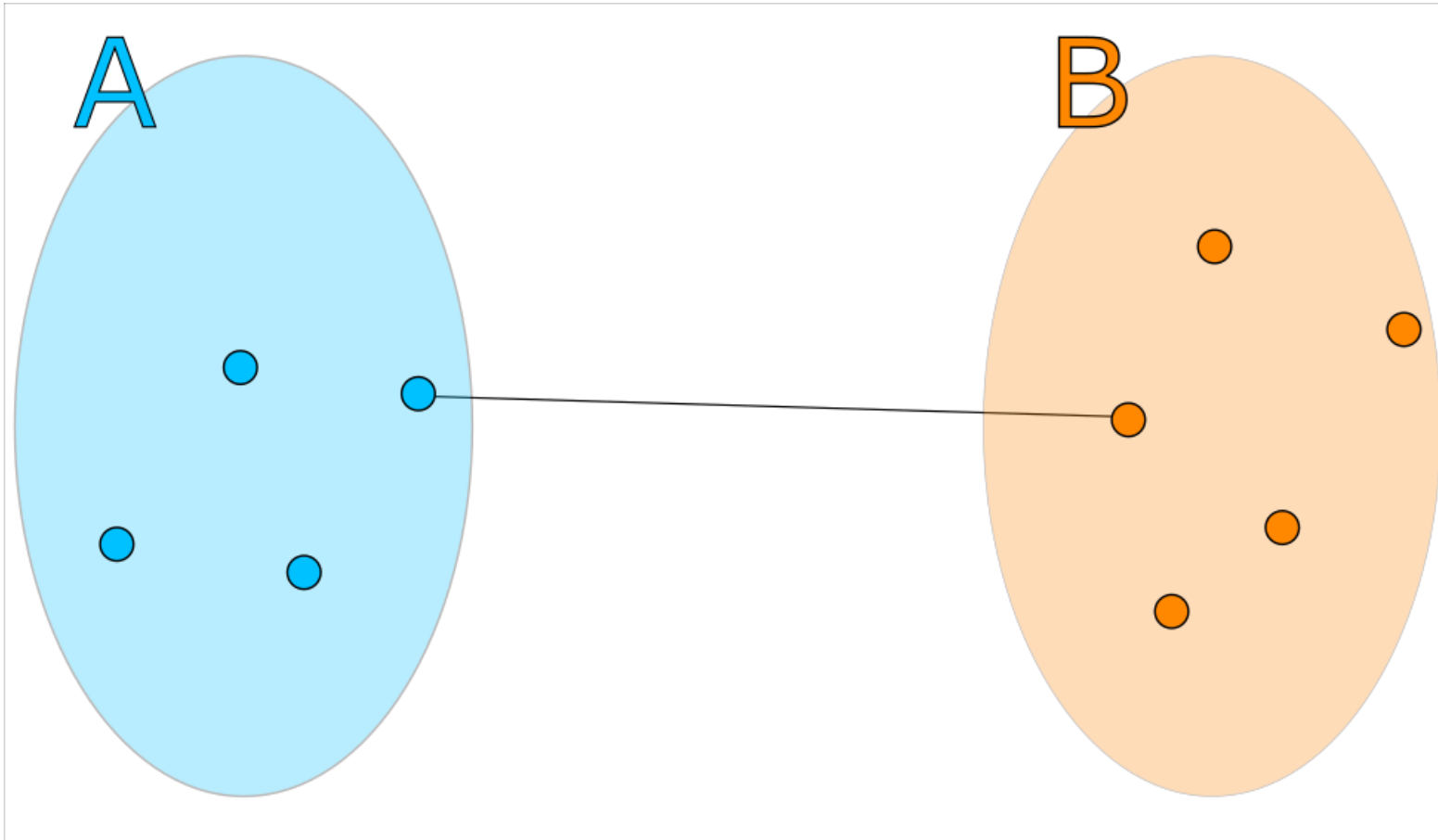
$$\|\mathbf{x} - \mathbf{y}\|_\infty = \max_i |x_i - y_i|$$



Linkage

- Single-Linkage (minimum)

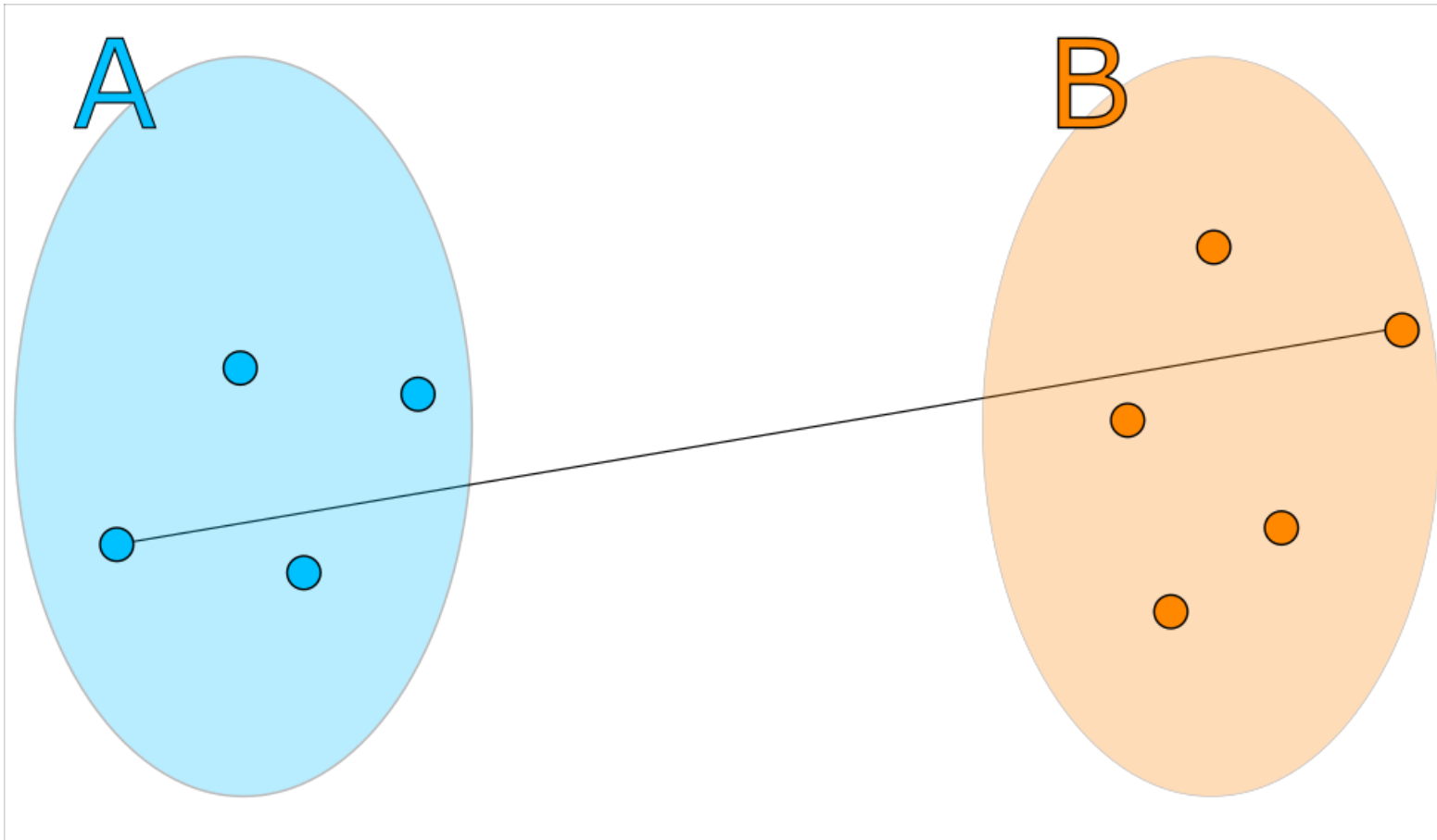
$$D_s(A, B) = \min_{a \in A, b \in B} d(a, b)$$



Linkage

- Complete-Linkage (maximum)

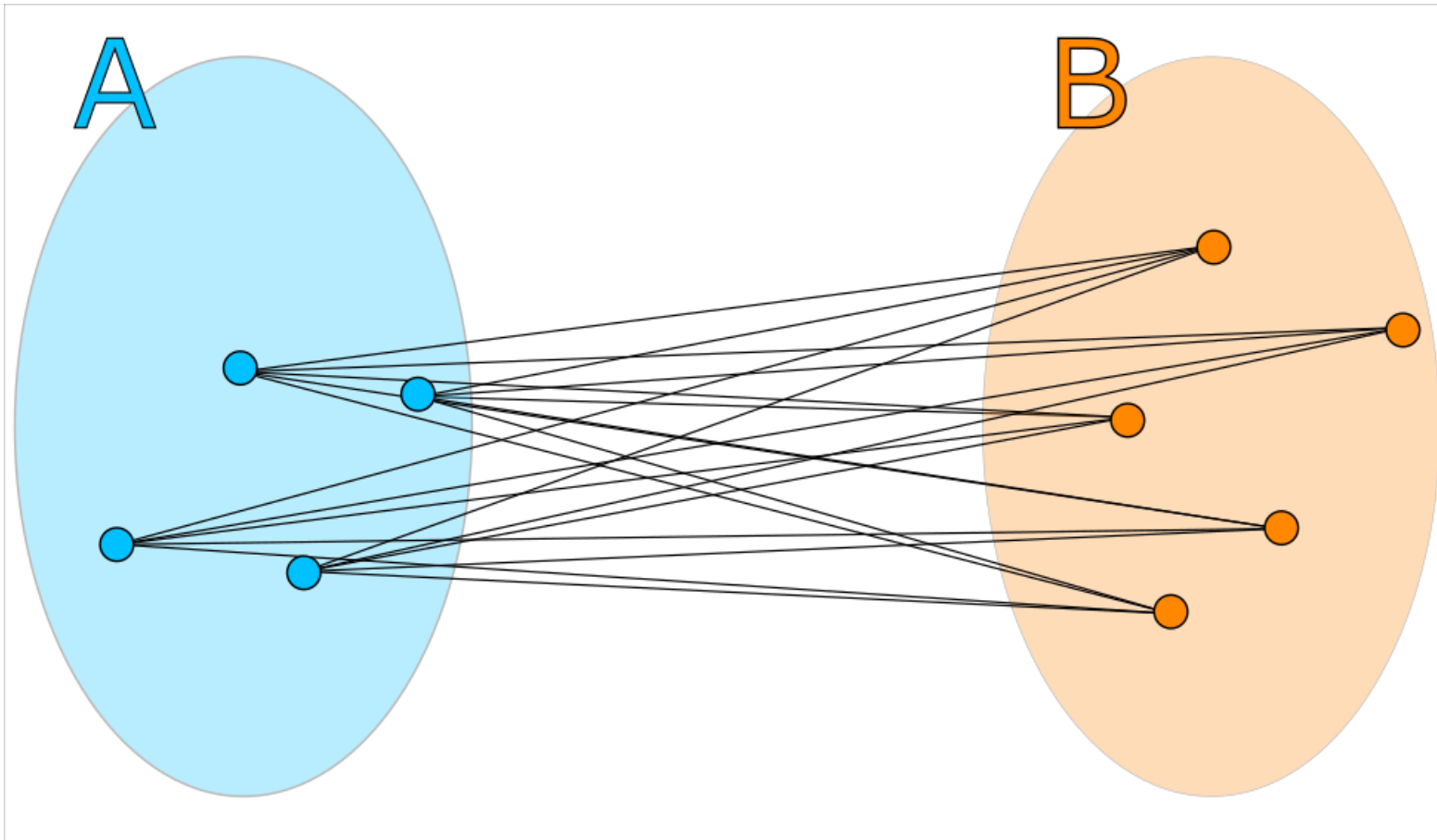
$$D_c(A, B) = \max_{a \in A, b \in B} d(a, b)$$



Linkage

- Average-Linkage

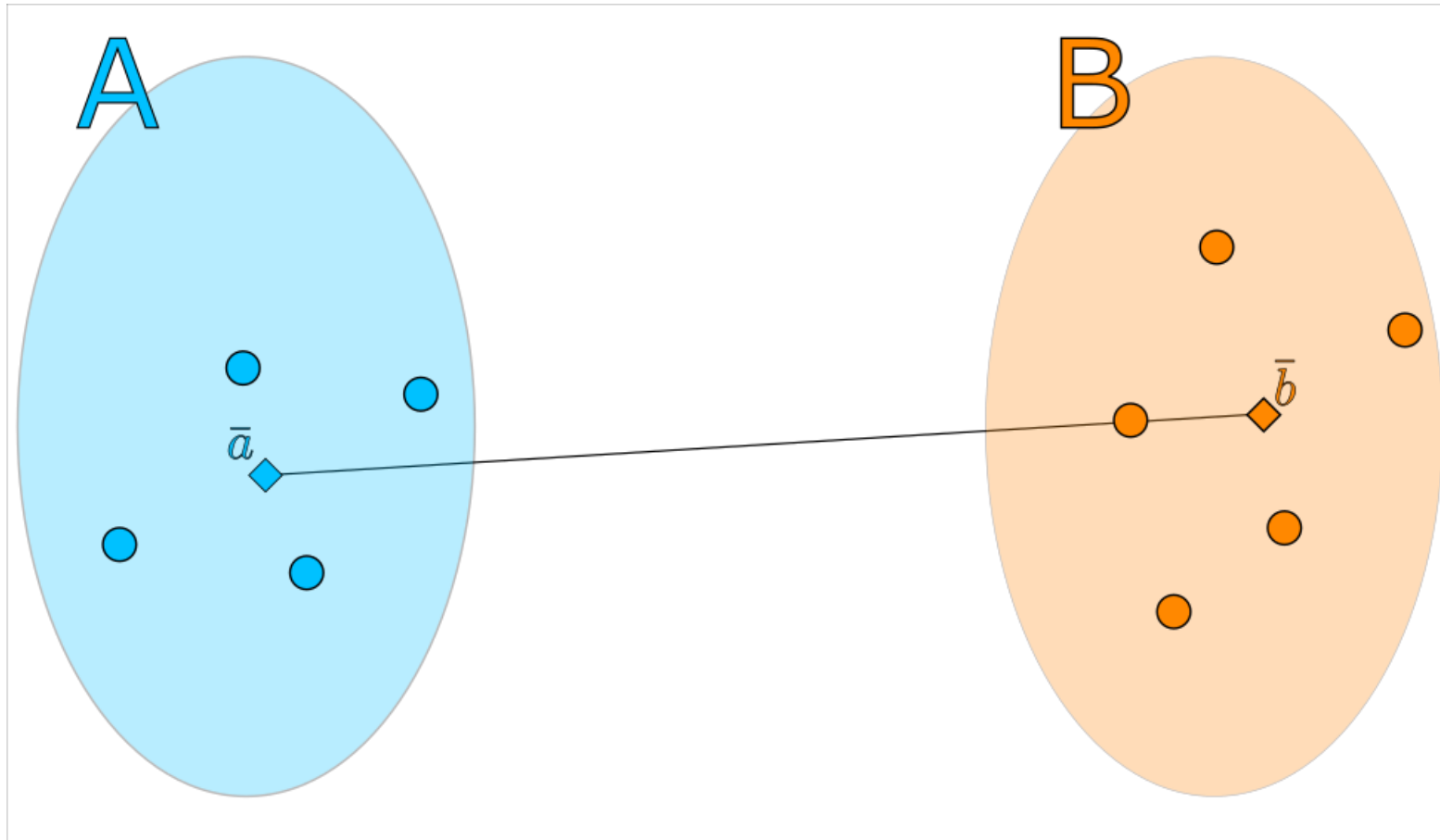
$$D_a(A, B) = \frac{1}{|A||B|} \sum_{a \in A, b \in B} d(a, b)$$



Linkage

- Ward's minimum variance

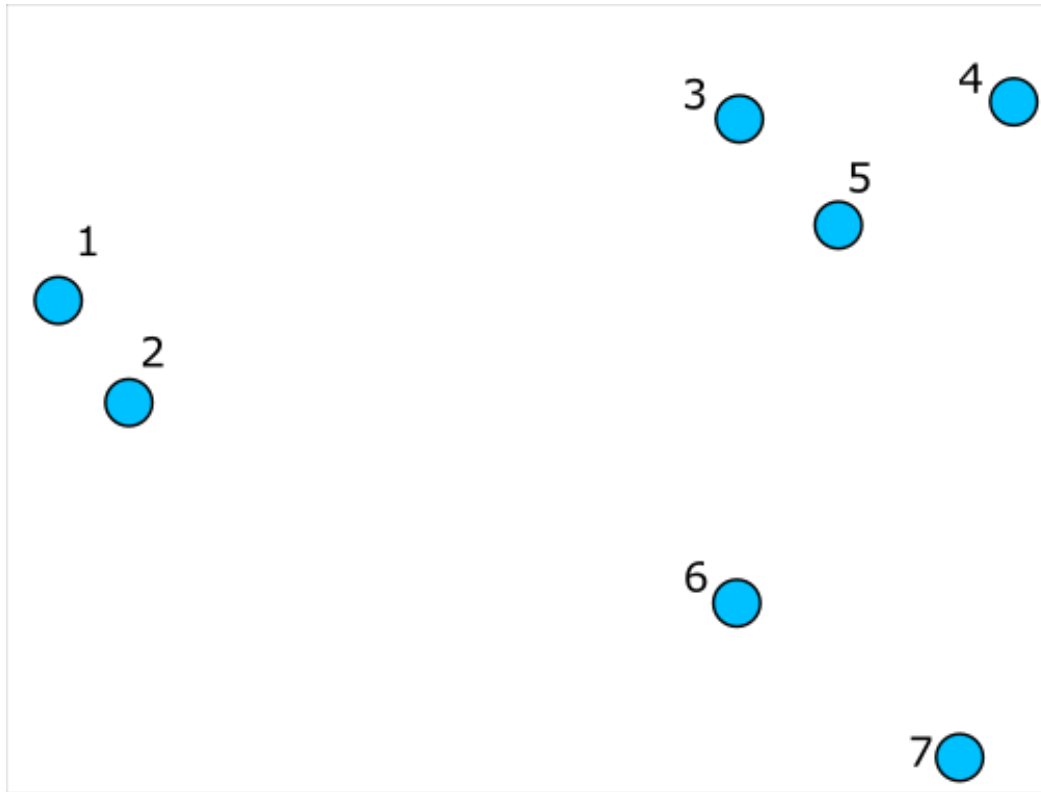
$$D_W(A, B) = \frac{d(\bar{a}, \bar{b})^2}{1/|A| + 1/|B|}$$



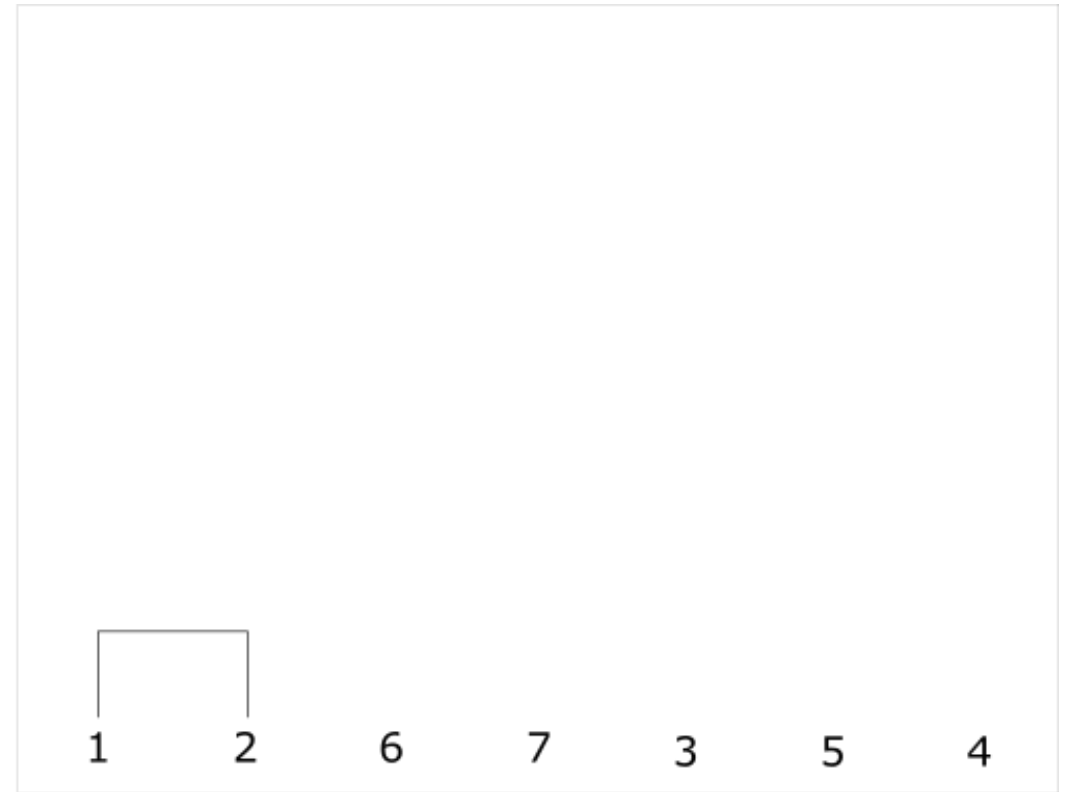
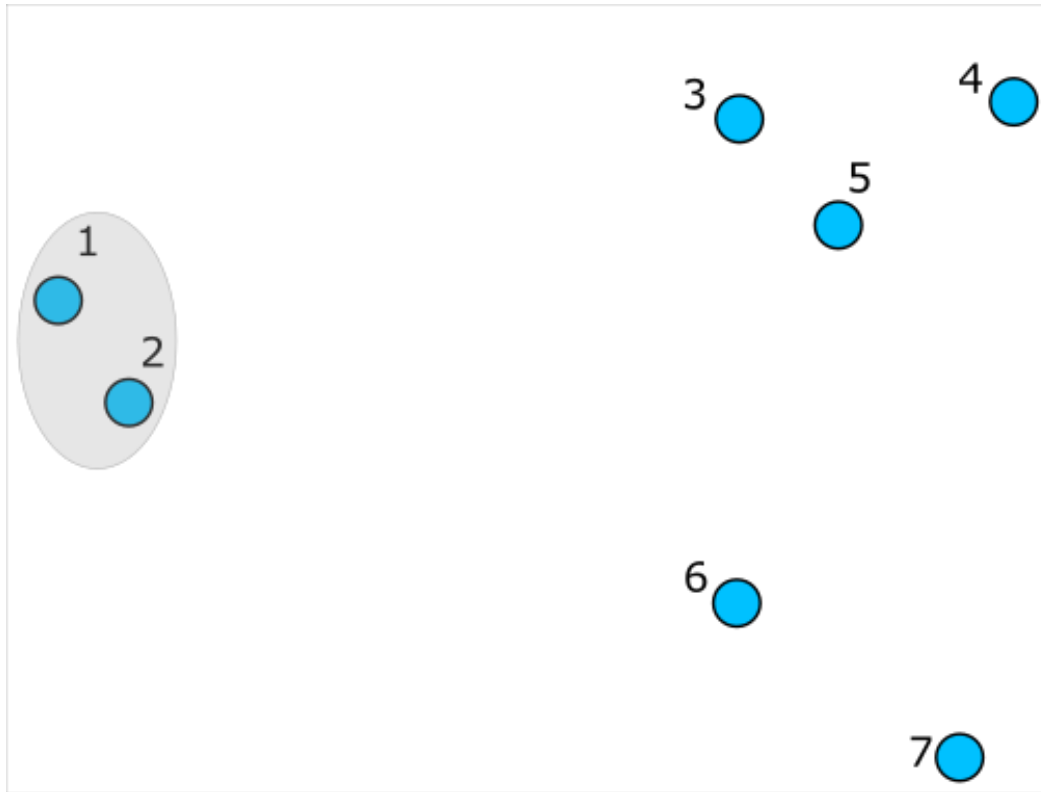
Algorithm (Agglomerative)

- Every element starts as a cluster
- Merge cluster according to the smallest linkage value

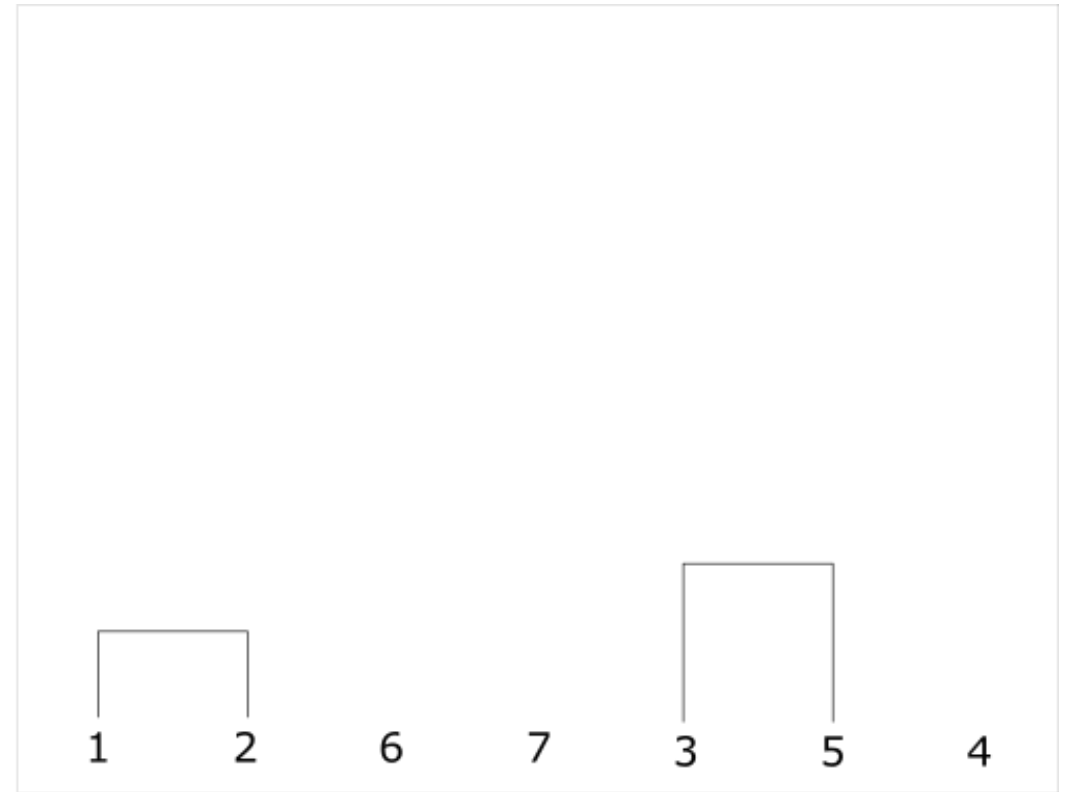
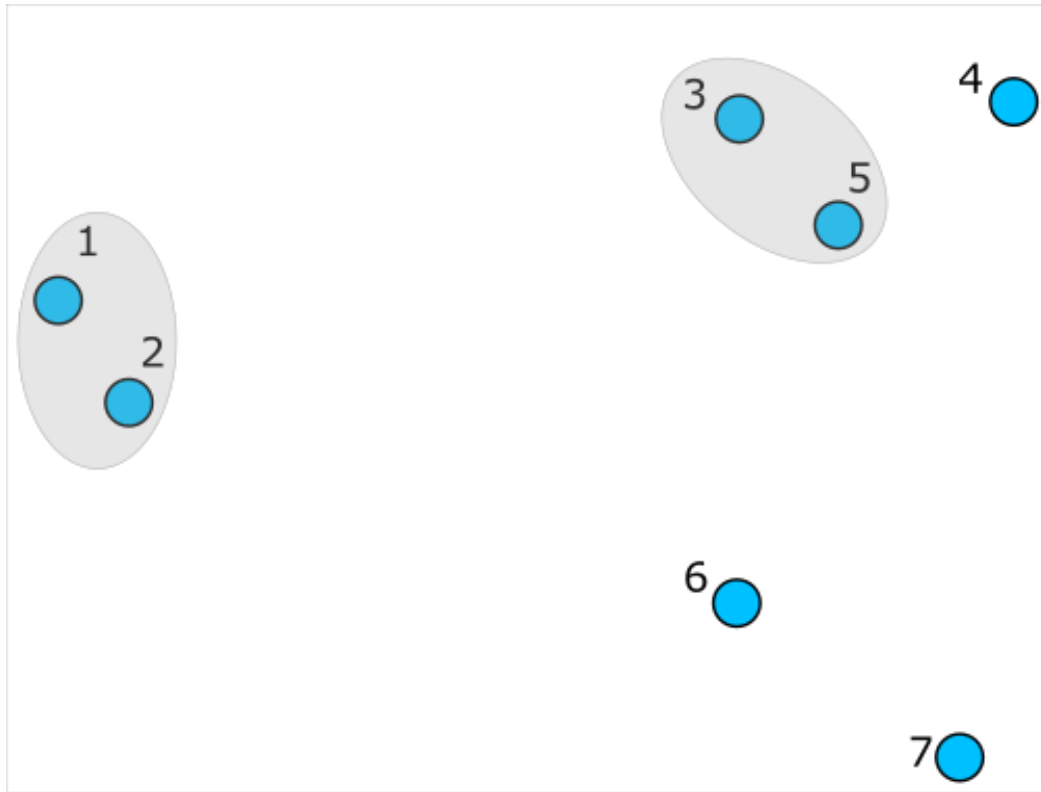
Algorithm (Agglomerative)



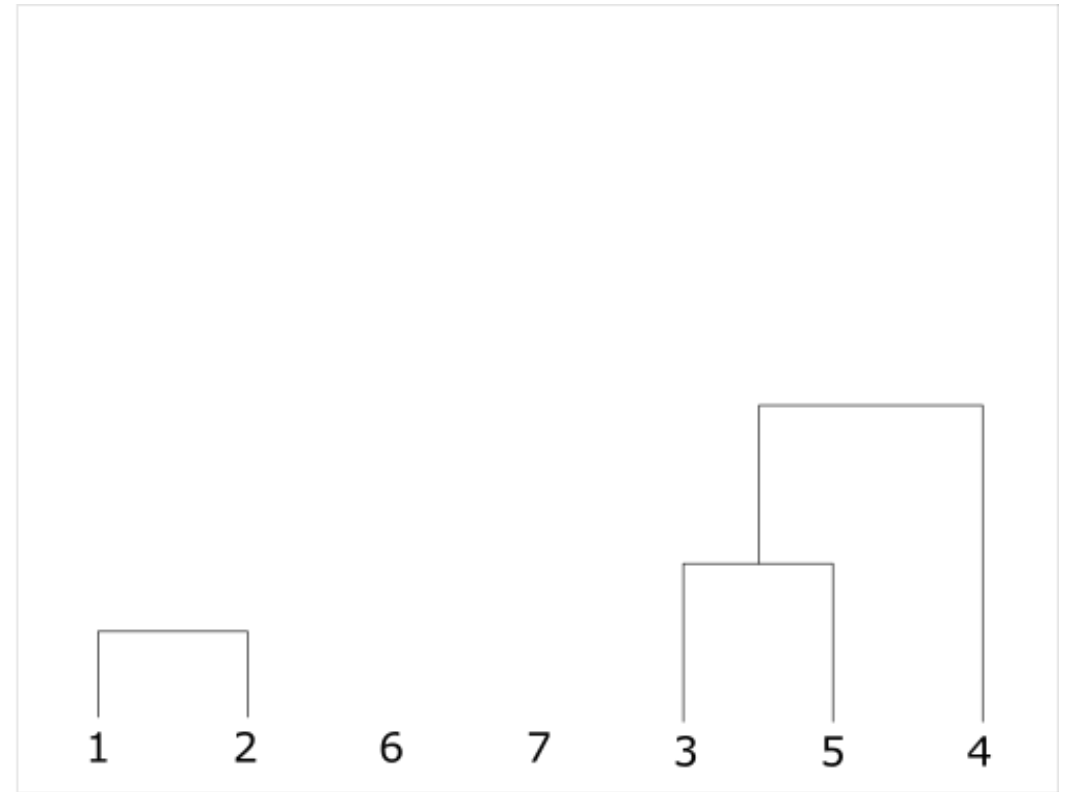
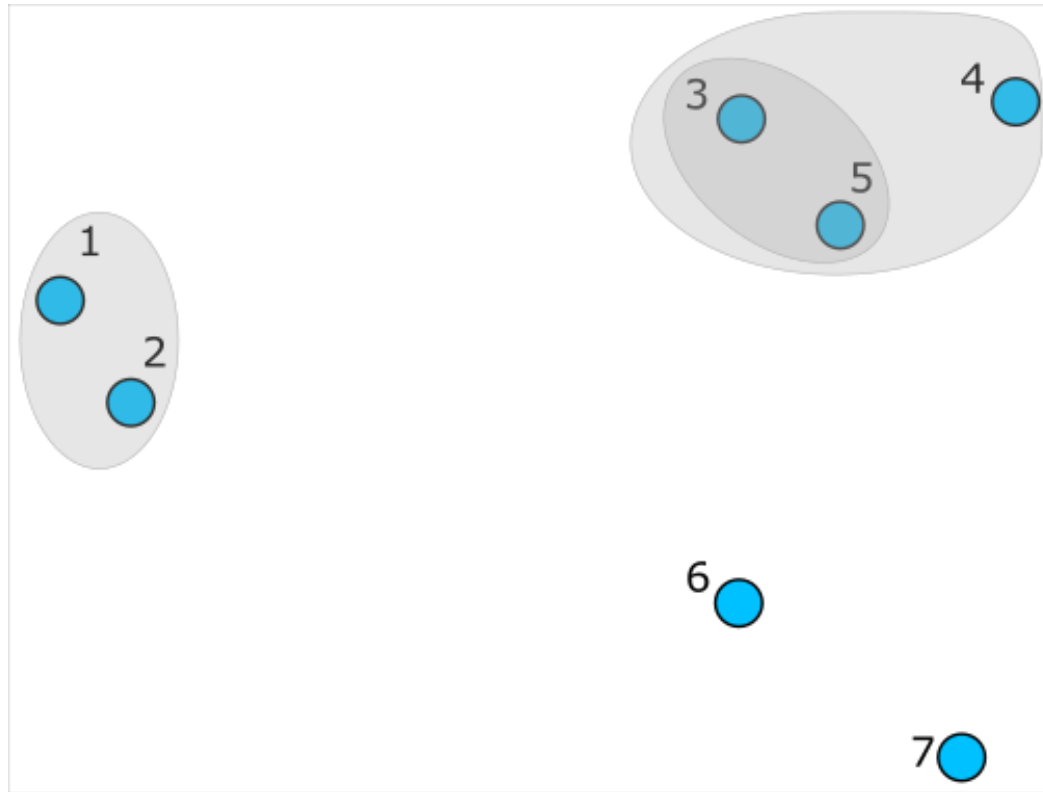
Algorithm (Agglomerative)



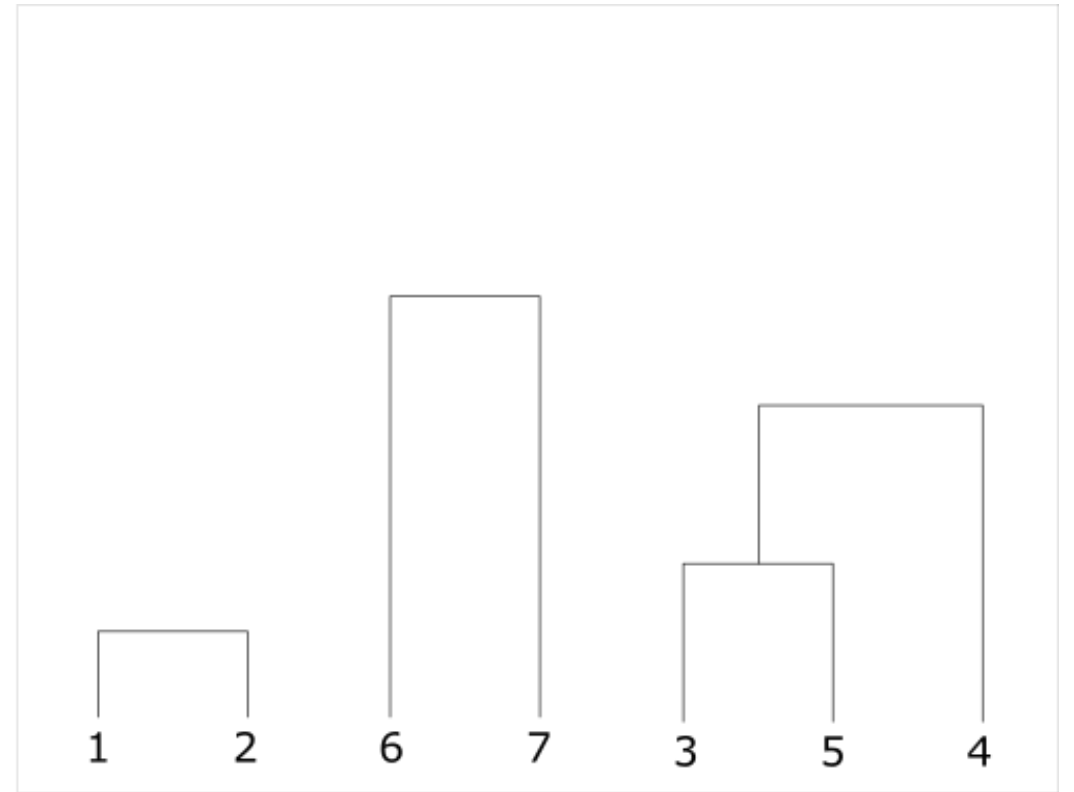
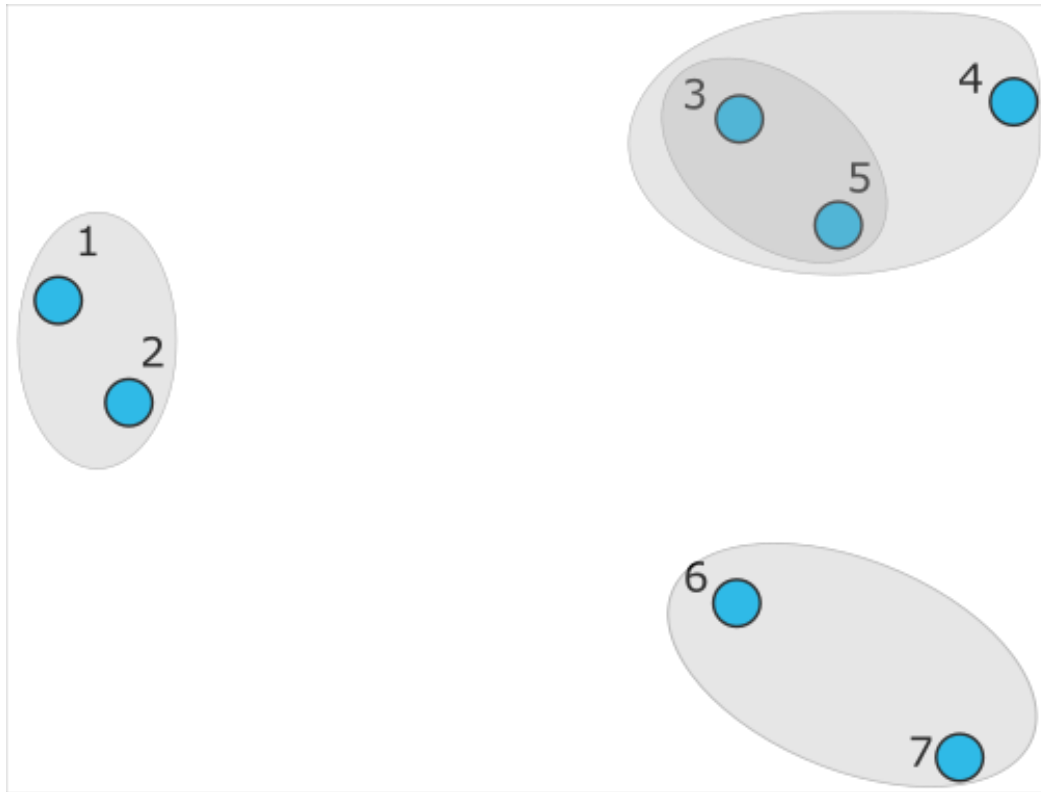
Algorithm (Agglomerative)



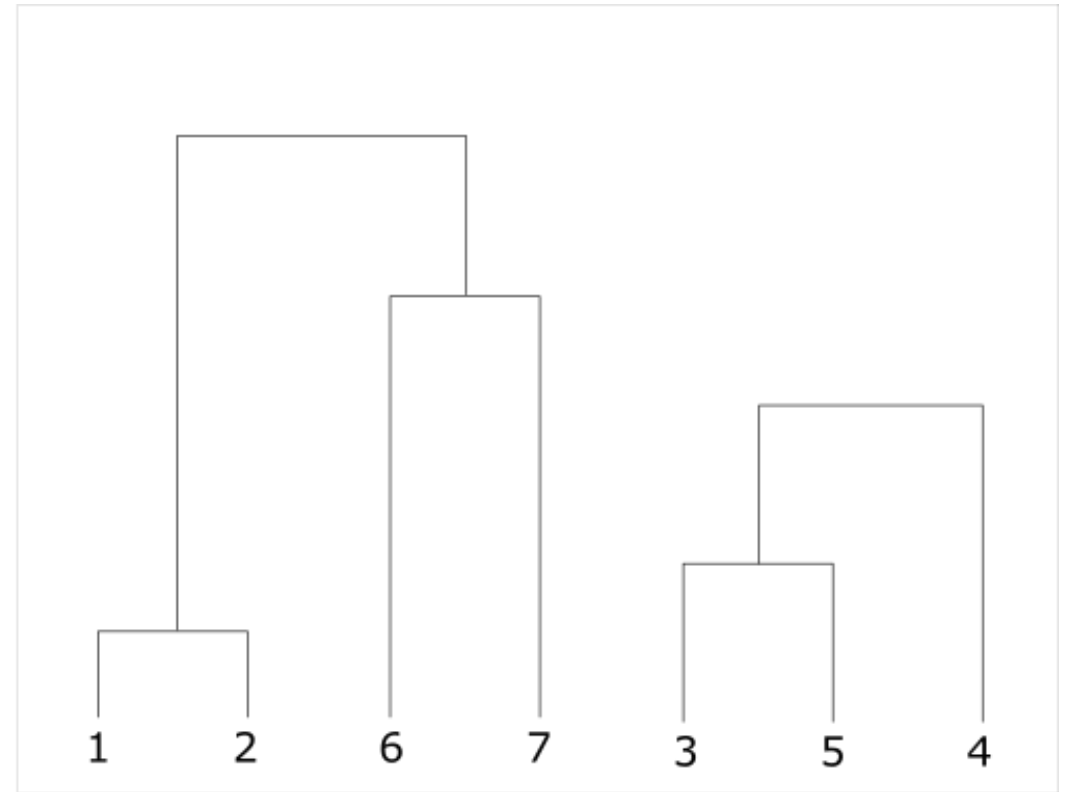
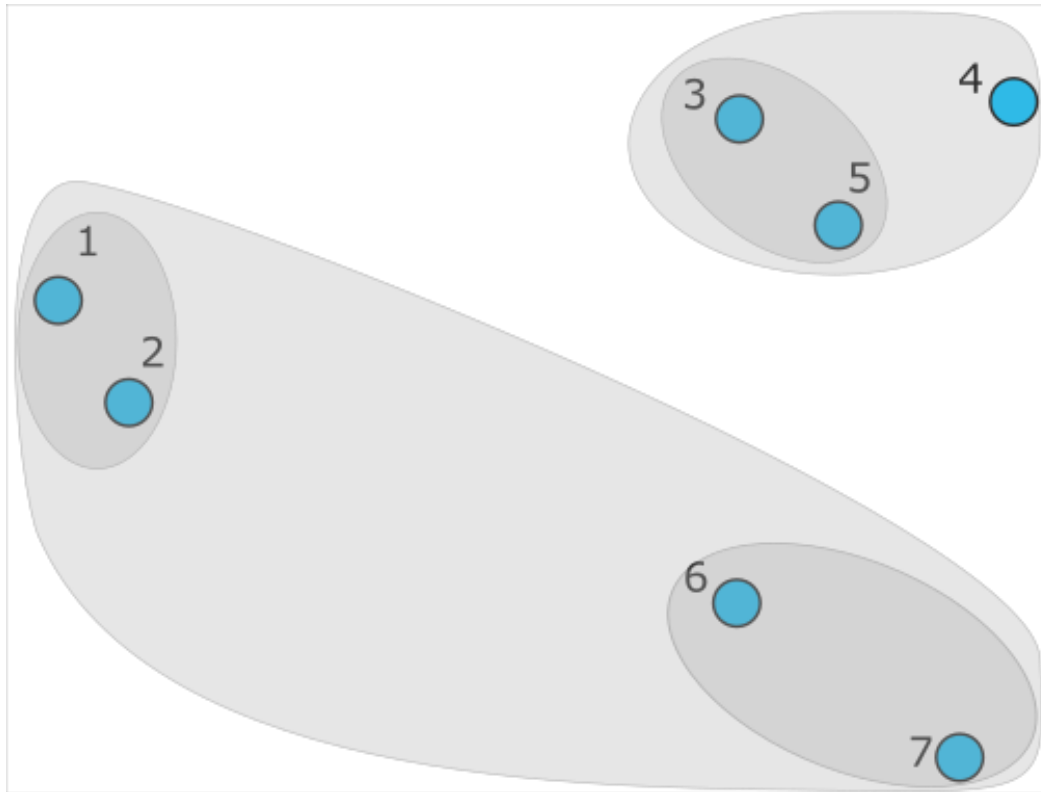
Algorithm (Agglomerative)



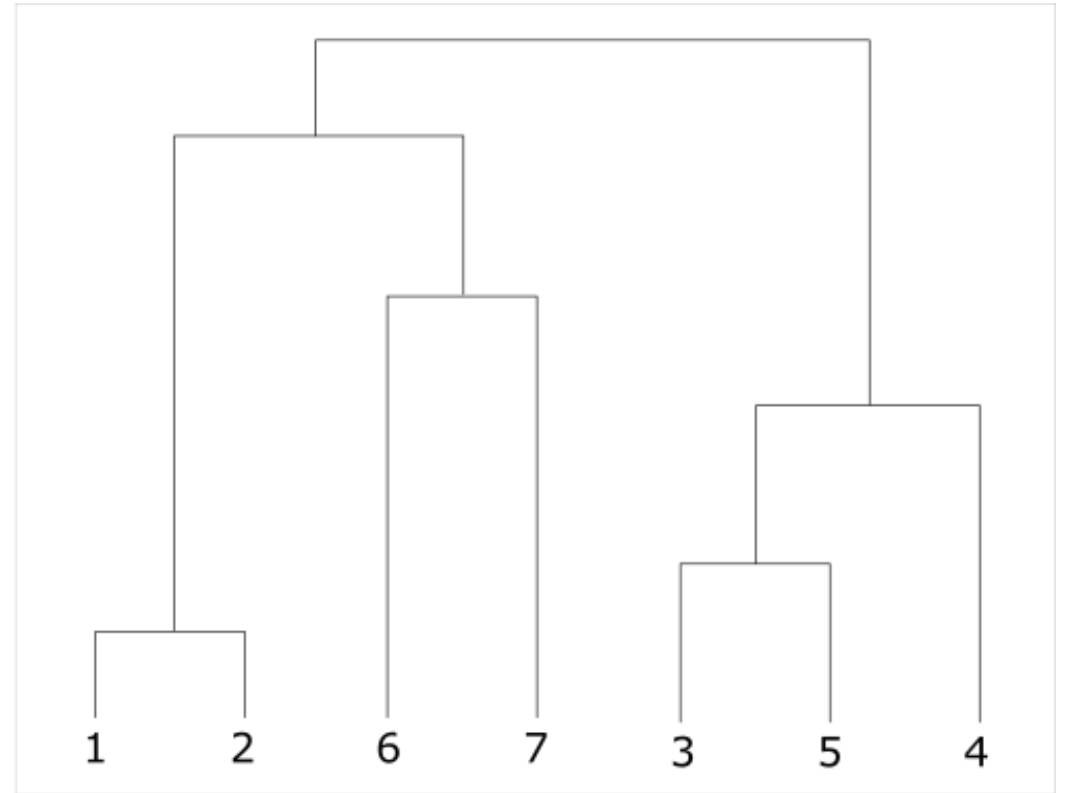
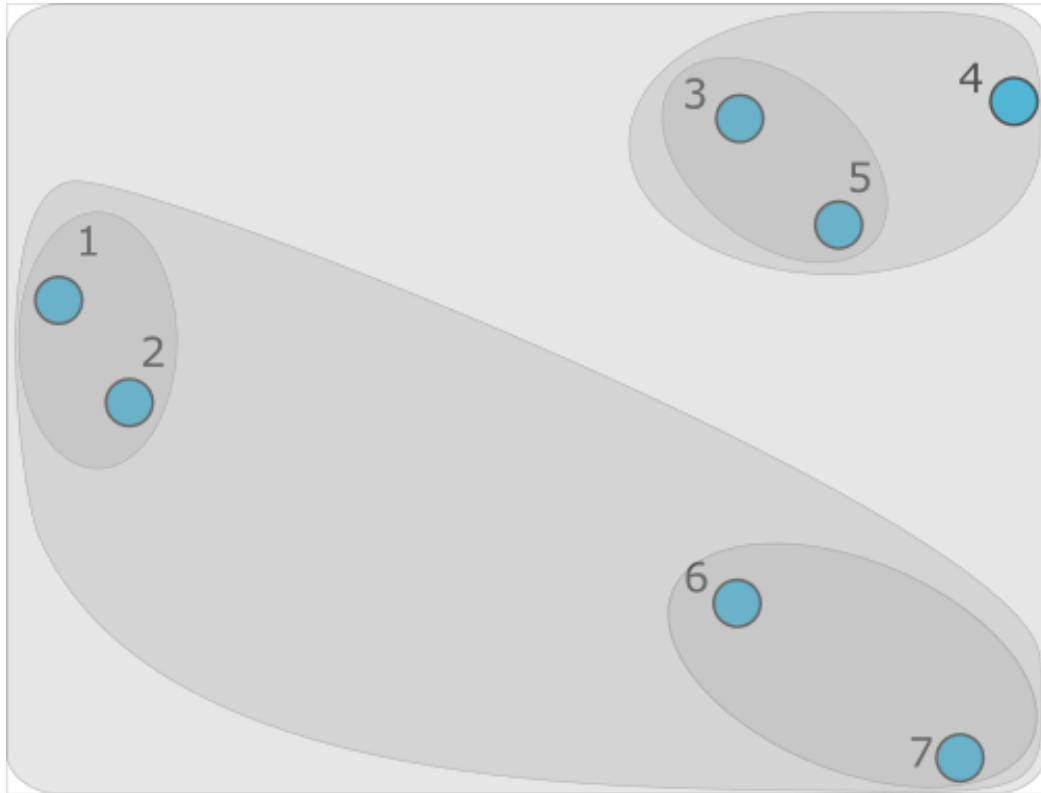
Algorithm (Agglomerative)



Algorithm (Agglomerative)

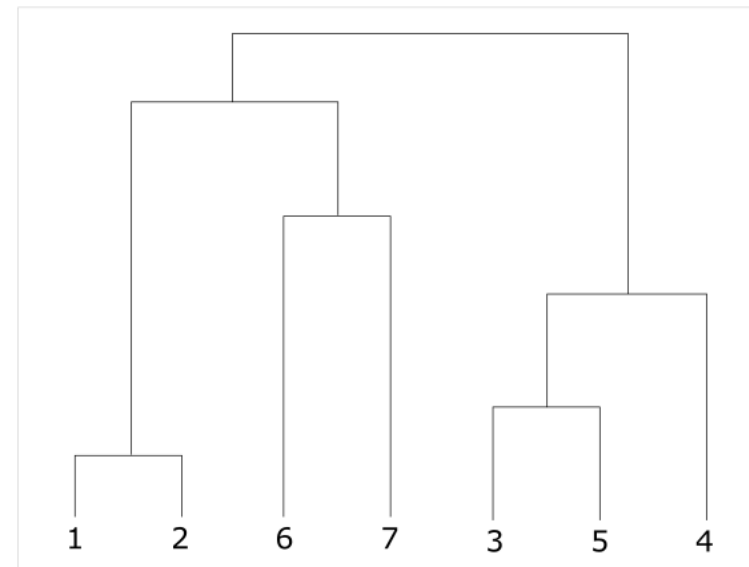


Algorithm (Agglomerative)



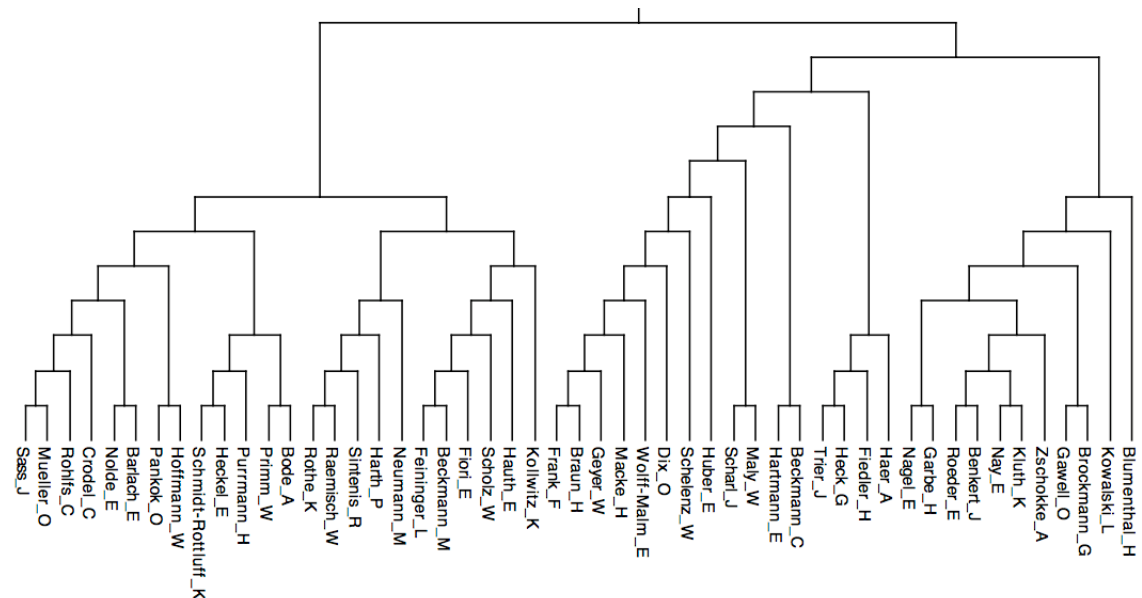
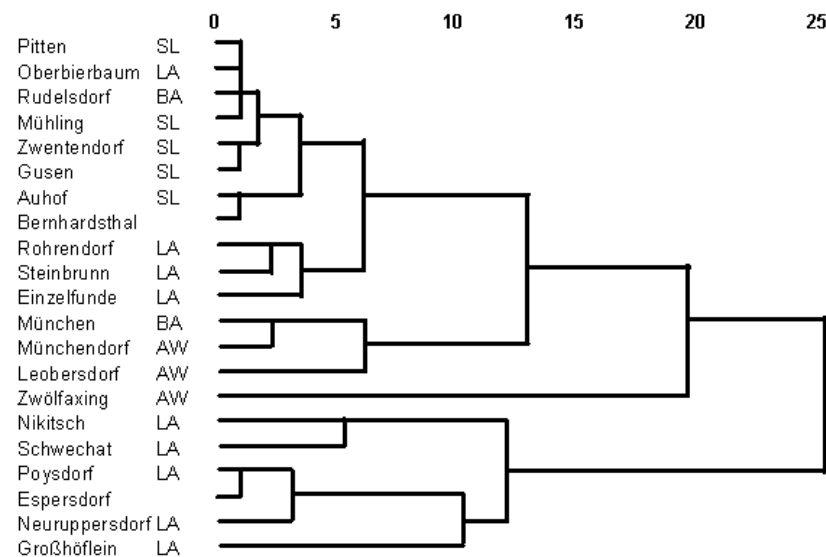
Algorithm (Agglomerative)

- Dendrogramm:
 - Relative positions of subtrees are determined and integrated in a larger tree by placing subtrees as close as possible (Reingold, 1981)
 - Not suitable for large element numbers (scalability)
 - For navigation, the user needs a facility to adjust the hierarchy level (feedback: number of sel. clusters)



Clustering Methods: Hierarchical

- Dendrograms horizontally or vertically aligned
- Left: Dendrograms indicate old skeletons, the location where they were found and their similarity
- Right: Artists are analyzed w.r.t. similarity, assessed by participation in exhibitions

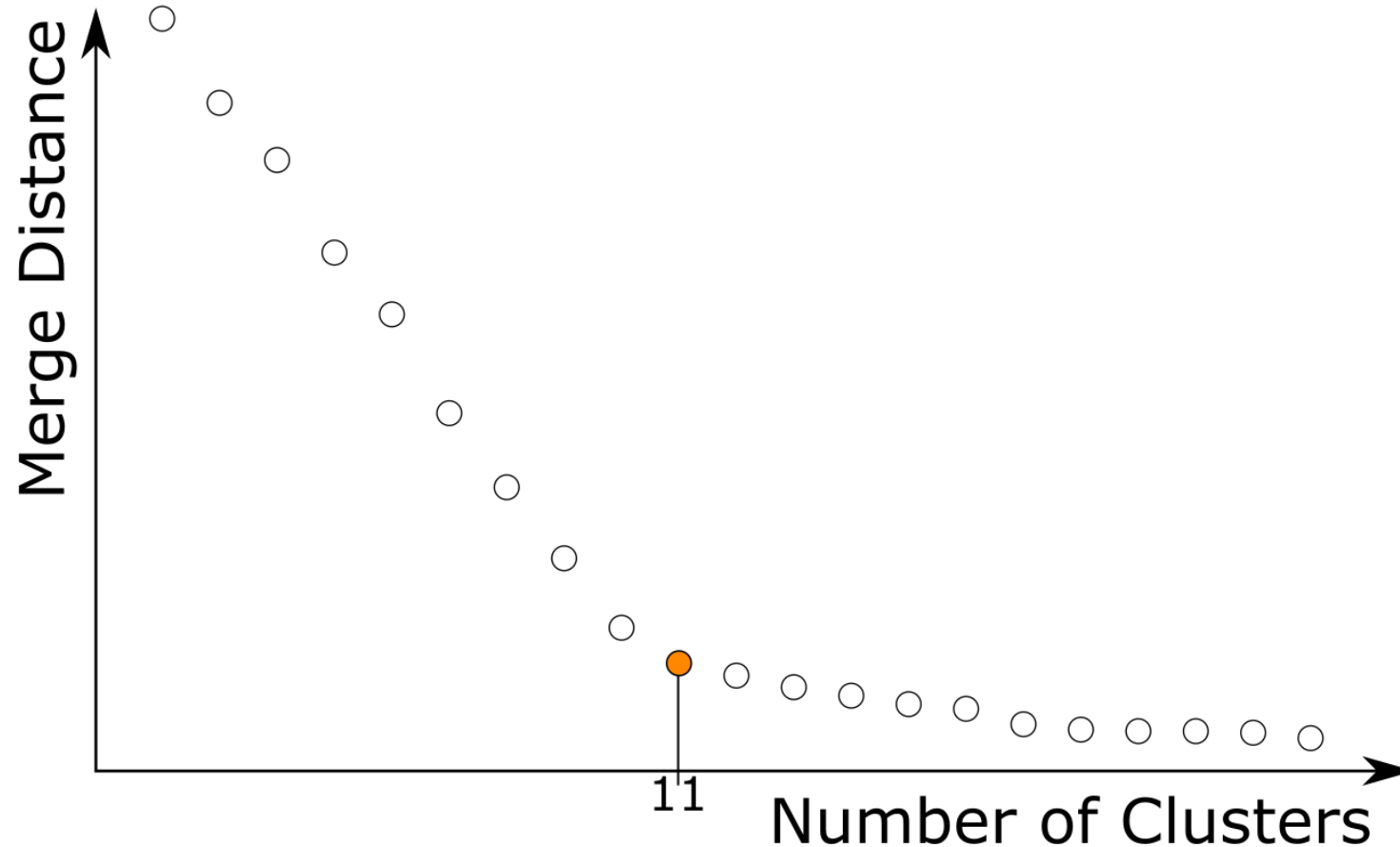


Clustering Methods: Hierarchical

- How to determine a reasonable number of clusters?
- L Method is widely used

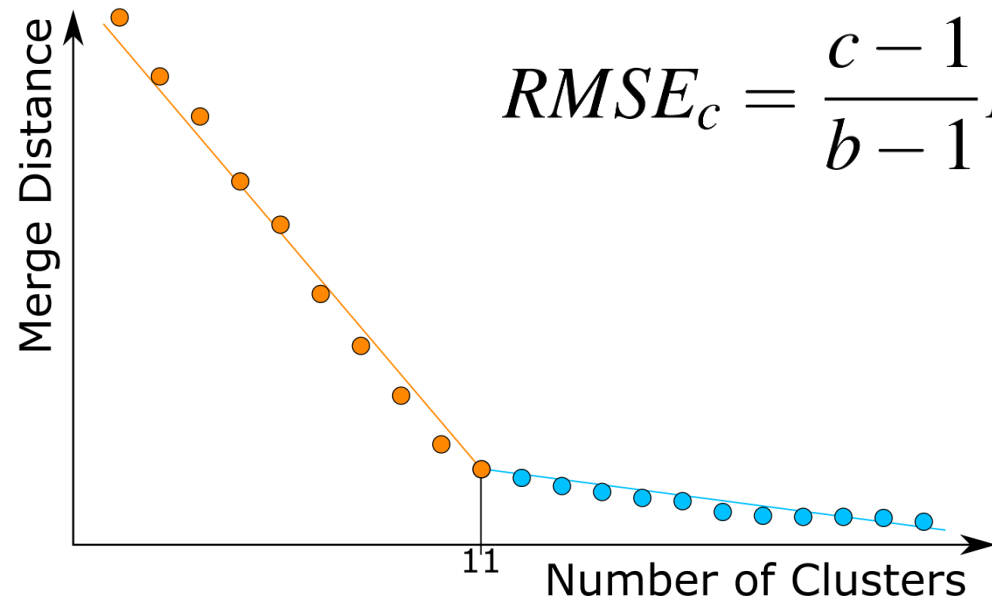
Clustering Methods: Hierarchical

- Create a 'Number of Cluster'-' Merge Distance' diagram and find the 'elbow'



Clustering Methods: Hierarchical

- Fit two lines that mostly fit the curve and determine the error
- Number of clusters $\in [2, b]$
- Left and right line: L_c, R_c
- RMSE approximation error



$$RMSE_c = \frac{c-1}{b-1}RMSE(L_c) + \frac{b-c}{b-1}RMSE(R_c)$$

Clustering Methods: Hierarchical

- Interactive exploration of dendrograms (Seon, 2002)
- Select subtrees to collapse/display
- Adjust a global similarity threshold (range slider)
- Combine with a detailed view to display features of the selected cluster
- Enable comparison of two clustering results

Clustering Categorical and Mixed Data

- High-dimensional data often comprise ordinal and categorical data (low ranges or even no ordering)
- Most clustering methods tailored to scalar data
- Categorical data:
 - $dist(p_i, p_j) = 0$ if the attributes of both points are equal, otherwise 1 (known as overlap metric)
- Mixed data:
 - Integrated distance measure comprising binary and categorical data, e.g. the Heterogeneous Euclidean Overlap metric [Wilson, 1997]
- Alternative:
 - Discretize the scalar data to categorical data. Search for ideal cutoff points that minimize the entropy in the resulting classes [Fayyad, 1993]

Clustering Categorical and Mixed Data

- K-Prototypes (Huang, 1998)
 - Appropriate for categorical data
 - Requires an a priori estimation of the number of clusters

Clustering Categorical and Mixed Data

- K-Prototypes (Huang, 1998)
- Distance measure: sum of Euclidean distances of numerical values and the sum of categorical distances
- With $\delta(p, q) = 0$ if $p = q$ and $\delta(p, q) = 1$ otherwise

$$d(X_i, Q_l) = \sum_{j=1}^{m_r} (x_{ij}^r - q_{lj}^r)^2 + \gamma_l \sum_{j=1}^{m_c} \delta(x_{ij}^c, q_{lj}^c)$$

The diagram illustrates the components of the distance formula. Three arrows originate from the equation: one from the first summation term pointing to 'Numerical values', one from the coefficient γ_l pointing to 'Weighting', and one from the second summation term pointing to 'Categorical values'.

Clustering with Constraints

- Results may be improved with further a priori knowledge (in addition to cluster models):
 - Must-link constraints: User specifies objects and „asks“ the algorithm to group these objects in the same cluster
 - Cannot-link constraints: Selected objects should belong to different clusters
- A clustering algorithm usually cannot fulfil all constraints
- The degree to which the constraints are met as an evaluation criterion
- Clustering is an unsupervised method; clustering with constraints is (semi-)supervised

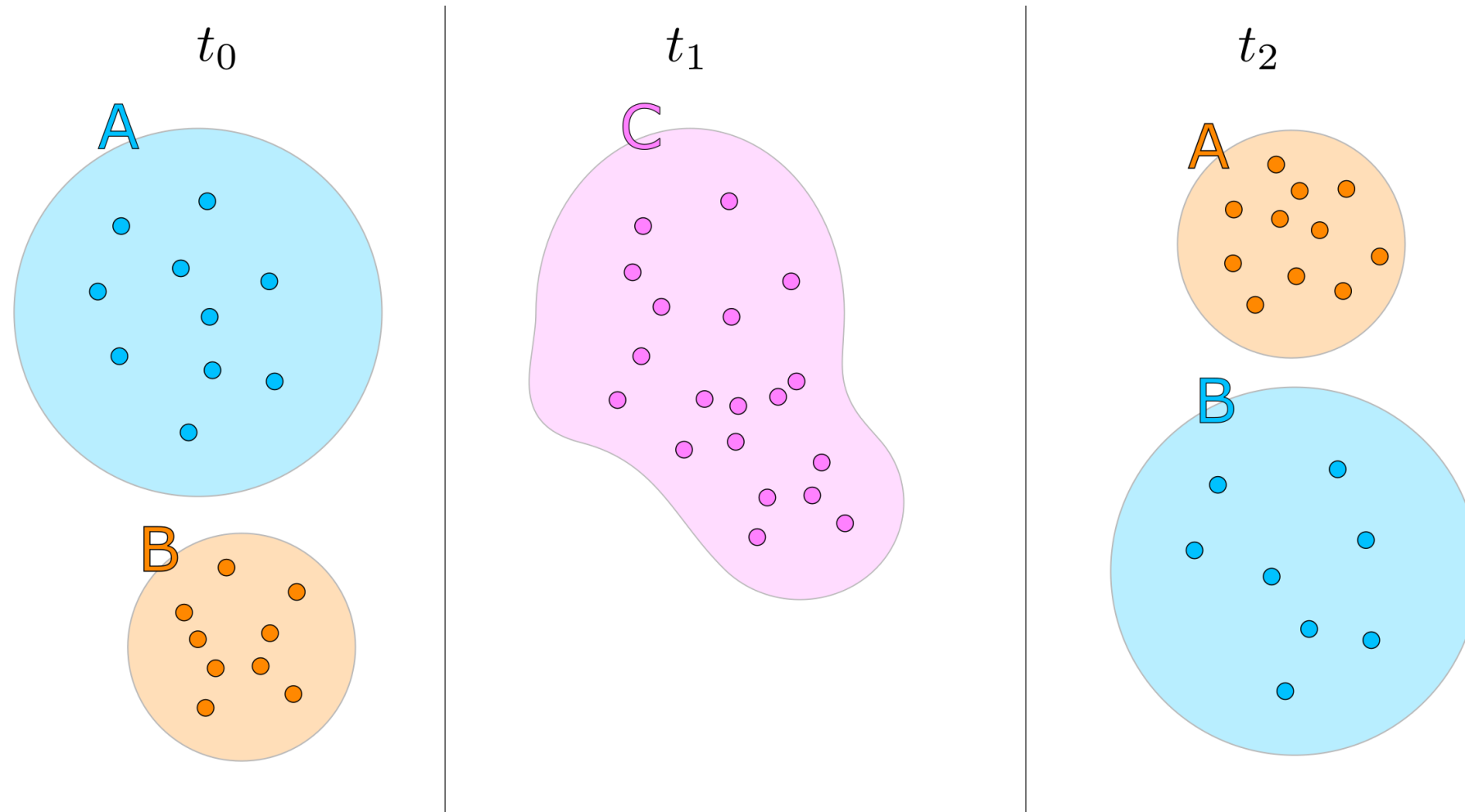
Temporal Clustering

- Many HD datasets reflect changes over time, e.g. data on student cohorts, cohorts in epidemiology, marketing-relevant data
- Distance metric
 - Consider each point in time independently or
 - Cluster the time series w.r.t. similar trends (slightly increasing, steady, ...)
- Clustering should reflect how clusters emerge, merge, split or disappear (temporal clustering, Turkey, 2011)
 - Temporal cluster view to assess the structural quality of clusters and type of structural changes. The cluster view displays the cluster quality and structural changes by encoding each cluster as axis in a parallel coordinate view

Temporal Clustering

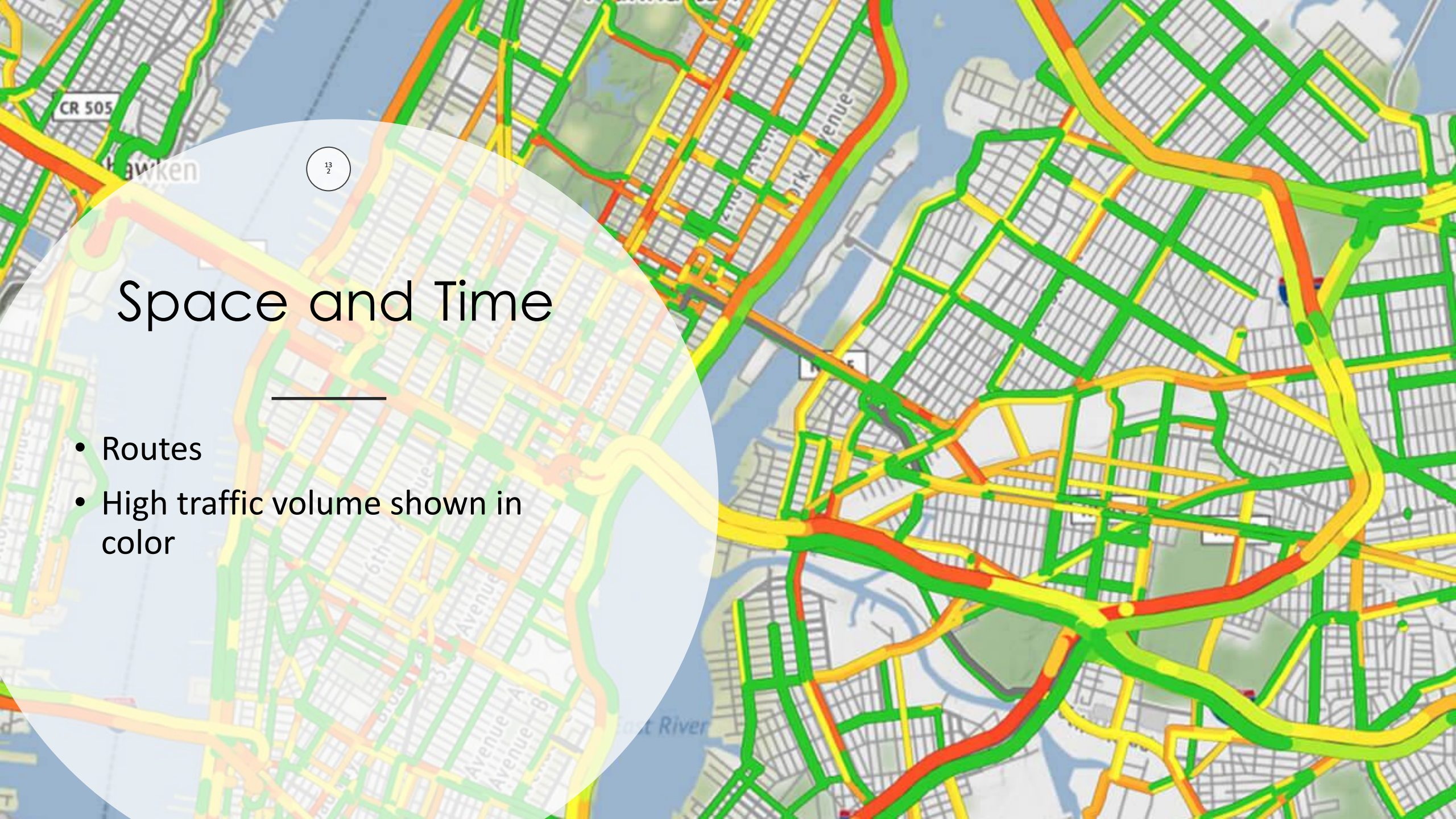
- Frequent questions of analysts (see also Turkay, 2011):
- How does the quality of clusters change over time?
- What type of structural changes do clusters exhibit?
- What are the characteristics of items that „move“ between clusters, e.g. move from slim to obese person or from healthy to pathological?
- What are the (relative) sizes of clusters over time?

Temporal Clustering



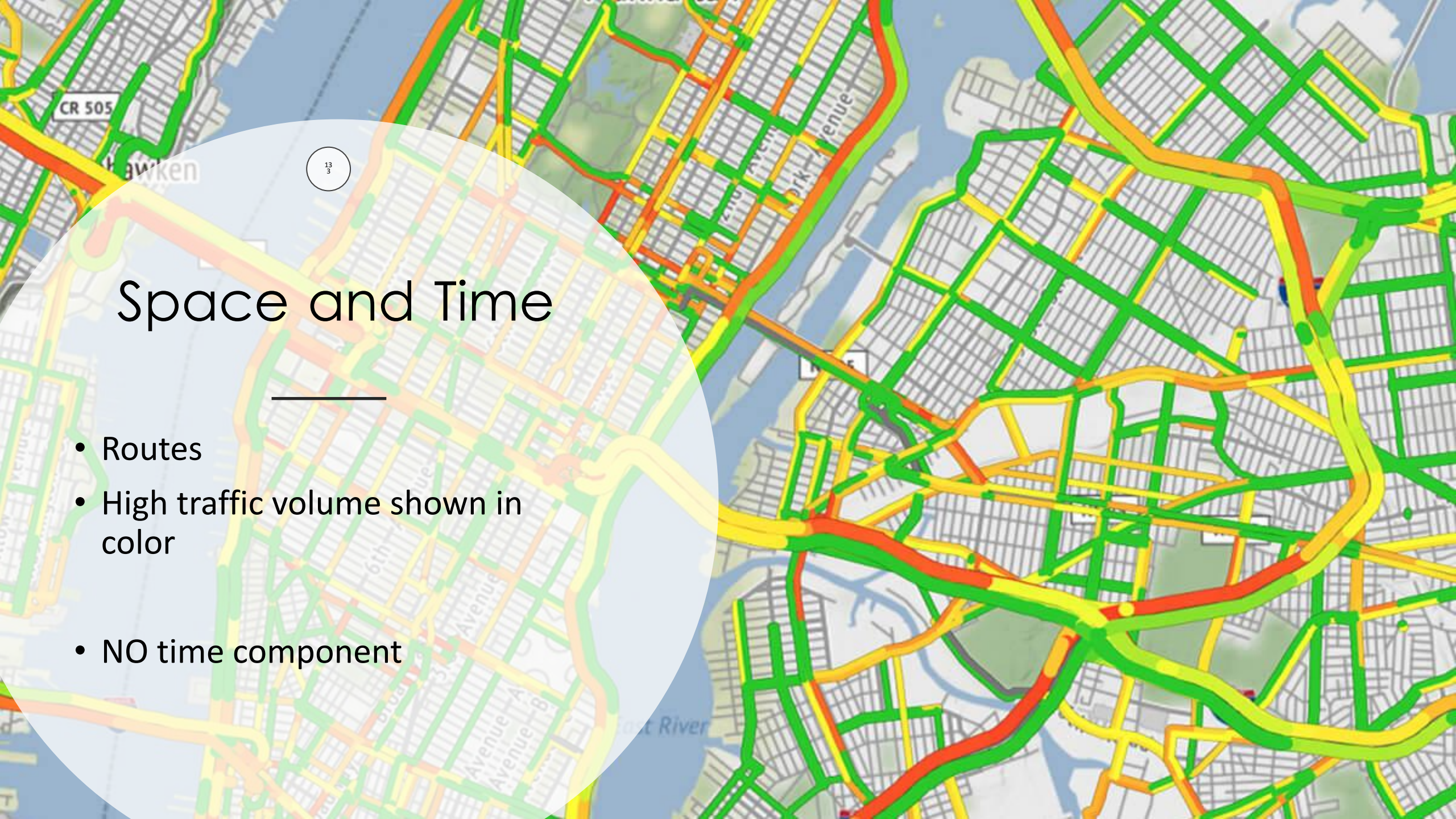


Data in which the time and location are
constantly changing



Space and Time

- Routes
- High traffic volume shown in color



Space and Time

- Routes
- High traffic volume shown in color
- NO time component

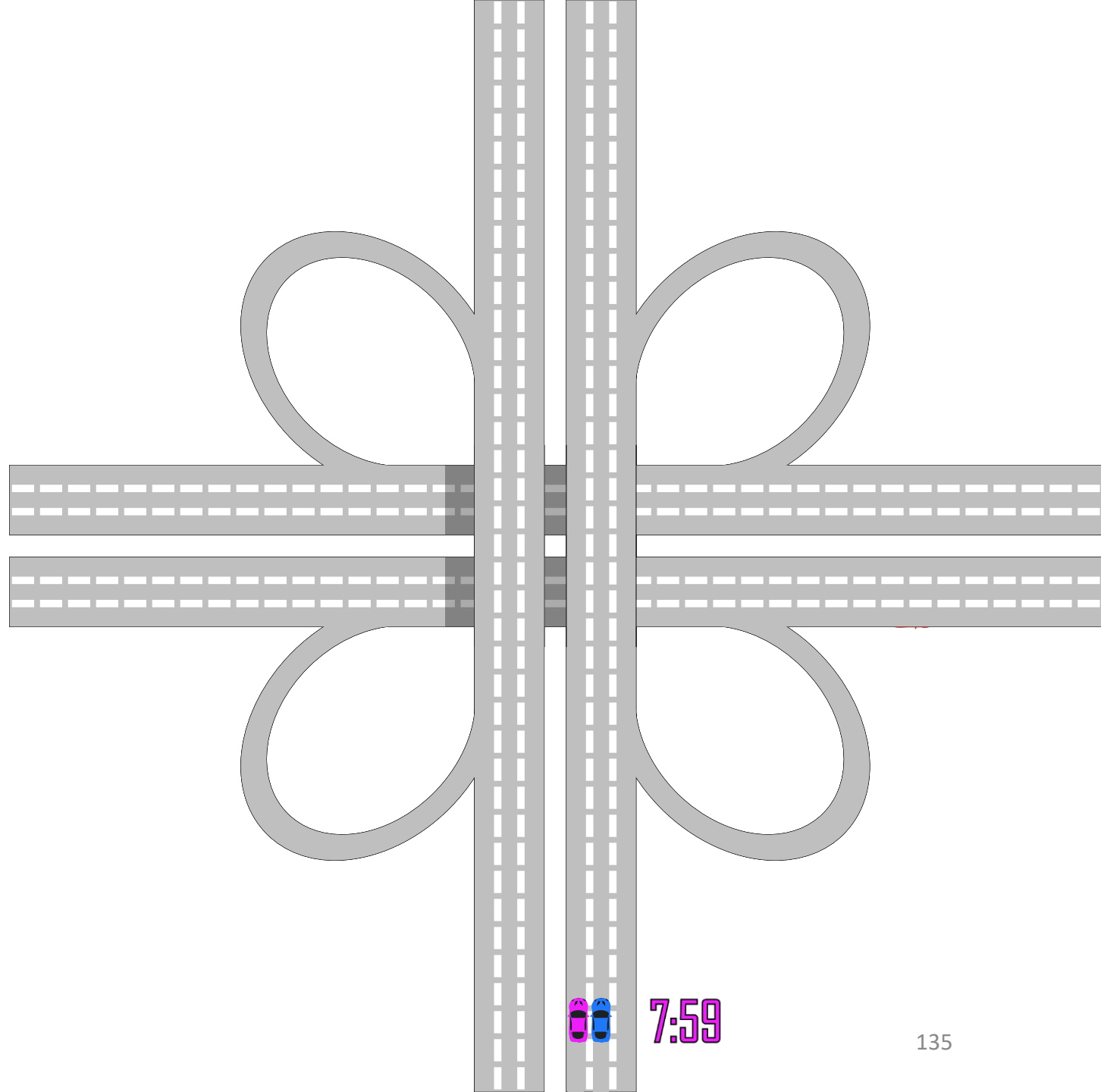


Route analysis

Summary of journeys taking into account the time

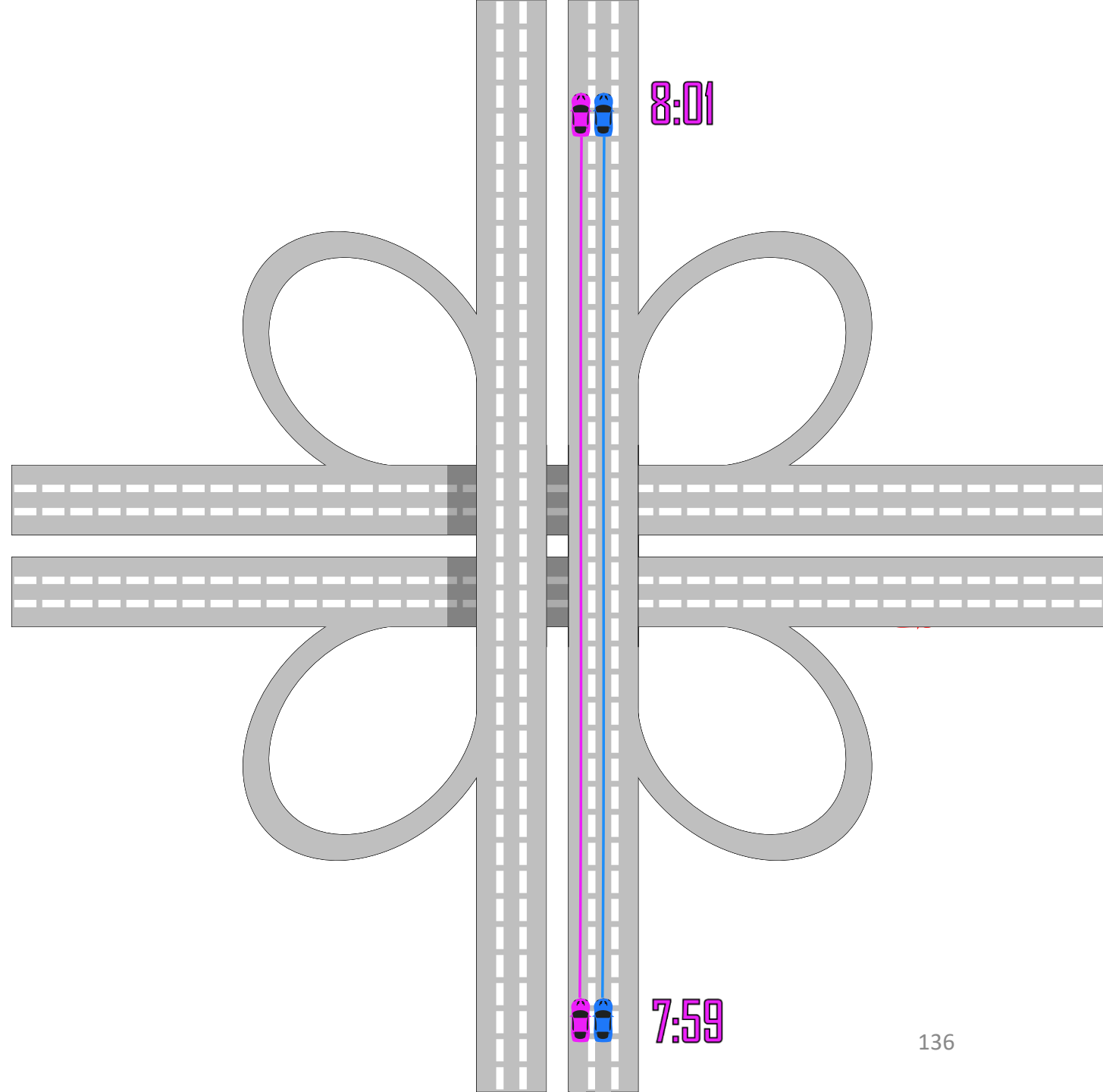
Route Analysis

- Example at a highway junction:
 - At 7:59 a.m. both cars are at the same place



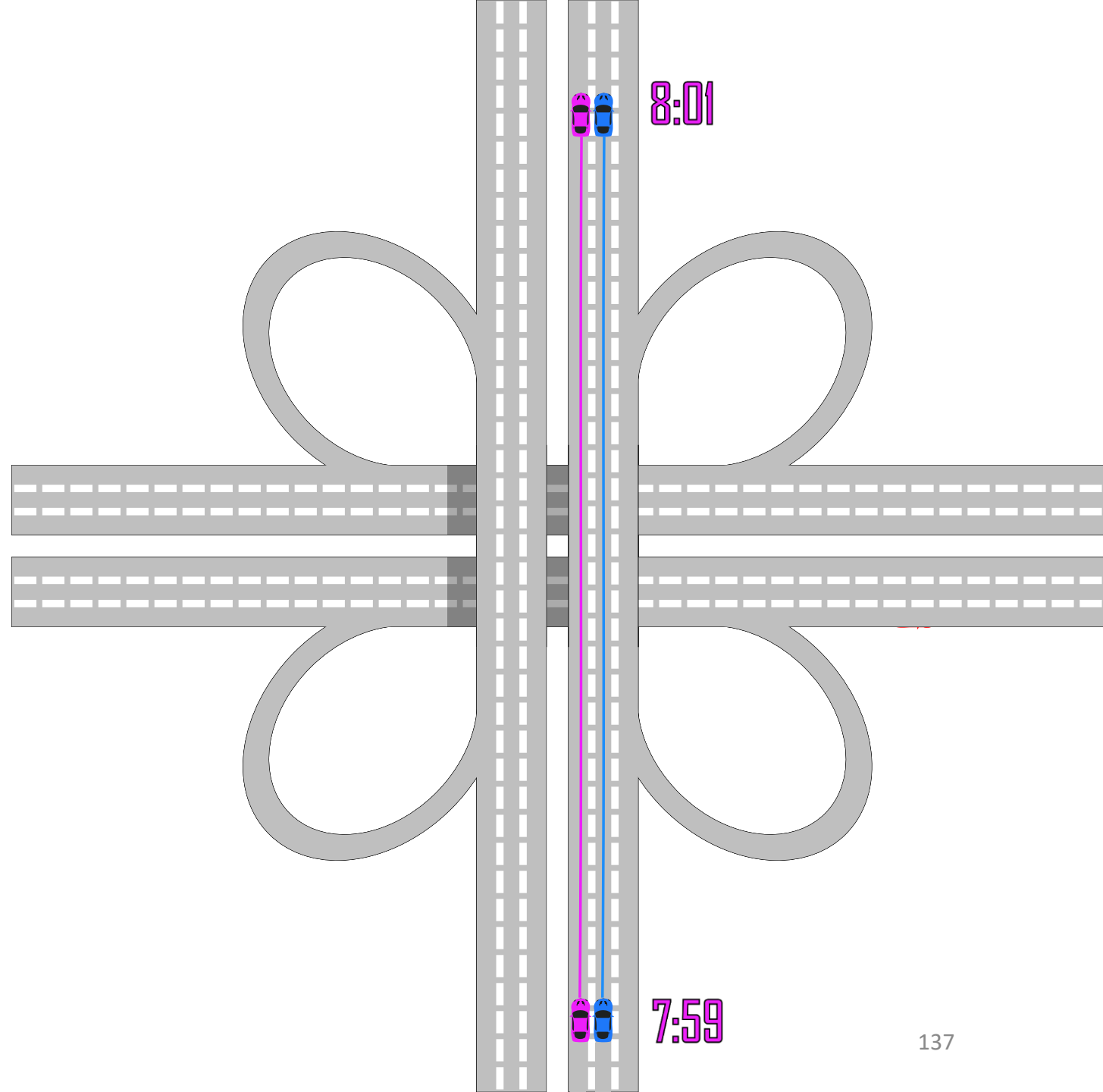
Route Analysis

- Example at a highway junction:
 - At 7:59 a.m. both cars are at the same place
 - Both drive at the same speed and leave the intersection at 8:01 a.m.



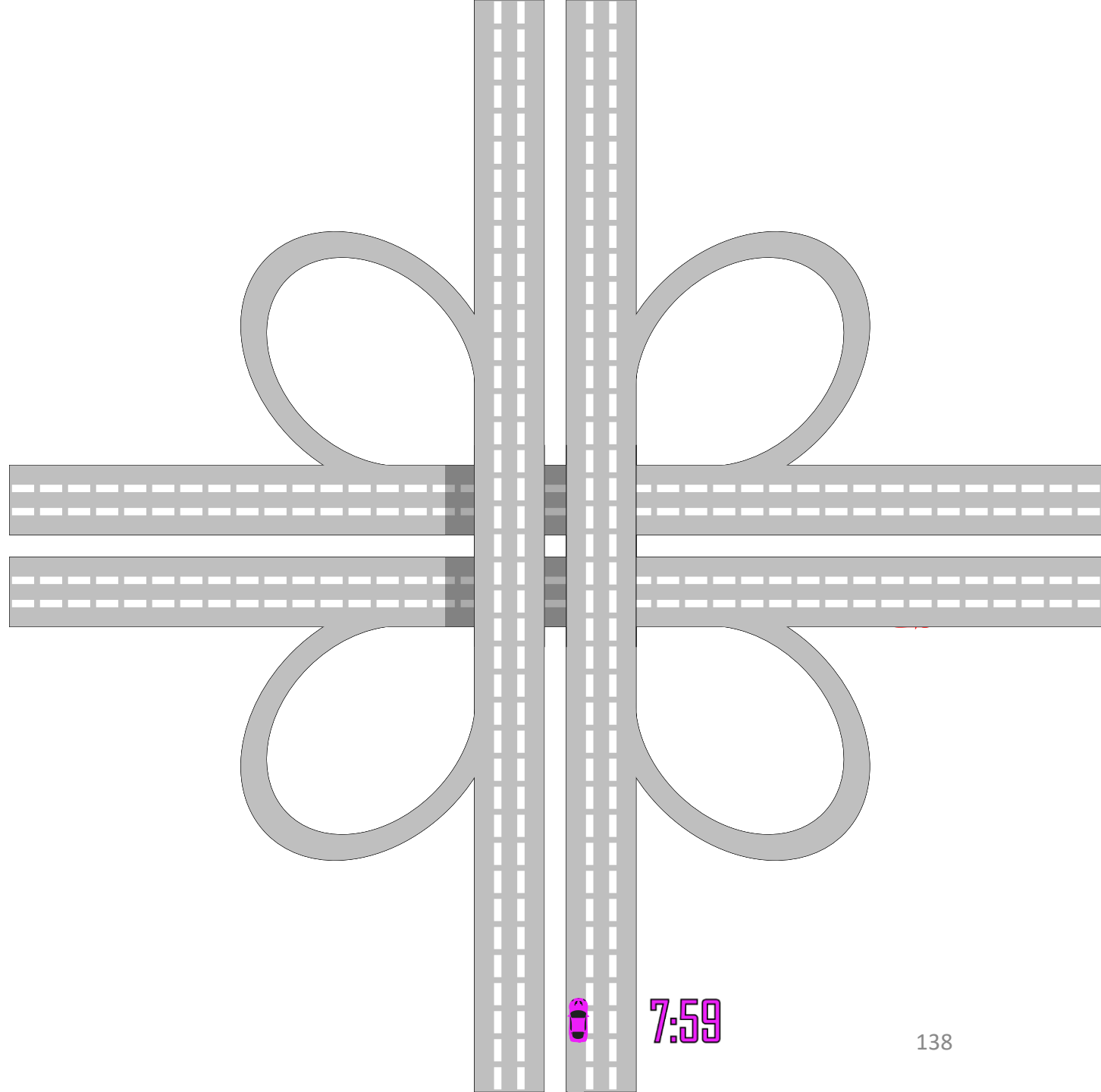
Route Analysis

- Example at a highway junction:
 - At 7:59 a.m. both cars are at the same place
 - Both drive at the same speed and leave the intersection at 8:01 a.m.
- Both cars can be combined!



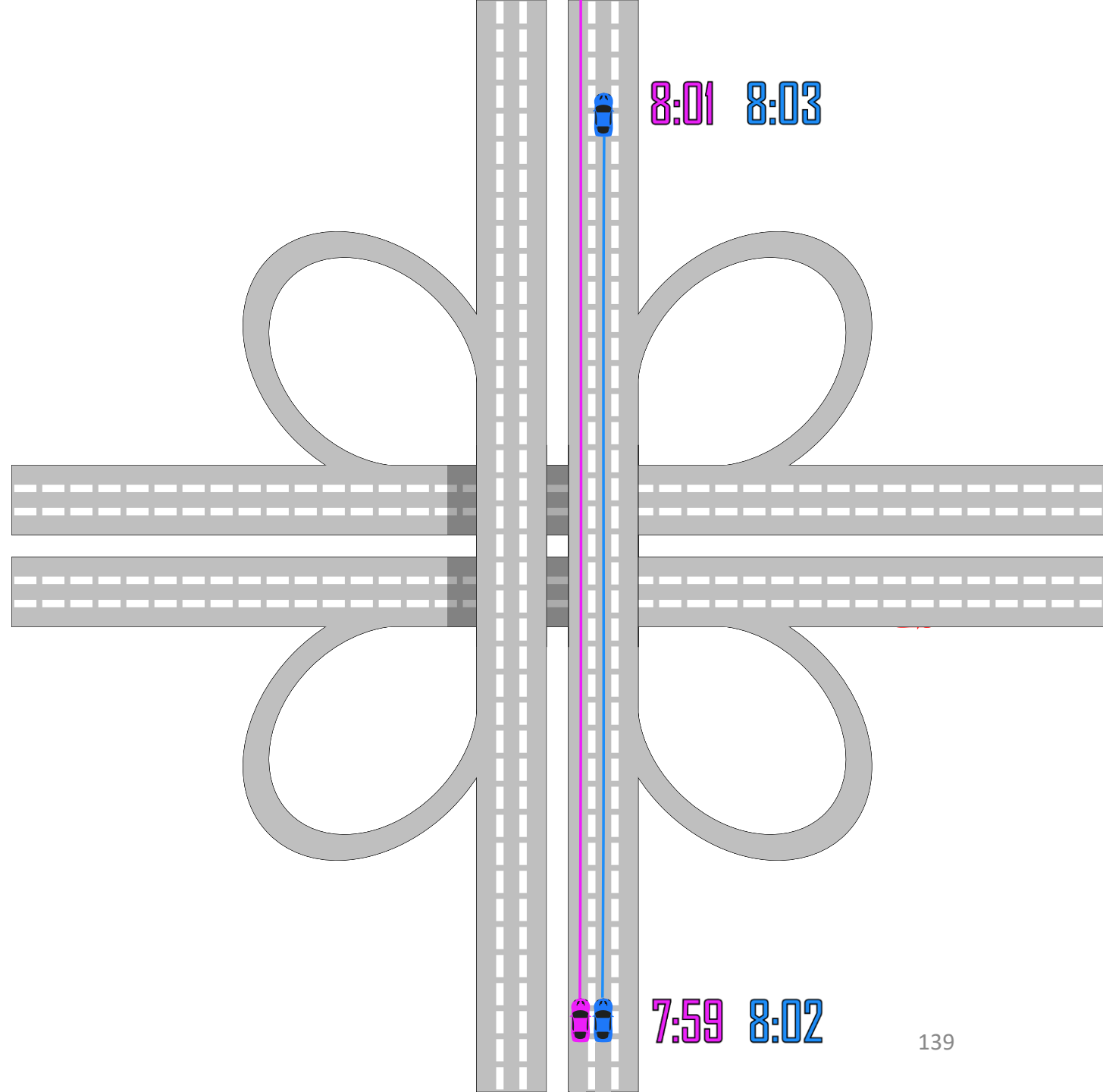
Route Analysis

- Example at a highway junction:
 - The **pink** vehicle enters the interchange at 7:59 and leaves at 8:01



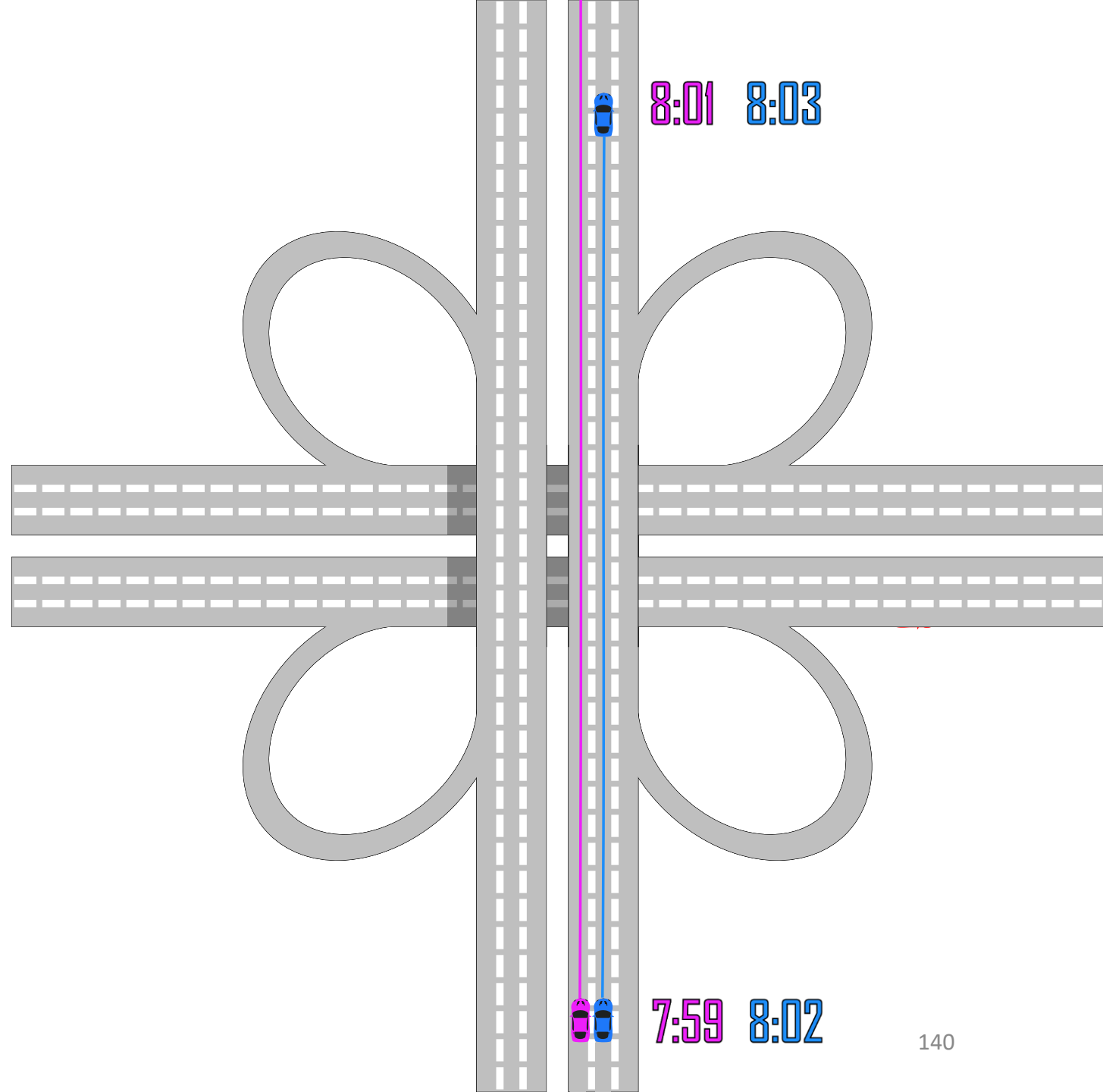
Route Analysis

- Example at a highway junction:
 - The **pink** vehicle enters the interchange at 7:59 and leaves at 8:01
 - The **blue** vehicle appears at 8:02 and disappears at 8:03



Route Analysis

- Example at a highway junction:
 - The **pink** vehicle enters the interchange at 7:59 and leaves at 8:01
 - The **blue** vehicle appears at 8:02 and disappears at 8:03
- Can the two be combined?



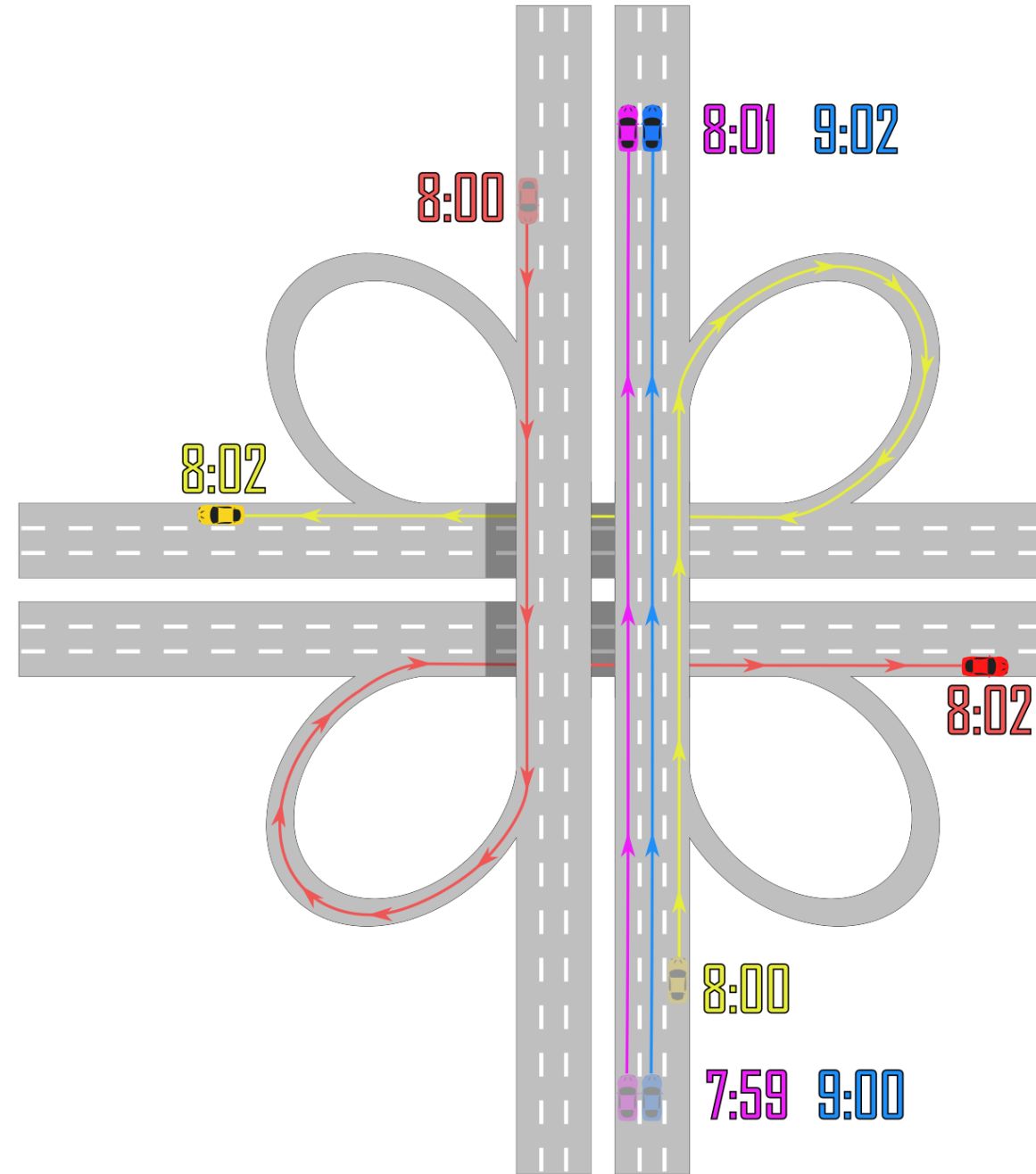


Clustering

There are several clustering methods for this problem
Distance measure required

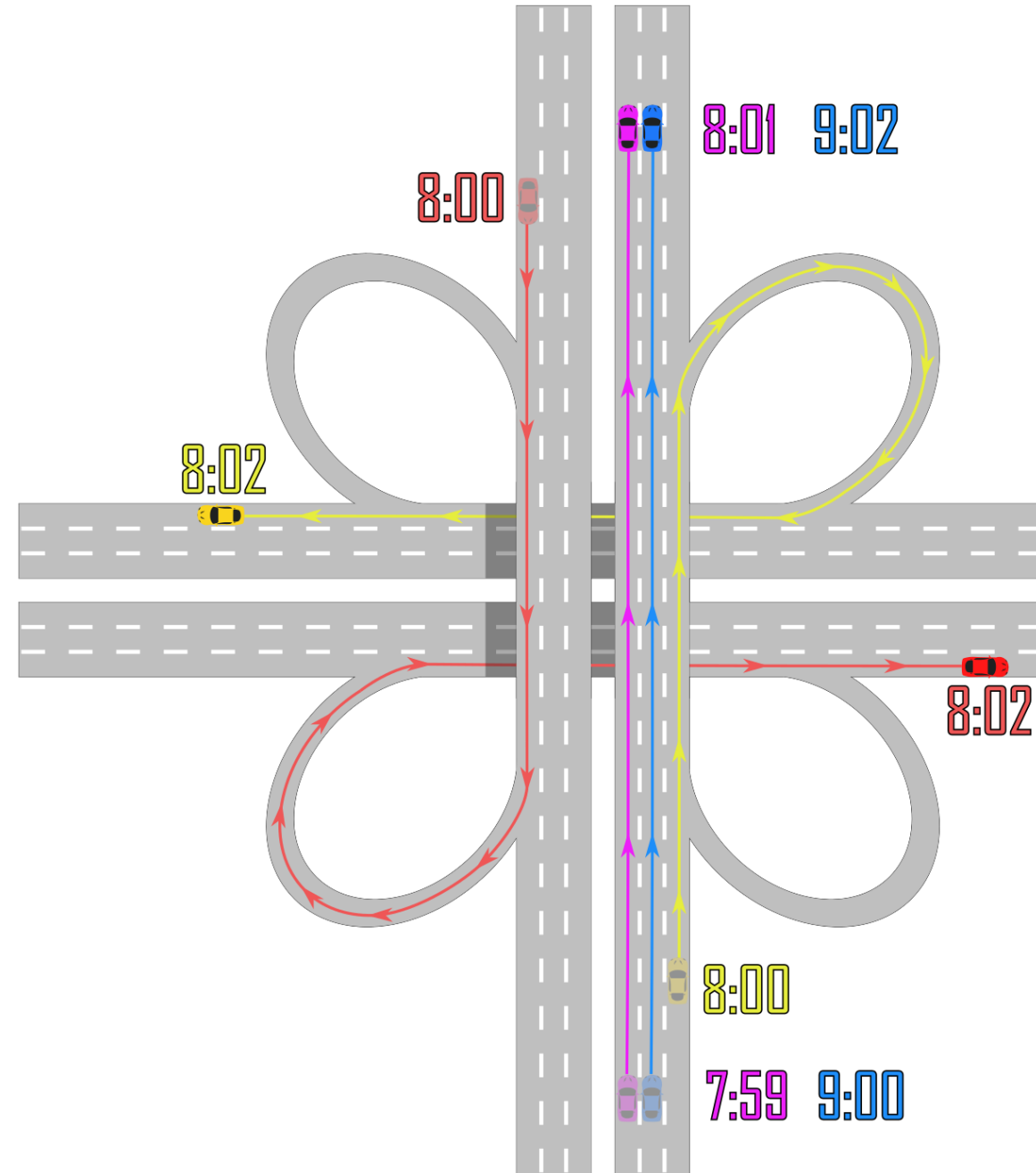
Route Analysis

- We now have four cars



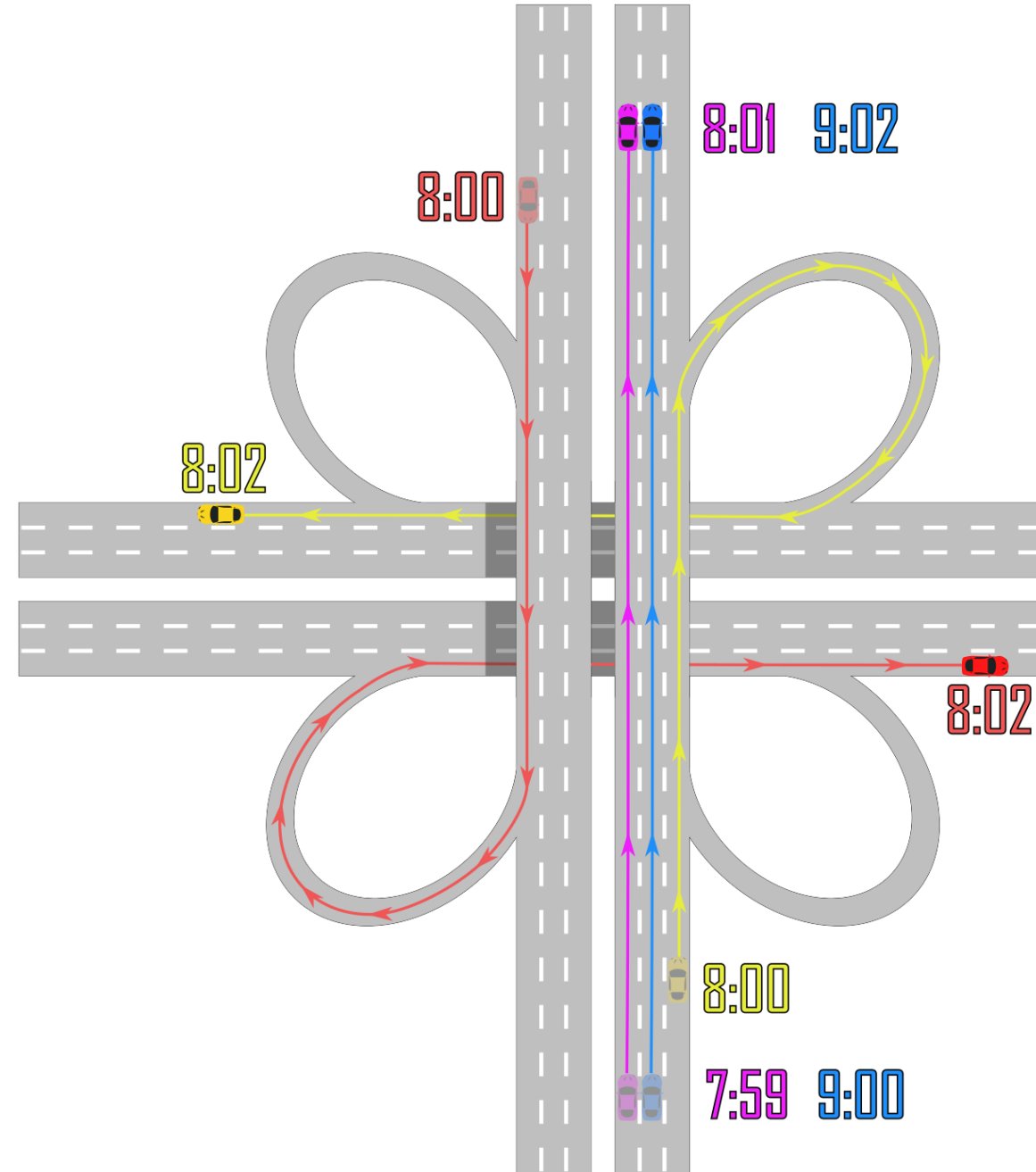
Route Analysis

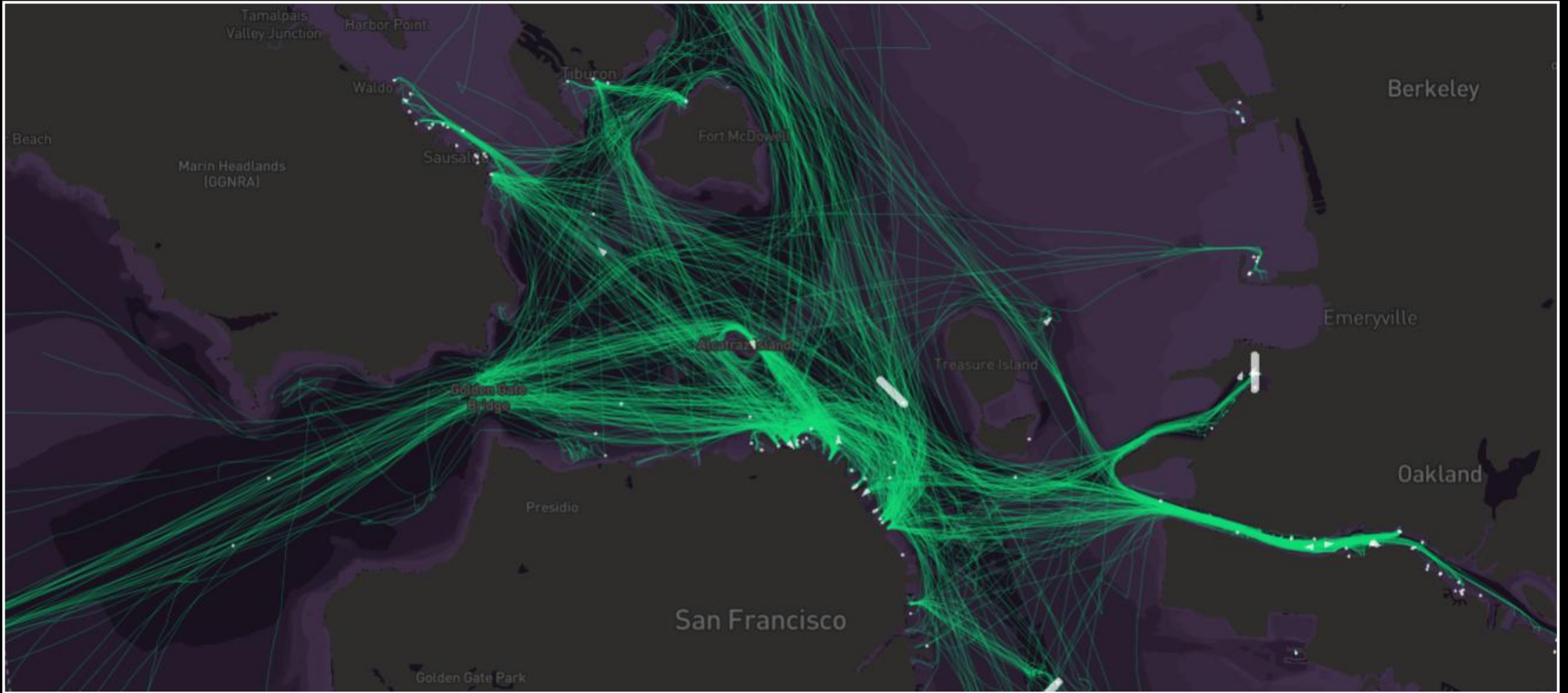
- Some clustering methods only consider geometric patterns
 - Red and yellow car belong to the same cluster



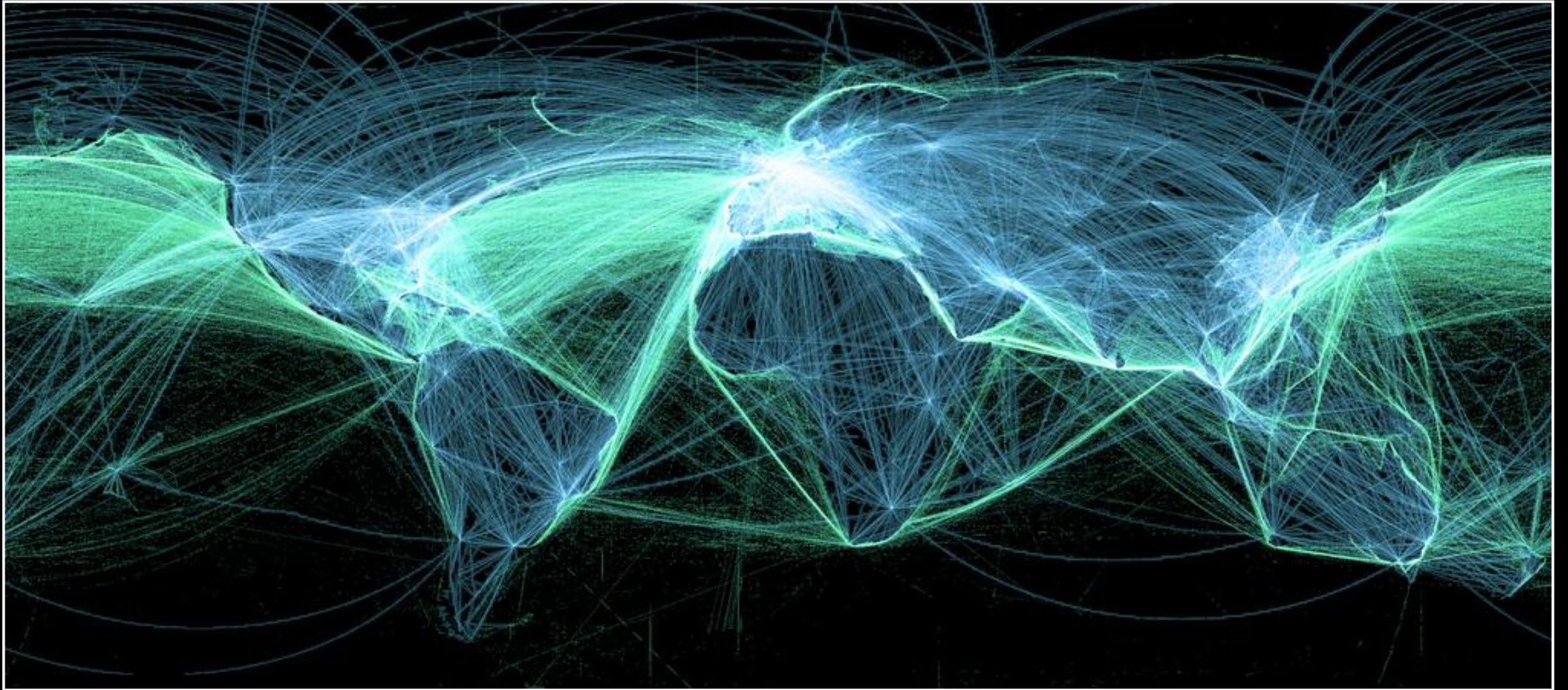
Route Analysis

- Some clustering methods only consider geometric patterns
 - Red and yellow car belong to the same cluster
- Distance measure that mixes the Euclidean distance and the time component:
 - Pink und blue car belong to the same cluster

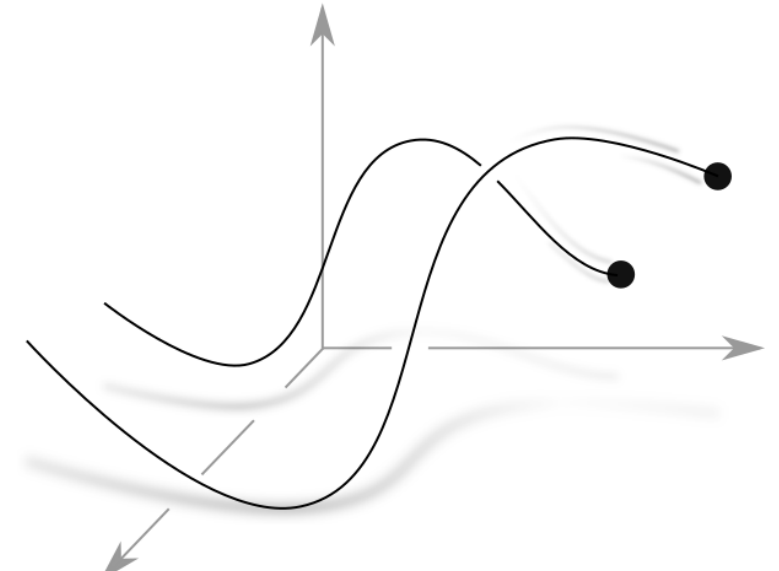
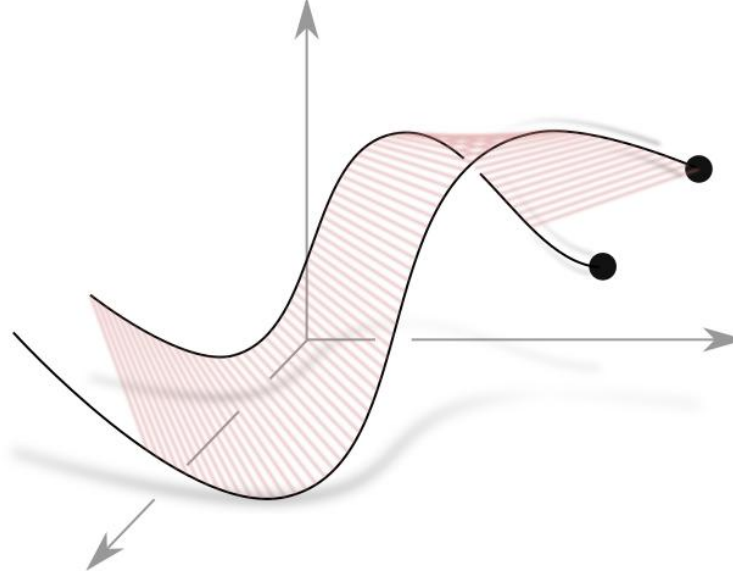
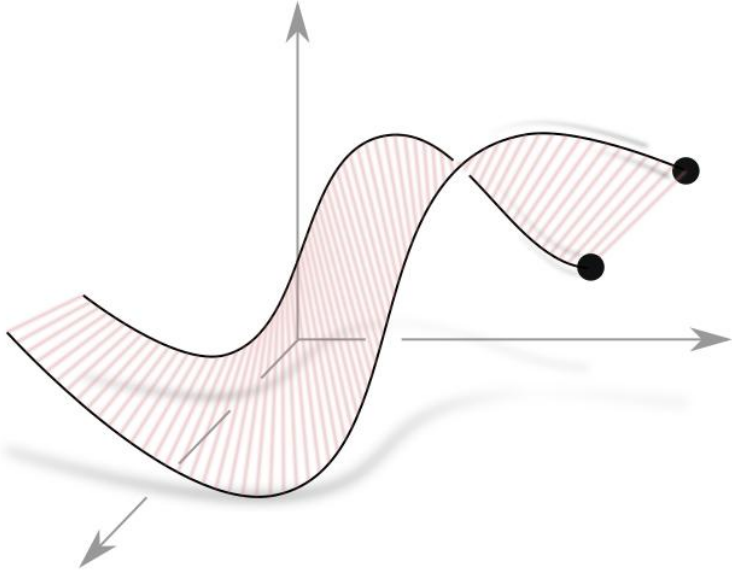




No constraints (routes)



3D

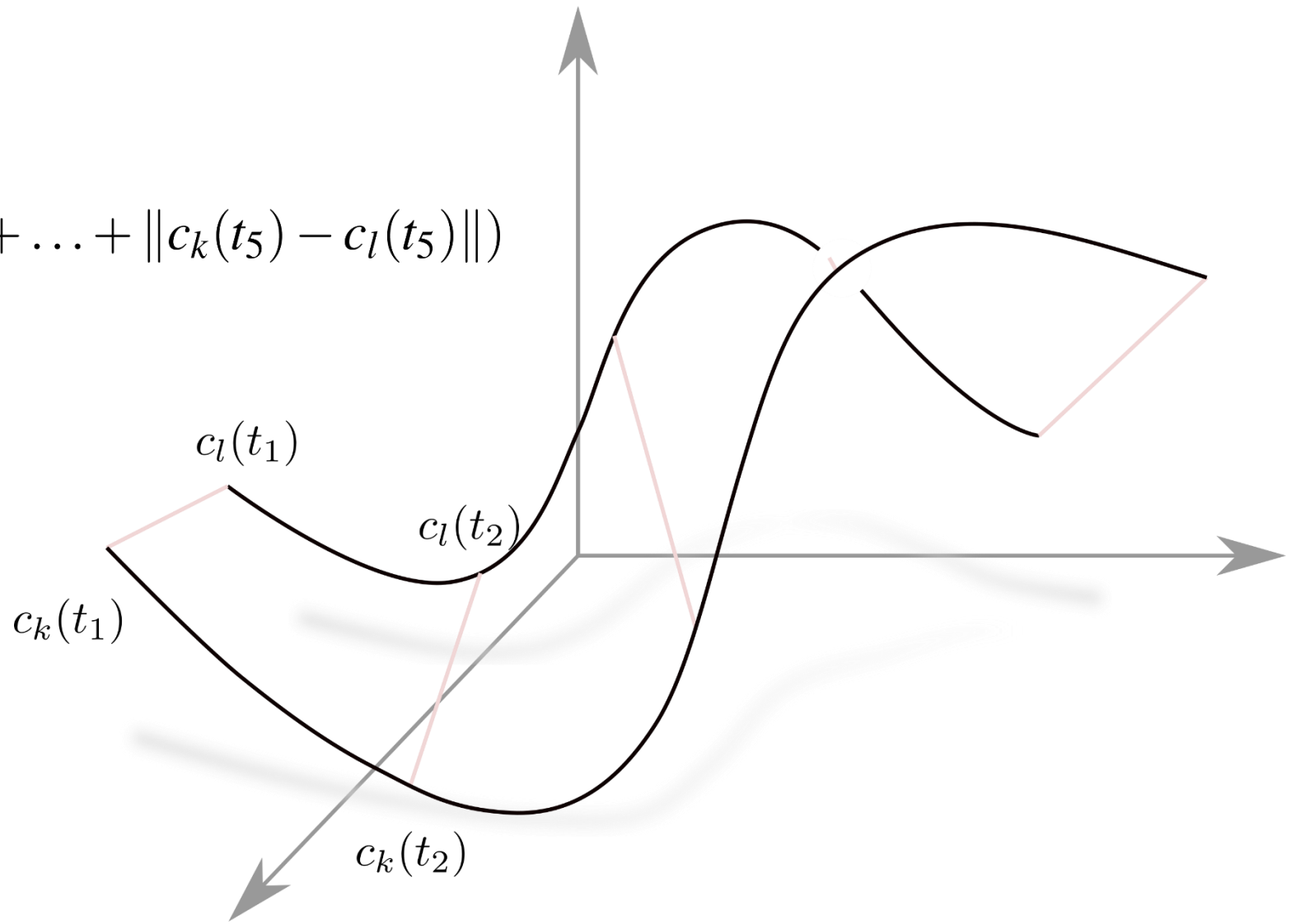


Three Cases

- 1. Time intervals coincide
- 2. Parts of time intervals overlap
- 3. Path lines occur at different points in time

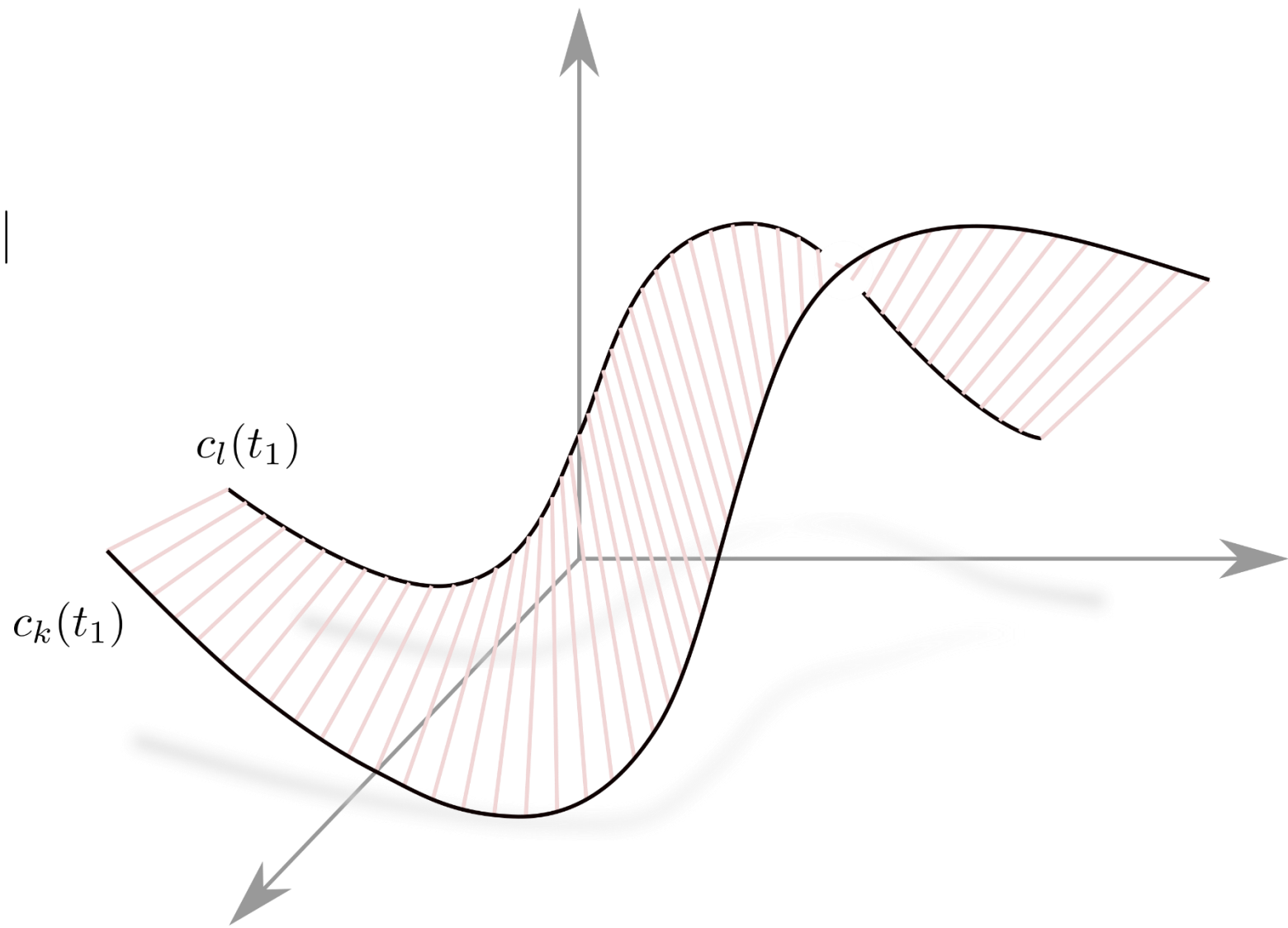
Idea

$$D(c_k, c_l) = \frac{1}{5} (\|c_k(t_1) - c_l(t_1)\| + \dots + \|c_k(t_5) - c_l(t_5)\|)$$



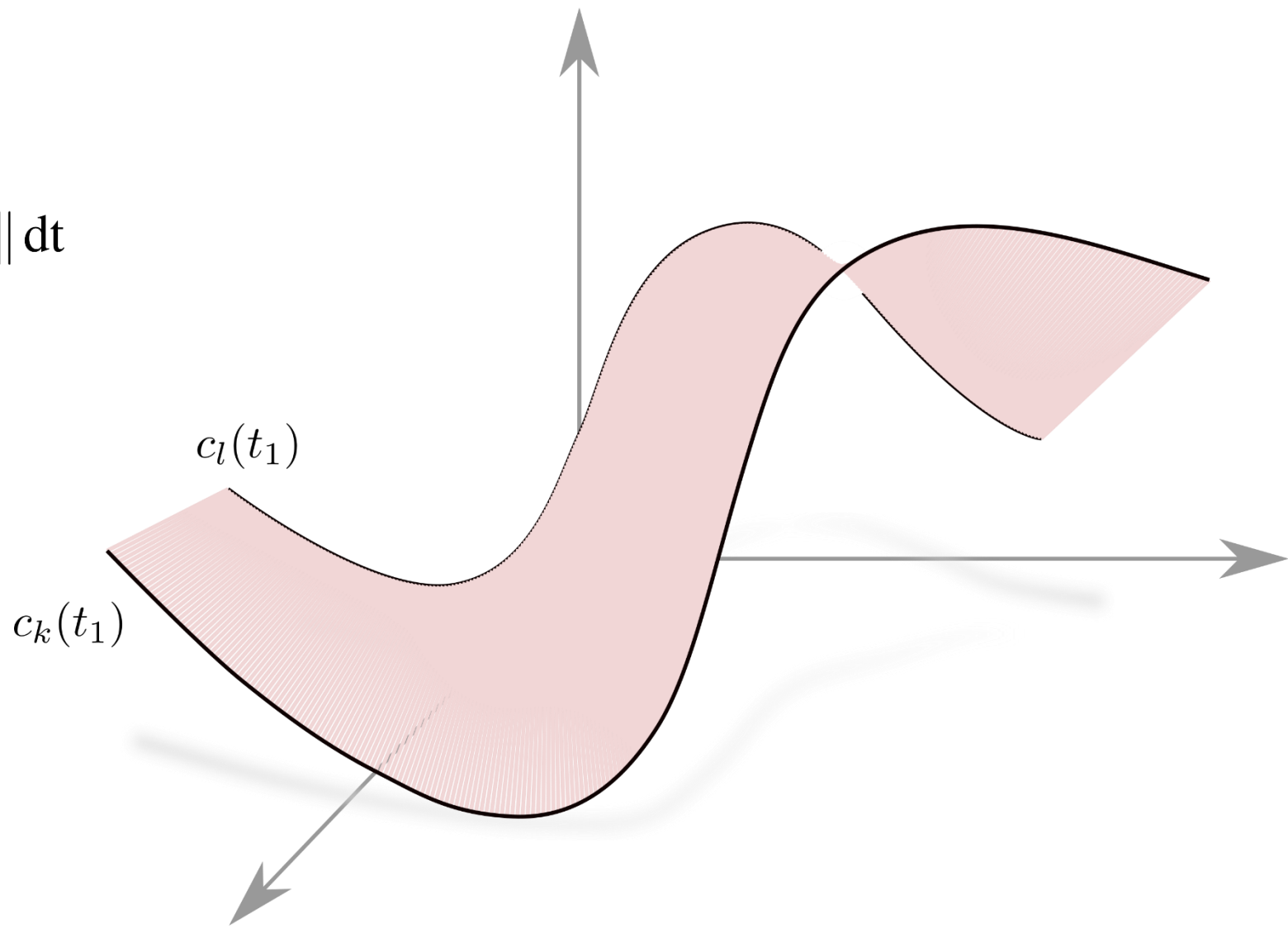
Idea

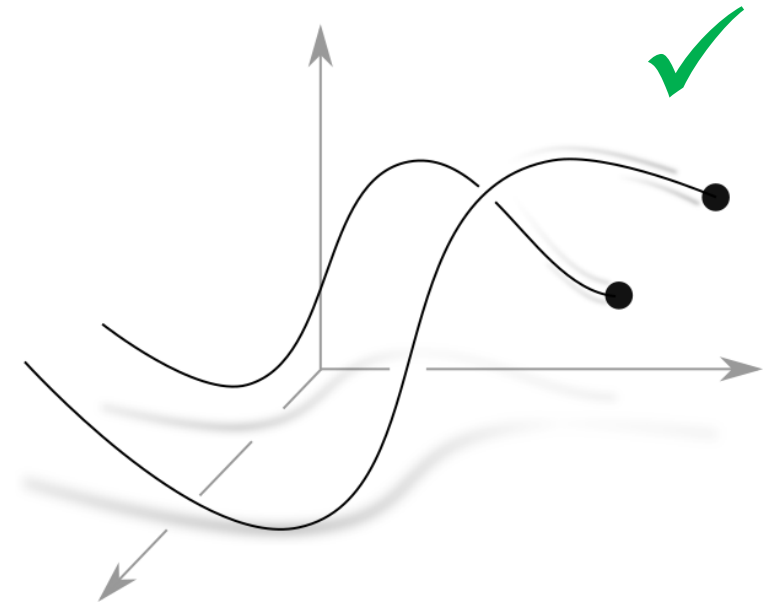
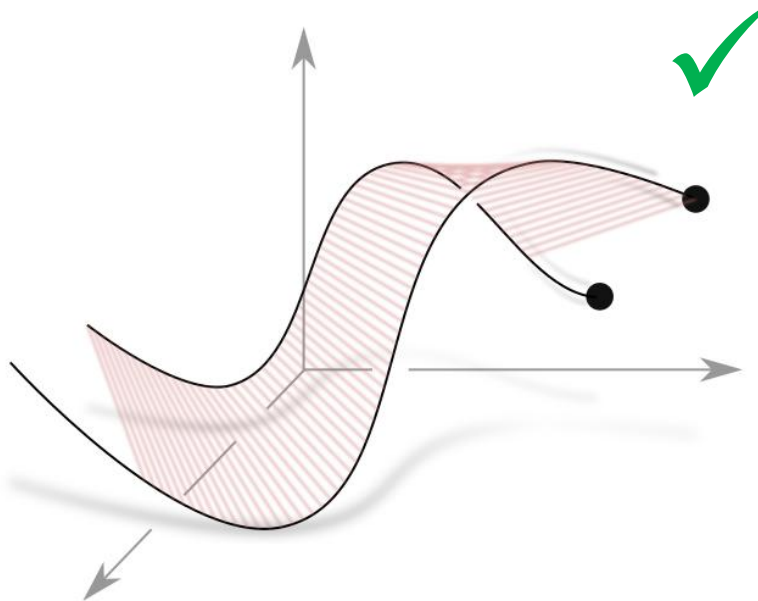
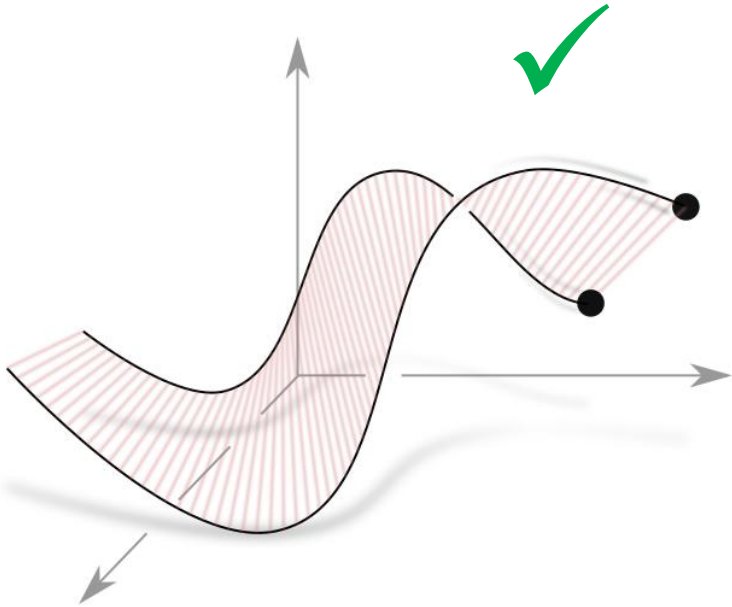
$$D(c_k, c_l) = \frac{1}{N} \sum_{i=1}^N \|c_k(t_i) - c_l(t_i)\|$$



Idea

$$D(c_k, c_l) = \frac{1}{|T|} \int_T \|c_k(t) - c_l(t)\| dt$$





Three Cases

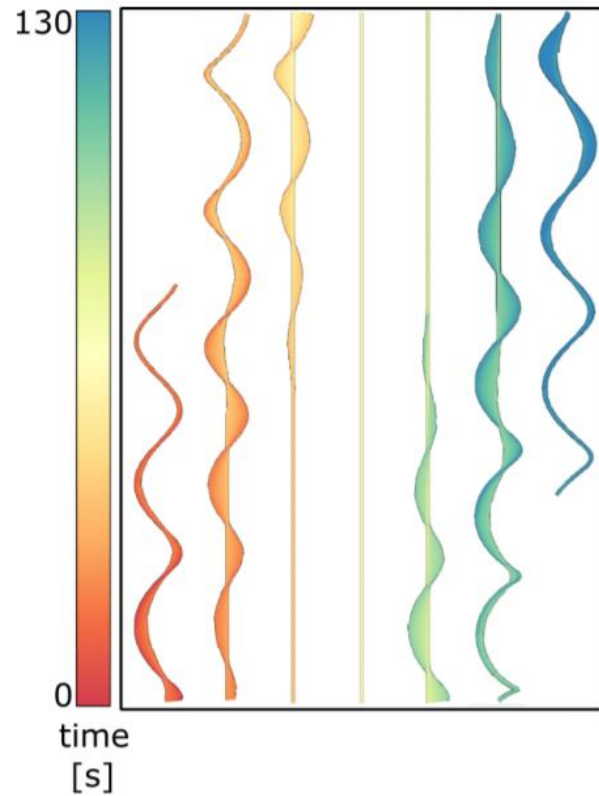
- 1. Time intervals coincide
- 2. Parts of time intervals overlap
- 3. Path lines occur at different points in time

Experiment

- Synthetic vortex
- Color scale - Time



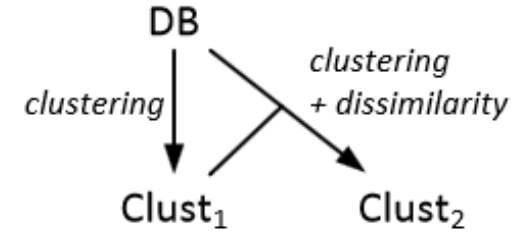
Experiment



Multiple Clustering

- Clustering results depend on
 - the algorithm used,
 - the parameters and
 - in case of stochastic algorithms on the initialization.
- Display of a single clustering result not ideal given the uncertainty about the right solution.
- Often, there is not even one right solution!
- Better approach: computation and visualization of a set of clustering results (alternative clusterings).
- Naive approach: sample parameter space and stochastic behavior.
 - While this supports a cluster stability analysis, it leads to many similar clusters and does not support an analyst well.

Multiple Clustering



- Strategies aiming at quality and diversity:
- Compute a first clustering C_1 and compute a second clustering C_2 that fulfils two properties:
 - The clustering quality is high.
 - The difference between C_2 and C_1 is high.
- Extension: compute further clusterings C_{i+1} with a high quality and a high distance to all C_i .
- Compute multiple clusters simultaneously (aiming at high distances to previous solutions)
- Example: COALA (Bae, 2006)

Applications

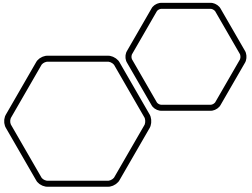
- Many applications, e.g. in finance data, security analysis, and medicine.
- Document clustering is growing (Steinbach, 2000).
 - K-means and hierarchical clustering are employed.
 - Textmining and ontologies are integrated (Hotho, 2003).

Summary

- Clustering for explorative data analysis
- A variety of clustering methods (different cluster models).
- Multiple clusterings provide alternative (and valid) perspectives on the data.
 - Visual analysis to understand the result and the influence of parameters
- Clustering is evaluated by means of quality measures (silhouette coefficient, ...) and expert feedback based on appropriate visualizations
- Cluster quality has many aspects – too many to be sufficiently represented by one scalar quality value

Summary and Outlook

- Global clustering restricted to 10-15 dimensions (Hund, 2016)
- For higher-dimensional data: search for clusterable subspaces and cluster within them (next lecture)
- Temporal clustering as a current research topic
 - Clusters arise, disappear, merge, change (in size), ...



Questions???